

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



HCM-UTE

LUẬN ÁN TỐT NGHIỆP

**THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG ĐIỀU
KHIỂN THEO VÙNG TRÊN Ô TÔ DỰA TRÊN GIAO
THỨC SOME/IP VÀ MẠNG ETHERNET**

NGÀNH HỆ THỐNG NHÚNG & IOT

SVTH: **TRƯƠNG LÝ MINH HOÀNG**

MSSV: 22139025

GVHD: **ThS. TRƯƠNG NGỌC HÀ**

TP. HỒ CHÍ MINH – 06/2026

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN - ĐIỆN TỬ
BỘ MÔN KỸ THUẬT MÁY TÍNH - VIỄN THÔNG



LUẬN ÁN TỐT NGHIỆP

**THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG ĐIỀU
KHIỂN THEO VÙNG TRÊN Ô TÔ DỰA TRÊN GIAO
THỨC SOME/IP VÀ MẠNG ETHERNET**

NGÀNH HỆ THỐNG NHÚNG & IOT

SVTH: **TRƯƠNG LÝ MINH HOÀNG**

MSSV: 22139025

GVHD: **ThS. TRƯƠNG NGỌC HÀ**

TP. HỒ CHÍ MINH – 06/2026



HCM-UTE

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
Ho Chi Minh City University of Technology and Engineering

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM

Độc lập – Tự do – Hạnh phúc

---***---

Thủ Đức, ngày 26 tháng 06 năm 2026

NHIỆM VỤ ĐỒ ÁN TỐT NGHIỆP

Họ và tên sinh viên :

Trương Lý Minh Hoàng

MSSV: 22139025

Lớp: 22139C

Ngành: Hệ thống nhúng & IoT

Giảng viên hướng dẫn: ThS. Trương Ngọc Hà

Ngày nhận đề tài: 11/03/2026

Ngày nộp đề tài: 26/06/2026

1. Tên đề tài: Thiết kế và triển khai hệ thống điều khiển theo vùng trên ô tô dựa trên giao thức SOME/IP và mạng Ethernet.

2. Các số liệu, tài liệu ban đầu:

- Kiến thức về hệ thống nhúng, vi điều khiển và hệ điều hành thời gian thực.
- Kiến thức về mạng truyền thông trên ô tô, Ethernet và giao thức SOME/IP.
- Các tài liệu tham khảo về kiến trúc vùng, kiến trúc hướng dịch vụ và xe định nghĩa bằng phần mềm.

3. Nội dung thực hiện đề tài:

- Nghiên cứu kiến trúc vùng và giao thức SOME/IP.
- Lựa chọn linh kiện và thiết kế mô hình hệ thống.
- Vẽ sơ đồ khối, sơ đồ nguyên lý và sơ đồ kết nối.
- Lập trình cho hệ thống.
- Triển khai mô hình hoàn chỉnh.
- Viết báo cáo.
- Bảo vệ đồ án tốt nghiệp.

4. Sản phẩm: Mô hình hệ thống điều khiển theo vùng sử dụng mạng Ethernet.

TRƯỜNG NGÀNH
(Ký & ghi rõ họ tên)

GIẢNG VIÊN HƯỚNG DẪN
(Ký & ghi rõ họ tên)

BẢN NHẬN XÉT KHÓA LUẬN TỐT NGHIỆP

(Dành cho giảng viên hướng dẫn)

Đề tài: Thiết kế và triển khai hệ thống điều khiển theo vùng
thần ô tô dựa trên giao thức SOME/IP và mạng Ethernet.

Sinh viên: + Trường Lý Ninh Hoàng MSSV: 020139025

+ MSSV:

Hướng dẫn: TR.S. Trường Ngọc Hà

Nhận xét bao gồm các nội dung sau đây:

1. Tính hợp lý trong cách đặt vấn đề và giải quyết vấn đề; ý nghĩa khoa học và thực tiễn:

Đặt vấn đề rõ ràng, mục tiêu cụ thể; đề tài có tính mới, cấp thiết; đề tài có khả năng ứng dụng, tính sáng tạo.

Phù hợp

2. Phương pháp thực hiện/ phân tích/ thiết kế:

Phương pháp hợp lý và tin cậy dựa trên cơ sở lý thuyết; có phân tích và đánh giá phù hợp; có tính mới và tính sáng tạo.

Phù hợp

3. Kết quả thực hiện/ phân tích và đánh giá kết quả/ kiểm định thiết kế:

Phù hợp với mục tiêu đề tài; phân tích và đánh giá/ kiểm thử thiết kế hợp lý; có tính sáng tạo/ kiểm định chặt chẽ và đảm bảo độ tin cậy.

Phù hợp

4. Kết luận và đề xuất:

Kết luận phù hợp với cách đặt vấn đề, đề xuất mang tính cải tiến và thực tiễn; kết luận có đóng góp mới mẻ, đề xuất sáng tạo và thuyết phục.

Phù hợp

5. Hình thức trình bày và bố cục báo cáo:

Văn phong nhất quán, bố cục hợp lý, cấu trúc rõ ràng, đúng định dạng mẫu; có tính hấp dẫn, thể hiện năng lực tốt, văn bản trau chuốt.

Phù hợp

6. Kỹ năng chuyên nghiệp và tính sáng tạo:

Thể hiện các kỹ năng giao tiếp, kỹ năng làm việc nhóm, sử dụng kỹ năng chuyên nghiệp khác trong việc thực hiện đề tài.

.....

7. Tài liệu trích dẫn

Tính trung thực trong việc trích dẫn tài liệu tham khảo; tính phù hợp của các tài liệu trích dẫn; trích dẫn theo đúng chỉ dẫn APA.

Có trích dẫn

8. Đánh giá về sự trùng lặp của đề tài

Cần không định đề tài có trùng lặp hay không? Nếu có, đề nghị ghi rõ mức độ, tên đề tài, nơi công bố, năm công bố của đề tài đã công bố.

Chưa phát hiện

9. Những nhược điểm và thiếu sót, những điểm cần được bổ sung và chỉnh sửa*

.....

.....

10. Nhận xét tinh thần, thái độ học tập, nghiên cứu của sinh viên

.....

Đề nghị của giảng viên hướng dẫn

Ghi rõ: "Bảo vệ đơn/ không đạt yêu cầu của một khóa luận tốt nghiệp kỹ sư và được phép/ không được phép bảo vệ khóa luận tốt nghiệp"

Bảo vệ đạt yêu cầu, được phép bảo vệ

Tp. HCM, ngày 26 tháng 06 năm 2026

Người nhận xét

(Ký và ghi rõ họ tên)

Thùy Ngọc Hà

LỜI CẢM ƠN

Trong suốt quá trình học tập tại Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh, chúng tôi đã nhận được sự dạy dỗ tận tình và những kiến thức quý báu từ quý Thầy Cô. Đây là cơ sở quan trọng giúp chúng tôi hoàn thành đồ án tốt nghiệp này.

Đặc biệt, chúng tôi xin bày tỏ lòng biết ơn sâu sắc đến ThS. Trương Ngọc Hà, người đã trực tiếp hướng dẫn, định hướng nghiên cứu và hỗ trợ chúng tôi trong suốt quá trình thực hiện đề tài. Những góp ý chuyên môn, sự tận tình chỉ bảo cùng những kinh nghiệm thực tiễn mà Thầy chia sẻ đã giúp chúng tôi từng bước hoàn thiện nội dung nghiên cứu cũng như vượt qua những khó khăn trong quá trình triển khai hệ thống.

Bên cạnh đó, chúng tôi xin gửi lời cảm ơn sâu sắc đến quý Thầy Cô Khoa Điện – Điện tử, Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh, những người đã trang bị cho chúng tôi kiến thức chuyên môn và kỹ năng cần thiết trong suốt quá trình học tập. Những kiến thức đó là hành trang quý báu giúp chúng tôi có đủ nền tảng để thực hiện và hoàn thành đề tài.

Mặc dù đã nỗ lực hết sức để hoàn thành đề tài, tuy nhiên do hạn chế về thời gian và kinh nghiệm thực tế nên đồ án khó tránh khỏi những thiếu sót nhất định. Chúng tôi rất mong nhận được những ý kiến đóng góp quý báu từ quý Thầy Cô và bạn bè để đề tài được hoàn thiện hơn.

Sinh viên thực hiện đồ án tốt nghiệp
(Ký và ghi rõ họ tên)

Trương Lý Minh Hoàng

LỜI CAM ĐOAN

Tôi là **Trương Lý Minh Hoàng**, mã số sinh viên 22139025, sinh viên ngành Hệ thống nhúng và IoT, Khoa Điện – Điện tử, Trường Đại học Công nghệ Kỹ thuật Thành phố Hồ Chí Minh. Tôi xin cam đoan đề tài này được là kết quả của quá trình nghiên cứu và thực hiện của bản thân tôi.

Tôi cam đoan không sao chép nội dung, kết quả từ bất kỳ đồ án, luận văn hoặc công trình nghiên cứu nào khác mà không có trích dẫn rõ ràng. Mọi tài liệu tham khảo được sử dụng trong đồ án đều đã được trích dẫn và liệt kê đầy đủ trong danh mục tài liệu tham khảo của đồ án theo đúng quy định.

Sinh viên thực hiện đồ án tốt nghiệp
(Ký và ghi rõ họ tên)

Trương Lý Minh Hoàng

TÓM TẮT

Đồ án tập trung nghiên cứu và triển khai mô hình điều khiển theo vùng trên ô tô dựa trên giao thức SOME/IP và mạng Ethernet, hướng đến các kiến trúc điện - điện tử hiện đại trong xe được định nghĩa bằng phần mềm (Software Defined Vehicle - SDV). Hệ thống được xây dựng theo mô hình kiến trúc vùng (Zonal Architecture), trong đó các bộ điều khiển vùng (ZECU) đảm nhiệm việc thu thập, xử lý dữ liệu từ cảm biến và điều khiển các cơ cấu chấp hành thông qua hạ tầng truyền thông Ethernet.

Về phần mềm, đồ án xây dựng nền tảng nhúng trên ESP-IDF và FreeRTOS với các tác vụ được tổ chức độc lập nhằm bảo đảm khả năng xử lý đồng thời và tính ổn định trong quá trình vận hành. Đồng thời, một ứng dụng giám sát được phát triển bằng PySide6 cho phép theo dõi dữ liệu thời gian thực, quản lý trạng thái hoạt động của các ZECU và thực hiện điều khiển từ giao diện người dùng.

Kết quả thực nghiệm cho thấy mô hình có khả năng truyền nhận dữ liệu ổn định trên mạng Ethernet, thực hiện trao đổi thông tin giữa các bộ điều khiển vùng thông qua giao thức SOME/IP và điều khiển cơ cấu chấp hành theo dữ liệu cảm biến thu thập được. Mô hình bước đầu cho thấy tính khả thi của việc ứng dụng kiến trúc vùng kết hợp với giao thức SOME/IP trong các hệ thống điện - điện tử trên ô tô, đồng thời tạo nền tảng cho các nghiên cứu tiếp theo về Ethernet ô tô, cập nhật phần mềm từ xa và kiến trúc xe được định nghĩa bằng phần mềm.

MỤC LỤC

LỜI CẢM ƠN.....	I
LỜI CAM ĐOAN.....	II
TÓM TẮT.....	III
DANH MỤC HÌNH.....	VIII
DANH MỤC BẢNG.....	X
DANH MỤC CÁC TỪ VIẾT TẮT.....	XI
CHƯƠNG 1 TỔNG QUAN.....	1
1.1 GIỚI THIỆU.....	1
1.2 TÌNH HÌNH NGHIÊN CỨU.....	2
1.3 PHƯƠNG PHÁP NGHIÊN CỨU.....	4
1.4 BỐ CỤC ĐỒ ÁN.....	5
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	7
2.1 TỔNG QUAN LÝ THUYẾT.....	7
2.1.1 Kiến trúc theo miền.....	7
2.1.2 Kiến trúc theo vùng.....	9
2.1.3 Tổng quan các giao thức truyền thông trên ô tô.....	11
2.1.4 Xe được định nghĩa bằng phần mềm.....	12
2.1.5 Kiến trúc hướng dịch vụ.....	13
2.1.6 Giới thiệu CMake.....	13
2.1.7 Giới thiệu platform ESP-IDF.....	14
2.1.8 Giới thiệu FreeRTOS.....	14
2.1.9 Giới thiệu pySide6.....	14
2.2 MỘT SỐ LINH KIỆN VÀ DỊCH VỤ TRONG HỆ THỐNG.....	15
2.2.1 Vi điều khiển ESP32-WROOM-32.....	15
2.2.2 Thiết bị chuyển mạch mạng (Network Switch).....	16

2.2.3	Dịch vụ giao tiếp Ethernet W5500	17
2.2.4	Cảm biến khoảng cách siêu âm HC-SR04	20
2.2.5	Mạch cầu H.....	20
2.2.6	Mạch hạ áp DC-DC	22
2.3	CÁC CHUẨN GIAO TIẾP VÀ GIAO THỨC TRUYỀN DỮ LIỆU	23
2.3.1	Chuẩn giao tiếp SPI.....	23
2.3.2	Giao thức truyền thông UDP	25
2.3.3	Giao thức Ethernet trên ô tô	26
2.3.4	Khung truyền Ethernet	28
2.3.5	Giao thức SOME/IP	29
2.3.6	Giao thức SOME/IP Discovery	30
	CHƯƠNG 3 THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG	32
3.1	YÊU CẦU VÀ SƠ ĐỒ KHỐI HỆ THỐNG.....	32
3.1.1	Sơ đồ khối hệ thống và chức năng của từng khối	33
3.1.1.1	Khối điều khiển vùng F-ZECU và B-ZECU	33
3.1.1.2	Khối cảm biến F-SD và B-SD	33
3.1.1.3	Khối động cơ F-DC và B-DC.....	34
3.1.1.4	Khối chuyển mạch trung tâm CW	34
3.1.1.5	Khối giám sát và điều khiển MAC	34
3.1.1.6	Khối nguồn PS.....	34
3.2	THIẾT KẾ PHẦN CỨNG	34
3.2.1	Khối điều khiển vùng	34
3.2.2	Khối cảm biến.....	36
3.2.2.1	Thiết kế dùng chung ngõ vào TRIGGER.....	36
3.2.2.2	Nguyên lý đo đúng cách.....	36
3.2.3	Khối động cơ	37
3.2.4	Khối nguồn	39
3.3	THIẾT KẾ PHẦN MỀM	41
3.3.1	Sơ đồ phân tầng kiến trúc phần mềm	41
3.3.2	Bộ khởi tạo dịch vụ	42
3.3.3	Dịch vụ giao tiếp mạng.....	43

3.3.3.1	Thiết kế tầng giao tiếp phần cứng	43
3.3.3.2	Cơ chế xử lý ngắt và quản lý hàng đợi gói tin	45
3.3.3.3	Cơ chế gửi và nhận gói tin Ethernet	47
3.3.4	Dịch vụ điều khiển động cơ.....	48
3.3.5	Dịch vụ giám sát hệ thống	50
3.3.5.1	Giám sát trạng thái cảm biến.....	50
3.3.5.2	Quản lý trạng thái cảm biến	51
3.3.5.3	Khẩn cấp.....	51
3.3.5.4	Điều khiển động cơ	51
3.3.5.5	Ghép nối CCU	52
3.3.6	Cấu trúc phân bổ các dịch vụ	53
3.3.7	Cơ chế đồng bộ và giao tiếp giữa các dịch vụ.....	54
3.3.7.1	Cơ chế hàng đợi (Queue/Message)	54
3.3.7.2	Cơ chế khóa (Mutex).....	55
3.3.7.3	Cơ chế ngủ/đánh thức (Sleep/Wakeup).....	56
3.3.8	Thiết kế gói tin tầng ứng dụng	56
3.3.8.1	Thiết kế phần mở đầu ZECU Header	57
3.3.8.2	Thiết kế phần thân Body Payload.....	58
3.3.8.3	Thiết kế phần cuối ZECU Trailer	63
3.3.9	Thiết kế ứng dụng điều khiển.....	64
3.3.9.1	Giao diện chính.....	64
3.3.9.2	Giao diện ghép nối ZECU	65
CHƯƠNG 4 THI CÔNG HỆ THỐNG		66
4.1	THI CÔNG KHỐI NGUỒN VÀ ĐỘNG CƠ.....	66
4.2	THI CÔNG MÔ HÌNH TỔNG THỂ.....	67
CHƯƠNG 5 KẾT QUẢ VÀ ĐÁNH GIÁ HỆ THỐNG.....		68
5.1	KẾT QUẢ KHỐI CẢM BIẾN.....	68
5.2	KẾT QUẢ KHỐI ĐỘNG CƠ	68
5.3	KẾT QUẢ ỨNG DỤNG GIÁM SÁT	69
5.3.1	Tổng quan.....	69

5.3.2	Quá trình ghép nối	70
5.4	THỜI GIAN ĐÁP ỨNG CỦA HỆ THỐNG	72
CHƯƠNG 6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		74
6.1	KẾT LUẬN	74
6.2	HƯỚNG PHÁT TRIỂN.....	75
TÀI LIỆU THAM KHẢO		77

DANH MỤC HÌNH

Hình 2.1 Kiến trúc hệ thống điện ô tô phân tách theo miền chức năng.....	8
Hình 2.2 Giai đoạn chuyển đổi của kiến trúc điện - điện tử (E/E) trên ô tô.	10
Hình 2.3 Chuyển đổi E/E sang Software-Defined Vehicle (SDV).....	13
Hình 2.4 Sơ đồ chân bo mạch phát triển sử dụng ESP-WROOM-32.....	15
Hình 2.5 Router Mercusys MW305R.	17
Hình 2.6 Sơ đồ giao diện của W5500.	18
Hình 2.7 Cảm biến siêu âm HC-SR04.....	20
Hình 2.8 Mô-đun mạch cầu H sử dụng IC L298N.	21
Hình 2.9 Mô-đun mạch hạ áp DC-DC sử dụng IC LM2596S-ADJ.	22
Hình 2.10 Thiết kế khối SPI master và slave.....	24
Hình 2.11 Khung truyền IPv4 và UDP.	25
Hình 2.12 Cấu trúc nút Ethernet.	26
Hình 2.13 Các loại cáp / môi trường truyền Ethernet.....	27
Hình 2.14 Khung truyền Ethernet theo chuẩn IEEE 802.3.....	28
Hình 2.15 Khung truyền được gắn thẻ trong mạng cục bộ ảo.....	29
Hình 2.16 Khung truyền của SOME/IP.	29
Hình 2.17 Khung truyền quảng bá của SOME/IP.	30
Hình 3.1 Sơ đồ khối toàn hệ thống.	33
Hình 3.2 Sơ đồ nguyên lý khối điều khiển vùng Zone ECU.	35
Hình 3.3 Sơ đồ nguyên lý cảm biến F-SD, B-SD.....	36
Hình 3.4 Sơ đồ nguyên lý khối động cơ.	38
Hình 3.5 Mạch điện pin 4S - BMS.	39
Hình 3.6 Sơ đồ nguyên lý khối nguồn PS.....	40
Hình 3.7 Cấu trúc phân tầng của firmware cho ZECU.....	41
Hình 3.8 Lưu đồ quá trình khởi tạo các mô-đun.....	43
Hình 3.9 Quá trình định danh mô-đun giao tiếp Ethernet.	44
Hình 3.10 Quá trình định danh mô-đun giao tiếp Ethernet.	45
Hình 3.11 Quá trình xử lý gói tin.....	46
Hình 3.12 Hàng đợi gửi và nhận gói tin.....	46

Hình 3.13 Sơ đồ chuỗi gửi và nhận dữ liệu sử dụng cơ chế ngủ/đánh thức.	47
Hình 3.14 Lưu đồ gửi và nhận dữ liệu sử dụng cơ chế ngủ/đánh thức.	48
Hình 3.15 Quá trình thiết lập tốc độ động cơ.	50
Hình 3.16 Sơ đồ cơ chế giao tiếp hàng đợi giữa hai dịch vụ.	54
Hình 3.17 Lưu đồ cơ chế truy cập tài nguyên chung có khóa.	55
Hình 3.18 Sơ đồ cấu trúc phân tầng tổng quát của một Generic ZECU.	57
Hình 3.19 Sơ đồ cấu trúc phân bố bit của ZECU Frame Header.	57
Hình 3.20 Gói tin quảng bá dịch vụ.	59
Hình 3.21 Phần thân gói tin đồng bộ trạng thái.	60
Hình 3.22 Gói tin điều khiển động cơ.	61
Hình 3.23 Phần thân gói tin xác thực bảo mật - AuthTX / AuthRX.	62
Hình 3.24 Gói tin giữ kết nối.	62
Hình 3.25 Phần kết thúc chung của gói tin - ZECU Trailer.	63
Hình 3.26 Giao diện tổng thể của ứng dụng điều khiển.	64
Hình 3.27 Giao diện ghép nối ZECU.	65
Hình 4.1 Tầng dưới cùng của mô hình.	66
Hình 4.2 Mô hình xe hoàn chỉnh.	67
Hình 5.1 Dạng sóng trên đường truyền giao tiếp với khối cảm biến.	68
Hình 5.2 Dạng sóng ngõ ra bộ điều chế độ rộng xung điều khiển mạch cầu H.	69
Hình 5.3 Giao diện chính với đồ thị.	70
Hình 5.4 Giao diện chính với bản đồ.	70
Hình 5.5 Giao diện kết nối ZECU của ứng dụng giám sát.	71
Hình 5.6 Gói tin quảng bá của Back-ZECU.	71
Hình 5.7 Quá trình ghép nối F-ZECU.	72
Hình 5.8 Tần suất thời gian phản hồi.	73
Hình 5.9 Tần suất các chu kỳ gửi gói tin đồng bộ.	73

DANH MỤC BẢNG

Bảng 2.1 So sánh các giao thức truyền thông trên ô tô.	11
Bảng 2.2 Thông số kỹ thuật Dịch vụ ESP-WROOM-32.....	16
Bảng 2.3 Bảng thông số kỹ thuật W5500.	18
Bảng 2.4 Các thanh ghi chính trong Dịch vụ W5500.....	18
Bảng 2.5 Các thanh ghi Socket.	19
Bảng 2.6 Bảng thông số kỹ thuật mô-đun mạch cầu H.	21
Bảng 2.7 Bảng thông số kỹ thuật của mô-đun hạ áp LM2596S-ADJ.	23
Bảng 2.8 Danh sách các trường cần quan trọng của IPv4 và UDP.	26
Bảng 3.1 Chức năng của các thành phần trong khối Zone ECU.	35
Bảng 3.2 Trạng thái điều khiển chiều quay động cơ.	38
Bảng 3.3 Thông số nguồn cấp của các thiết bị trong hệ thống.	40
Bảng 3.4 Quá trình khởi tạo mô-đun giao tiếp Ethernet.....	43
Bảng 3.5 Các trường giá trị của AppService_t.	53
Bảng 3.6 Mức độ ưu tiên của dịch vụ.	53
Bảng 4.1 Chú thích các thành phần tầng dưới cùng của xe.	66

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Định nghĩa
ADAS	Advanced Driver Assistance Systems
API	Application Programming Interface
AUTOSAR	AUTomotive Open System ARchitecture
BMS	Battery Management System
CAN	Controller Area Network
CCU	Central Control Unit
DC	Direct Current
ECU	Electronic Control Unit
ESP-IDF	Espressif IoT Development Framework
GPIO	General Purpose Input/Output
HC-SR04	Ultrasonic Distance Sensor Module
HPC	High Performance Computing
IP	Internet Protocol
LAN	Local Area Network
MAC	Media Access Control
MCU	Microcontroller Unit
PCB	Printed Circuit Board
PHY	Physical Layer
RPC	Remote Procedure Call
RX	Receive
SD	Service Discovery
SDV	Software Defined Vehicle
SOA	Service-Oriented Architecture
SOME/IP	Scalable service-Oriented Middleware over IP
SOTA	Software Over-The-Air

SPI	Serial Peripheral Interface
TCP	Transmission Control Protocol
TSN	Time-Sensitive Networking
TX	Transmit
UDP	User Datagram Protocol
UI	User Interface
W5500	Ethernet Controller
ZA	Zonal Architecture
ZECU	Zonal Electronic Control Unit

CHƯƠNG 1

TỔNG QUAN

1.1 GIỚI THIỆU

Trong những năm gần đây, ngành công nghiệp ô tô đang trải qua quá trình chuyển đổi mạnh mẽ của sự phát triển của các công nghệ số, các phương tiện hiện đại không còn đơn thuần là phương tiện vận chuyển mà đang dần trở thành các hệ thống điện tử và phần mềm phức tạp với khả năng kết nối, tự động hóa và cập nhật chức năng trong suốt vòng đời sử dụng. Khái niệm xe được định nghĩa bằng phần mềm (Software Defined Vehicle - SDV) đã trở thành xu hướng phát triển chủ đạo của ngành công nghiệp ô tô, trong đó phần mềm đóng vai trò trung tâm trong việc quản lý, điều khiển và nâng cấp các chức năng của xe. Điều này đặt ra yêu cầu ngày càng cao đối với kiến trúc điện - điện tử và hạ tầng truyền thông bên trong phương tiện nhằm đáp ứng khả năng xử lý khối lượng dữ liệu lớn, hỗ trợ các tính năng thông minh và đảm bảo khả năng mở rộng trong tương lai.

Trong các hệ thống ô tô truyền thống, kiến trúc điều khiển thường được tổ chức theo mô hình phân tán hoặc theo miền chức năng với nhiều ECU hoạt động độc lập và được kết nối thông qua các mạng truyền thông như CAN hoặc LIN. Tuy nhiên, khi số lượng cảm biến, cơ cấu chấp hành và các tính năng hỗ trợ lái xe ngày càng gia tăng, kiến trúc này bắt đầu bộc lộ những hạn chế về khả năng mở rộng, độ phức tạp của hệ thống dây dẫn cũng như hiệu quả quản lý tài nguyên. Để giải quyết những vấn đề đó, kiến trúc vùng (Zonal Architecture) đã được đề xuất như một hướng phát triển mới cho các hệ thống điện - điện tử trên ô tô. Thay vì phân chia theo chức năng, các thiết bị được tổ chức theo vị trí vật lý trên xe và được quản lý bởi các bộ điều khiển vùng. Dữ liệu từ các cảm biến sẽ được thu thập và

xử lý tại vùng trước khi truyền đến bộ điều khiển trung tâm thông qua mạng Ethernet tốc độ cao, giúp giảm tải cho hệ thống truyền thông, đơn giản hóa cấu trúc dây dẫn và nâng cao khả năng mở rộng của toàn bộ hệ thống.

Bên cạnh sự thay đổi về kiến trúc phần cứng, các giải pháp phần mềm cũng đang được phát triển theo hướng mô-đun hóa nhằm tăng tính linh hoạt và khả năng tái sử dụng. Trong đó, kiến trúc hướng dịch vụ (Service-Oriented Architecture - SOA) được xem là một trong những nền tảng quan trọng của các hệ thống SDV hiện đại. Giao thức SOME/IP được phát triển để hiện thực hóa mô hình SOA trên nền tảng mạng Ethernet, cho phép các bộ điều khiển giao tiếp với nhau thông qua các dịch vụ thay vì phụ thuộc trực tiếp vào từng ứng dụng cụ thể. Việc kết hợp kiến trúc vùng, mạng Ethernet và giao thức SOME/IP không chỉ giúp nâng cao hiệu quả truyền thông mà còn tạo điều kiện thuận lợi cho việc cập nhật phần mềm từ xa, tích hợp các chức năng mới và phát triển các ứng dụng thông minh trong tương lai. Xuất phát từ những yêu cầu thực tiễn đó, đề tài “Thiết kế và triển khai hệ thống điều khiển phân tán theo vùng trên ô tô sử dụng giao thức hướng dịch vụ qua kết nối Ethernet” được thực hiện nhằm nghiên cứu, xây dựng và đánh giá một mô hình thực nghiệm dựa trên các công nghệ đang được áp dụng trong ngành công nghiệp ô tô hiện đại.

1.2 TÌNH HÌNH NGHIÊN CỨU

Sự phát triển mạnh mẽ của các công nghệ xe điện, xe kết nối và xe tự hành đã tạo ra những thay đổi lớn trong kiến trúc điện – điện tử trên ô tô hiện đại. Các phương tiện thế hệ mới ngày càng được tích hợp nhiều cảm biến, bộ điều khiển điện tử (ECU) và các chức năng phần mềm phức tạp nhằm nâng cao khả năng hỗ trợ người lái, tối ưu hóa hiệu suất vận hành và cải thiện trải nghiệm người dùng. Tuy nhiên, việc gia tăng số lượng thiết bị điện tử cũng làm phát sinh nhiều thách thức liên quan đến khối lượng hệ thống dây dẫn, khả năng truyền tải dữ liệu và hiệu quả quản lý tài nguyên tính toán trên xe. Trong bối cảnh đó, kiến trúc vùng (Zonal Architecture) kết hợp với kiến trúc hướng dịch vụ (Service-Oriented Architecture - SOA) đang được xem là xu hướng phát triển tất yếu của ngành công nghiệp ô tô [1]. Việc ứng dụng mạng Ethernet tốc độ cao cùng giao thức SOME/IP

cho phép xây dựng một nền tảng truyền thông linh hoạt, có khả năng đáp ứng yêu cầu trao đổi dữ liệu ngày càng lớn giữa các thành phần trong hệ thống. Chính vì vậy, các nghiên cứu liên quan đến kiến trúc vùng, Ethernet ô tô và giao thức SOME/IP đang nhận được sự quan tâm đặc biệt từ các hãng xe, các tổ chức nghiên cứu và cộng đồng học thuật trên toàn thế giới.

Trong nước: Trong những năm gần đây, cùng với sự phát triển của lĩnh vực hệ thống nhúng và công nghệ ô tô thông minh, nhiều trường đại học và viện nghiên cứu tại Việt Nam đã bắt đầu tiếp cận các công nghệ liên quan đến mạng truyền thông trên ô tô, hệ thống điều khiển điện tử và kiến trúc phần mềm hiện đại. Các nghiên cứu chủ yếu tập trung vào việc tìm hiểu nguyên lý hoạt động của các giao thức truyền thông như CAN, LIN, FlexRay và Automotive Ethernet nhằm đáp ứng nhu cầu đào tạo và nghiên cứu trong lĩnh vực ô tô thế hệ mới. Bên cạnh đó, một số công trình đã nghiên cứu việc ứng dụng kiến trúc hướng dịch vụ trong hệ thống nhúng, xây dựng các mô hình truyền thông dựa trên giao thức TCP/IP hoặc UDP/IP để phục vụ việc trao đổi dữ liệu giữa các thiết bị điều khiển phân tán. Một số đề tài cũng đã triển khai thử nghiệm giao thức SOME/IP trên các nền tảng phần cứng nhúng như Raspberry Pi hoặc vi điều khiển STM32 nhằm đánh giá khả năng truyền thông thời gian thực và tính linh hoạt của mô hình hướng dịch vụ [2]. Tuy nhiên, phần lớn các nghiên cứu hiện nay vẫn dừng lại ở mức mô phỏng hoặc triển khai trên quy mô nhỏ, chưa xây dựng được một hệ thống thực nghiệm hoàn chỉnh kết hợp giữa kiến trúc vùng, giao thức SOME/IP, mạng Ethernet và cơ chế cập nhật phần mềm từ xa. Do đó, việc nghiên cứu và xây dựng một mô hình điều khiển phân tán theo vùng có khả năng hoạt động thực tế sẽ góp phần bổ sung cơ sở nghiên cứu cũng như tạo tiền đề cho các hướng phát triển chuyên sâu hơn trong lĩnh vực ô tô thông minh tại Việt Nam.

Ngoài nước: Trên thế giới, đặc biệt tại các quốc gia có ngành công nghiệp ô tô phát triển như Đức, Mỹ, Nhật Bản và Hàn Quốc, các nghiên cứu về kiến trúc vùng đã được triển khai mạnh mẽ trong những năm gần đây nhằm phục vụ quá trình chuyển đổi sang mô hình xe được định nghĩa bằng phần mềm (Software Defined Vehicle - SDV). Nhiều hãng xe lớn như Tesla, Mercedes-Benz, BMW,

Volkswagen và Hyundai đã công bố chiến lược chuyển đổi từ kiến trúc miền truyền thống sang kiến trúc vùng nhằm giảm số lượng ECU, đơn giản hóa hệ thống dây dẫn và nâng cao hiệu quả quản lý tài nguyên tính toán [3]. Song song với đó, các tổ chức tiêu chuẩn hóa như AUTOSAR cũng đã phát triển các nền tảng phần mềm hỗ trợ triển khai kiến trúc hướng dịch vụ trên mạng Ethernet thông qua giao thức SOME/IP [4]. Nhiều công trình nghiên cứu đã đánh giá hiệu năng của SOME/IP trong các hệ thống điều khiển phân tán và cho thấy giao thức này có khả năng đáp ứng tốt các yêu cầu về độ trễ thấp, khả năng mở rộng và tính tương thích với các hệ thống thời gian thực. Ngoài ra, các nghiên cứu còn tập trung vào việc xây dựng các bộ điều khiển trung tâm hiệu năng cao kết hợp với các bộ điều khiển vùng nhằm hình thành kiến trúc tính toán tập trung trên xe. Một số mô hình thử nghiệm đã chứng minh khả năng giảm đáng kể khối lượng dây dẫn, tăng hiệu quả truyền thông và hỗ trợ cập nhật phần mềm từ xa thông qua hạ tầng mạng Ethernet [5]. Các kết quả nghiên cứu này đang đóng vai trò quan trọng trong việc định hình thể hệ phương tiện thông minh tiếp theo, nơi phần mềm trở thành yếu tố cốt lõi quyết định chức năng và khả năng cạnh tranh của sản phẩm.

Từ việc khảo sát tình hình nghiên cứu trong và ngoài nước có thể nhận thấy rằng kiến trúc vùng kết hợp với giao thức SOME/IP trên nền tảng Ethernet đang là một trong những hướng nghiên cứu trọng tâm của ngành công nghiệp ô tô hiện đại. Mặc dù đã có nhiều nghiên cứu và ứng dụng thực tiễn trên thế giới, việc xây dựng các mô hình thực nghiệm hoàn chỉnh phục vụ mục đích nghiên cứu và đào tạo vẫn còn hạn chế tại Việt Nam. Do đó, đề tài tập trung thiết kế và triển khai hệ thống điều khiển phân tán theo vùng sử dụng giao thức SOME/IP không chỉ có ý nghĩa về mặt học thuật mà còn góp phần tạo ra nền tảng thử nghiệm phục vụ nghiên cứu các công nghệ SDV, Ethernet ô tô và kiến trúc hướng dịch vụ trong tương lai.

1.3 PHƯƠNG PHÁP NGHIÊN CỨU

Để đạt được mục tiêu của đề tài, nhóm thực hiện sử dụng kết hợp các phương pháp nghiên cứu lý thuyết và thực nghiệm. Việc kết hợp các phương pháp này giúp xây dựng cơ sở khoa học cho hệ thống, đồng thời đánh giá được khả năng triển khai và ứng dụng trong thực tế.

Phương pháp nghiên cứu lý thuyết: Thu thập, nghiên cứu và tổng hợp các tài liệu, bài báo khoa học và tiêu chuẩn kỹ thuật liên quan đến kiến trúc vùng (Zonal Architecture), giao thức SOME/IP, mạng Ethernet ô tô và mô hình xe được định nghĩa bằng phần mềm (SDV). Từ đó làm cơ sở cho việc thiết kế và xây dựng hệ thống.

Phương pháp thiết kế và triển khai: Tiến hành thiết kế và xây dựng hệ thống gồm các bộ điều khiển vùng (ZECU) kết nối với nhau thông qua mạng Ethernet. Giao thức SOME/IP được triển khai để thực hiện truyền nhận dữ liệu và điều khiển giữa các nút mạng, đồng thời tạo nền tảng cho việc tích hợp bộ điều khiển trung tâm (CCU) trong tương lai.

Phương pháp thực nghiệm và đánh giá: Xây dựng các kịch bản thử nghiệm nhằm đánh giá khả năng truyền nhận dữ liệu, điều khiển cơ cấu chấp hành và cập nhật phần mềm từ xa. Các kết quả thu được được phân tích dựa trên các tiêu chí như độ ổn định, độ trễ truyền thông và khả năng mở rộng của hệ thống.

Tổng kết và phân tích kết quả: Dựa trên kết quả thực nghiệm, tiến hành đánh giá những ưu điểm, hạn chế của hệ thống và đề xuất các hướng phát triển nhằm nâng cao hiệu quả ứng dụng của kiến trúc vùng và giao thức SOME/IP trong các hệ thống ô tô hiện đại.

1.4 BỐ CỤC ĐỒ ÁN

Nội dung của đồ án được trình bày trong 06 chương, được sắp xếp theo trình tự từ cơ sở lý thuyết đến thiết kế, triển khai và đánh giá hệ thống, cụ thể như sau:

Chương 1: Tổng quan

Trình bày tính cấp thiết của đề tài, tình hình nghiên cứu, mục tiêu, đối tượng, phạm vi và phương pháp nghiên cứu.

Chương 2: Cơ sở lý thuyết

Giới thiệu các kiến thức nền tảng liên quan đến kiến trúc vùng, kiến trúc hướng dịch vụ, giao thức SOME/IP, mạng Ethernet và các công nghệ được sử dụng trong hệ thống.

Chương 3: Thiết kế hệ thống

Trình bày kiến trúc tổng thể của hệ thống, chức năng của các thành phần phần cứng và phần mềm cũng như thiết kế các khối chức năng chính.

Chương 4: Triển khai và xây dựng hệ thống

Trình bày quá trình xây dựng mô hình thực nghiệm, thiết kế phần cứng, phát triển phần mềm nhúng và ứng dụng giám sát.

Chương 5: Kết quả thực nghiệm và đánh giá

Trình bày các kết quả thử nghiệm, đánh giá khả năng truyền thông, điều khiển và hiệu quả hoạt động của hệ thống.

Chương 6: Kết luận và hướng phát triển

Tổng kết các kết quả đạt được, những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

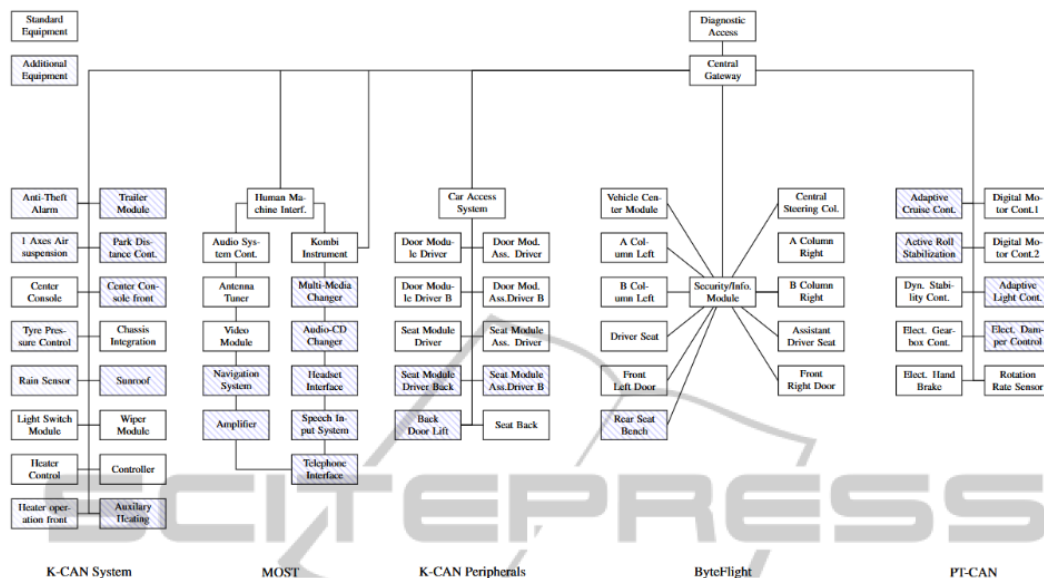
Phần này tập trung trình bày các khái niệm cơ bản, lý thuyết nền tảng, mô hình hoạt động và các nguyên lý cốt lõi liên quan đến hệ thống bãi giữ xe thông minh bao gồm cả phần cứng và phần mềm. Những nội dung này sẽ cung cấp cái nhìn toàn diện về các thành phần và công nghệ cần thiết, giúp định hình một cách rõ ràng và chi tiết về cách thức hoạt động và quản lý của hệ thống. Các nguyên lý khoa học được trình bày sẽ làm nền tảng vững chắc cho quá trình thiết kế, triển khai và tối ưu hóa hệ thống, nhằm đảm bảo hiệu quả, tính chính xác và độ bảo mật cao trong quá trình vận hành thực tế.

2.1 TỔNG QUAN LÝ THUYẾT

2.1.1 Kiến trúc theo miền

Kiến trúc theo miền (Domain Architecture) là bước tiên quan trọng trong hạ tầng điện tử ô tô, thay thế các hệ thống dây dẫn phẳng truyền thống. Đặc tính cốt lõi của kiến trúc này là tổ chức hệ thống dây điện và điều khiển dựa trên chức năng cụ thể thay vì vị trí vật lý. Mỗi nhóm chức năng chính sẽ được quản lý bởi một bộ điều khiển miền (Domain Control Unit - DCU) riêng biệt. Trong thực tế, các nhà sản xuất đang đối mặt với giới hạn của mô hình này khi số lượng ECU tăng lên từ 100 đến 150 đơn vị, gây ra sự bùng nổ về khối lượng dây dẫn và độ phức tạp trong lắp ráp thủ công. Một số ví dụ điển hình về các ECU quản lý theo miền bao gồm bộ điều khiển hệ thống truyền động (Powertrain), hệ thống an toàn (Safety systems) và hệ thống thông tin giải trí (Infotainment) [6]. Dù cải thiện tính linh

hoạt, kiến trúc này đang dần chuyển dịch sang cấu trúc vùng (Zonal) để tối ưu hóa hiệu suất.



Hình 2.1 Kiến trúc hệ thống điện ô tô phân tách theo miền chức năng.

Kiến trúc này tổ chức hệ thống điện - điện tử (E/E) dựa trên việc phân nhóm các đơn vị điều khiển điện tử (ECU) theo từng miền chức năng chuyên biệt, quản lý đồng bộ từ các thiết bị tiêu chuẩn (Standard Equipment) đến các tùy chọn bổ sung (Additional Equipment). Cụ thể:

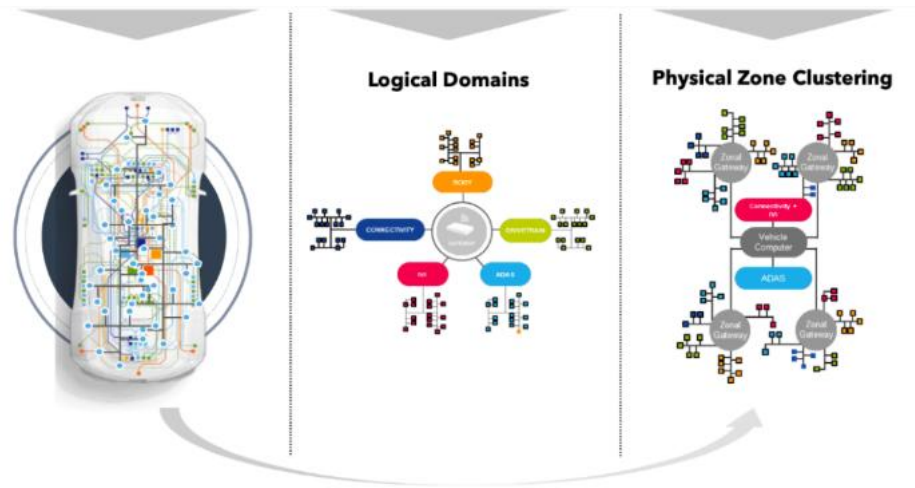
- Hệ thống K-CAN: Đảm nhiệm các chức năng tiện ích và thân xe, tiêu biểu như hệ thống chống trộm (Anti-Theft Alarm), cảm biến mưa (Rain Sensor) và Dịch vụ điều khiển sưởi (Heater Control).
- Mạng MOST: Tập trung vào các tính năng giải trí cao cấp và giao diện người - máy (HMI), bao gồm hệ thống định vị (Navigation System), bộ khuếch đại âm thanh (Amplifier) và giao diện kết nối điện thoại.
- Dịch vụ An ninh/Thông tin (Security/Info): Quản lý các thành phần liên quan đến an toàn thụ động và nội thất như hệ thống túi khí (A/B Column), điều khiển cửa và điều chỉnh vị trí ghế lái.
- Hệ thống PT-CAN (Powertrain CAN): Điều phối các chức năng vận hành cốt lõi của phương tiện như bộ điều khiển động cơ kỹ thuật số (Digital Motor Cont.), hệ thống phanh tay điện tử (Elect. Hand Brake) và kiểm soát ổn định động năng (Dyn. Stability Cont.).

Về ưu điểm, cấu trúc phân miền giúp Dịch vụ hóa các bài toán kỹ thuật thành những phần nhỏ độc lập, từ đó tối ưu hóa việc quản lý, rút ngắn khoảng cách dây dẫn đến các cảm biến/cơ cấu chấp hành và tăng tính linh hoạt khi triển khai các bản cập nhật phần mềm cục bộ. Tuy nhiên, hạn chế lớn nhất của kiến trúc này là sự bùng nổ về số lượng ECU dẫn đến độ phức tạp cực hạn của hệ thống dây dẫn (wire harness). Đồng thời, sự gia tăng lưu lượng dữ liệu giữa các miền tạo ra nguy cơ quá tải cho các đường bus truyền thông, đòi hỏi sự dịch chuyển sang các thể hệ kiến trúc tập trung hơn.

2.1.2 Kiến trúc theo vùng

Kiến trúc vùng (Zonal Architecture) là một bước ngoặt trong thiết kế mạng điện - điện tử (E/E) của xe hơi, giải quyết triệt để những hạn chế về độ phức tạp và trọng lượng dây dẫn của mô hình miền. Thay vì nhóm các đơn vị điều khiển điện tử (ECU) theo chức năng cốt lõi (như Powertrain hay Infotainment), kiến trúc này phân bổ các cảm biến, cơ cấu chấp hành và hệ thống điều khiển theo vị trí vật lý cục bộ trong xe (ví dụ: góc trước bên trái, góc sau bên phải).

Sự phân bổ dây dẫn trong kiến trúc vùng mang tính cách mạng: các thiết bị được kết nối trực tiếp với một "bộ điều khiển vùng" (Zonal Gateway) gần nhất, sau đó các cổng này giao tiếp với một máy tính trung tâm (High-Performance Computing Unit - HPCU) thông qua một số ít các liên kết tốc độ cao như Ethernet ô tô. Cách tiếp cận này giúp loại bỏ hệ thống dây dẫn điểm-điểm khổng lồ chạy dọc chiều dài xe. Về mặt ECU, kiến trúc vùng hợp nhất hàng chục ECU nhỏ lẻ thành một số ít các máy tính hiệu năng cao, cho phép quản lý tập trung và tái sử dụng hạ tầng phần mềm một cách linh hoạt, tạo tiền đề cho khái niệm xe điều khiển bằng phần mềm (Software-Defined Vehicles - SDV). Dù là một bước tiến lớn, nhiều nền tảng vùng trong tương lai gần vẫn sẽ kết hợp ("Domain Plus Zonalization") để tận dụng lại các chức năng miền mang tính chuyên môn cao, nơi vẫn cần khả năng xử lý cục bộ, giảm thiểu rủi ro khi chuyển đổi hoàn toàn sang điều khiển phần mềm trung tâm.



Hình 2.2 Giai đoạn chuyển đổi của kiến trúc điện - điện tử (E/E) trên ô tô.

Đặc Điểm Nổi Bật Của Kiến Trúc Vùng:

- Tập trung hóa phần mềm: Giao quyền quyết định từ các ECU phân tán cho một máy tính trung tâm duy nhất, cho phép nhà sản xuất xe (OEM) kiểm soát toàn bộ trải nghiệm phần mềm.
- Decentralization của phần cứng (Phi tập trung hóa công kết nối): Thiết lập các Zonal Gateway tại nhiều điểm trong xe để làm hub trung chuyển dữ liệu và phân phối nguồn điện cho thiết bị cục bộ, sau đó báo cáo về máy tính trung tâm.
- Sử dụng mạng Ethernet ô tô: Triển khai các chuẩn như 10BASE-T1S hoặc Gigabit Ethernet (TSN) thay thế một phần mạng CAN/LIN cũ, cung cấp băng thông cao cho cảm biến ADAS và giảm cáp chéo bằng cấu trúc multi-drop.
- Quản lý năng lượng thông minh: Các Zonal Controller tích hợp chức năng phân phối điện và cầu chì điện tử (smart switches/eFuses), thay thế hộp cầu chì truyền thống, nâng cao độ an toàn (đáp ứng ASIL B/D) và chẩn đoán sự cố theo thời gian thực.
- Thúc đẩy phát triển chất bán dẫn (Semiconductors): Yêu cầu chuyển dịch từ quy trình sản xuất chip cũ sang các tiến trình công nghệ tiên tiến (16nm, 7nm, 3nm) cho vi xử lý (SoC) và ứng dụng vật liệu thể hệ mới như Silicon Carbide (SiC) cho việc quản lý điện áp cao.

2.1.3 Tổng quan các giao thức truyền thông trên ô tô

Hệ thống truyền thông đóng vai trò là mạng lưới liên kết hàng trăm đơn vị điều khiển điện tử (ECU) để trao đổi dữ liệu trong toàn bộ phương tiện. Trong các kiến trúc truyền thống, các giao thức như LIN và CAN được sử dụng rộng rãi để điều khiển các chức năng rời rạc. Tuy nhiên, sự gia tăng của các tính năng phần mềm và nhu cầu về băng thông lớn cho hệ thống hỗ trợ lái nâng cao (ADAS) đang thúc đẩy sự chuyển dịch sang các chuẩn truyền thông tốc độ cao hơn như Ethernet ô tô. Việc lựa chọn giao thức phù hợp phụ thuộc vào yêu cầu cụ thể của từng miền chức năng về tốc độ xử lý, độ tin cậy và chi phí triển khai.

Bảng 2.1 So sánh các giao thức truyền thông trên ô tô.

Tiêu chí	LIN (Local Interconnect Network)	CAN / CAN-FD (Controller Area Network)	Ethernet ô tô (100BASE-T1 trở lên)	MOST (Media Oriented Systems Transport)
Tốc độ truyền (Baud rate)	Thấp (thường dưới 20 kbps).	Trung bình (lên đến 2-5 Mbps với CAN-FD).	Cao (100 Mbps đến vài Gbps).	Rất cao (chuyên dụng cho đa phương tiện).
Đặc tính quan trọng	Chi phí thấp, đơn giản cho ứng dụng không trọng yếu.	Trọng tài không phá hủy: Phát hiện ưu tiên để dừng truyền nếu cần.	Băng thông toàn phần ở cả hai hướng, không xảy ra va chạm dữ liệu.	Băng thông lớn, hỗ trợ truyền tải video/âm thanh thời gian thực.
Cơ chế truyền dữ liệu (Mới)	Chủ/Tớ (Master/Slave): Chạy theo	Đa chủ (Multi-master): Các nút truyền bất cứ	Điểm-đến-điểm (Point-to-Point) ở lớp	Truyền dữ liệu nối tiếp theo thời

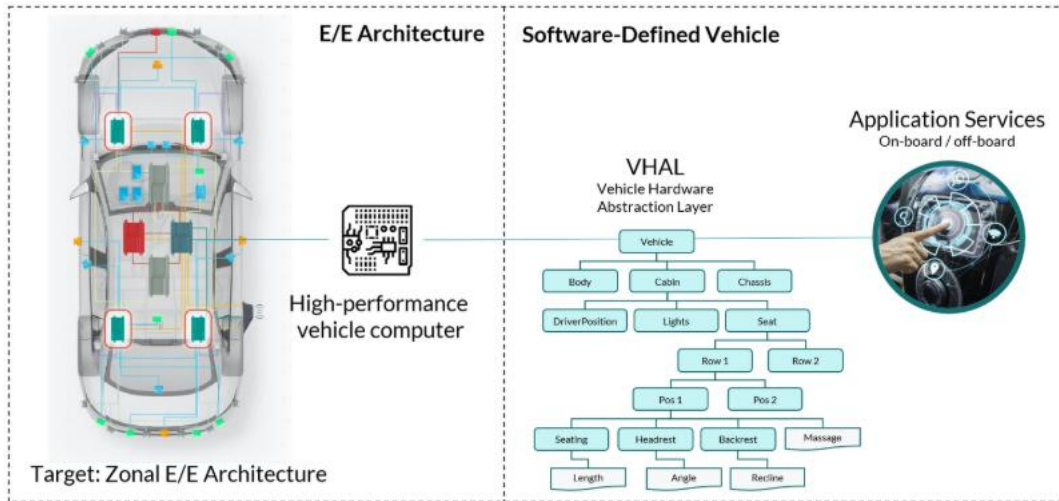
	Bảng lập lịch (Schedule Table) từ nút gốc.	lúc nào dựa trên mức ưu tiên mã nhận diện.	vật lý, truyền nhận song công (Full duplex).	gian thực cho các luồng media.
Kiểu kết nối (Topology)	Đơn dây, cấu trúc Master-Slave.	Bus đa điểm, sử dụng cặp dây xoắn.	Điểm-đến-điểm qua cặp dây xoắn duy nhất.	Vòng (Ring) hoặc hình sao.
Ứng dụng chính	Điều khiển cửa sổ, gương, ghế và các cảm biến đơn giản.	Hệ thống truyền động, phanh, túi khí và chẩn đoán.	Kết nối máy tính trung tâm, cổng vùng, ADAS và radar.	Hệ thống thông tin giải trí, màn hình hiển thị và âm thanh.

2.1.4 Xe được định nghĩa bằng phần mềm

Xe được định nghĩa bằng phần mềm (SDV) đánh dấu sự chuyển dịch từ kiến trúc lấy phần cứng làm trung tâm sang phần mềm làm cốt lõi. Khác với xe truyền thống, SDV cho phép nâng cấp tính năng và sửa lỗi liên tục sau sản xuất thông qua cập nhật qua mạng (OTA). Hệ thống thay thế các ECU phân tán bằng kiến trúc vùng tập trung, tích hợp đa nhiệm (như ADAS và giải trí) trên nền tảng tính toán hiệu năng cao.

Cấu trúc SDV gồm ba lớp chính: phần cứng (cảm biến, tính toán), phần mềm (hệ điều hành, ứng dụng) và giao diện API giúp tách rời linh hoạt phần mềm khỏi phần cứng.

Hình bên dưới minh họa sự chuyển đổi từ kiến trúc E/E truyền thống sang kiến trúc xe định nghĩa bằng phần mềm (Software-Defined Vehicle), trong đó các ECU phân tán được thay thế bằng máy tính hiệu năng cao theo mô hình zonal, kết hợp với lớp trừu tượng phần cứng (VHAL) để cung cấp dịch vụ ứng dụng linh hoạt.



Hình 2.3 Chuyển đổi E/E sang Software-Defined Vehicle (SDV).

2.1.5 Kiến trúc hướng dịch vụ

Kiến trúc hướng dịch vụ (SOA) trong lĩnh vực ô tô là tư duy thiết kế tập trung vào việc đóng gói các tính năng chức năng thành các "Dịch vụ" (Services) độc lập và chuẩn hóa. Thông qua các giao diện lập trình ứng dụng (API), SOA thiết lập sự tách biệt rõ ràng giữa thành phần cung cấp dịch vụ (Service Provider) và thành phần tiêu thụ (Service Consumer). Trong hệ thống phân tán theo vùng (Zonal Architecture), mô hình này mang lại lợi ích cốt lõi là tính kết nối lỏng (Loose Coupling), giúp các bộ điều khiển (ECU) tương tác mà không cần hiểu rõ chi tiết triển khai phần cứng của nhau, đồng thời tối ưu hóa khả năng mở rộng (Scalability), cho phép cập nhật hoặc bổ sung tính năng mới mà không làm thay đổi cấu trúc mạng vật lý nền tảng.

2.1.6 Giới thiệu CMake

Ngoài khung truyền thông tiêu chuẩn, SOME/IP còn định nghĩa về quá trình quảng bá dịch vụ, gọi là SOME/IP-SD. Hình bên dưới mô tả khung truyền của SOME/IP-SD.

CMake là một hệ thống quản lý và xây dựng dự án (build system) mã nguồn mở, cho phép tự động hóa quy trình biên dịch trên nhiều nền tảng và môi trường phát triển khác nhau. Thay vì viết các tệp Makefile phức tạp, CMake sử dụng các tệp cấu hình CMakeLists.txt để mô tả cấu trúc dự án, từ đó tự động tạo ra các tệp

tin xây dựng tương thích với trình biên dịch của từng hệ điều hành. Trong phát triển hệ thống nhúng, CMake đóng vai trò thiết yếu trong việc quản lý thư viện, tùy chỉnh cấu hình biên dịch và tích hợp chặt chẽ với các bộ công cụ phát triển (toolchain), giúp đồng nhất quy trình xây dựng phần mềm trên nhiều cấu trúc phần cứng khác nhau.

2.1.7 Giới thiệu platform ESP-IDF

ESP-IDF (Espressif IoT Development Framework) là bộ khung phát triển phần mềm chính thức dành cho các dòng chip điều khiển ESP32 của Espressif. Nền tảng này cung cấp một môi trường toàn diện bao gồm trình biên dịch, hệ điều hành thời gian thực (RTOS), bộ thư viện ngoại vi (Drivers), và các giao thức mạng như Wi-Fi, BLE, Ethernet. Nhờ cấu trúc phân lớp linh hoạt và hỗ trợ mạnh mẽ cho các chuẩn công nghiệp, ESP-IDF cho phép lập trình viên tập trung vào việc hiện thực hóa các ứng dụng IoT phức tạp, đồng thời đảm bảo hiệu năng tối ưu và khả năng quản lý tài nguyên hiệu quả trên phần cứng của hệ thống.

2.1.8 Giới thiệu FreeRTOS

FreeRTOS là hệ điều hành thời gian thực (RTOS) phổ biến nhất trong thế giới nhúng, được thiết kế để quản lý các tác vụ (tasks) theo cơ chế ưu tiên và thời gian thực. Trong các hệ thống điều khiển theo vùng (Zonal Architecture) yêu cầu độ tin cậy cao, FreeRTOS cung cấp các cơ chế quản lý tài nguyên quan trọng như hàng đợi (queues), semaphore, và mutex để đảm bảo các tiến trình giao tiếp đồng bộ, an toàn và không bị xung đột. Đặc điểm nổi bật của FreeRTOS là dung lượng bộ nhớ nhỏ, tốc độ phản hồi nhanh và khả năng vận hành ổn định trong môi trường đa nhiệm, giúp hệ thống duy trì được tính logic và tính thời gian thực cho các ứng dụng giám sát và điều khiển.

2.1.9 Giới thiệu pySide6

PySide6 là bộ thư viện Python chính thức được cung cấp bởi Qt, cho phép phát triển các ứng dụng giao diện người dùng đồ họa (GUI) chuyên nghiệp trên nhiều nền tảng. Trong bối cảnh dự án, PySide6 được sử dụng để xây dựng các công cụ hỗ trợ, phần mềm giám sát hoặc giao diện điều khiển (HMI) trên máy tính, cho

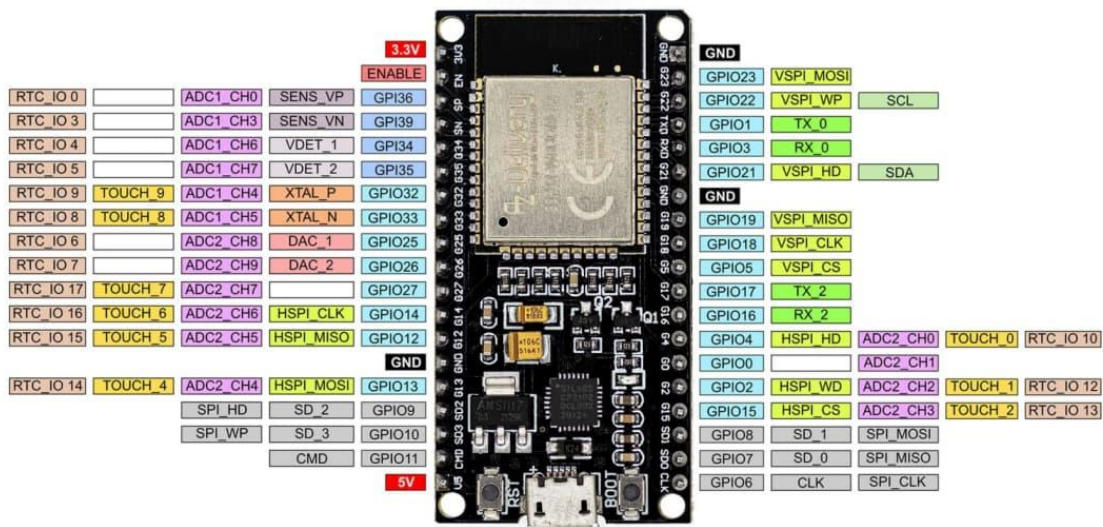
phép tương tác trực tiếp với các node trong hệ thống thông qua mạng Ethernet. Với khả năng tích hợp linh hoạt với các thư viện xử lý dữ liệu và hỗ trợ đa luồng (multi-threading), PySide6 giúp hiện thực hóa các bảng điều khiển trực quan, cung cấp khả năng quan sát trạng thái hệ thống theo thời gian thực và cấu hình các thông số dịch vụ một cách hiệu quả.

2.2 MỘT SỐ LINH KIỆN VÀ DỊCH VỤ TRONG HỆ THỐNG

2.2.1 Vi điều khiển ESP32-WROOM-32

Được phát triển bởi Espressif Systems (Trung Quốc), ESP-WROOM-32 là dòng Dịch vụ SoC (System-on-Chip) tích hợp Wi-Fi và Bluetooth công suất thấp, có kích thước cực kỳ nhỏ gọn. Đây là nền tảng phần cứng mạnh mẽ, chi phí thấp, được thiết kế tối ưu cho các ứng dụng Internet vạn vật (IoT), từ các mạng cảm biến công suất thấp đến các tác vụ đòi hỏi xử lý phức tạp như mã hóa giọng nói và truyền phát nhạc.

Hình 2.4 mô tả sơ đồ phân bổ các chân (Pinout) cũng như các chức năng ngoại vi tích hợp trên bo mạch phát triển sử dụng ESP-WROOM-32.



Hình 2.4 Sơ đồ chân bo mạch phát triển sử dụng ESP-WROOM-32.

ESP-WROOM-32 được sử dụng trong nghiên cứu này với vai trò là khối xử lý trung tâm thay vì một Dịch vụ truyền thông không dây thông thường. Với kiến trúc lõi kép và khả năng vận hành ở tần số 240 MHz, Dịch vụ đáp ứng tốt các yêu cầu về tính toán đa tác vụ trên nền tảng FreeRTOS. Giải pháp này không chỉ đảm bảo

hiệu suất thực thi các thuật toán điều khiển mà còn tối ưu hóa quá trình phát triển nhờ sự hỗ trợ mạnh mẽ từ cộng đồng và tính tương thích cao của phần cứng.

Bảng 2.2 Thông số kỹ thuật Dịch vụ ESP-WROOM-32.

Thành phần	Đặc tính	Chi tiết (Thông số)
Vi xử lý	Loại	Xtensa® Dual-core 32-bit LX6
	Số nhân	2
	Xung nhịp	Điều chỉnh từ 80 MHz đến 240 MHz
Bộ nhớ	ROM	448 KB (Cho booting và các hàm cốt lõi)
	SRAM	520 KB
	Flash	Thường là 4 MB (Tích hợp ngoài qua SPI)
Kết nối	Wi-Fi	802.11 b/g/n (802.11n lên đến 150 Mbps)
	Bluetooth	v4.2 BR/EDR và Bluetooth Low Energy (BLE)
Ngoại vi	GPIO	38 chân (Hỗ trợ Capacitive Touch, ADC, DAC, SPI, I2C, UART, PWM)
	Cảm biến	Cảm biến Hall, Cảm biến nhiệt độ tích hợp
Nguồn	Điện áp	2.2V đến 3.6V (Điện hình 3.3V)

2.2.2 Thiết bị chuyển mạch mạng (Network Switch)

Switch là thiết bị hoạt động ở tầng liên kết dữ liệu (Layer 2) trong mô hình OSI, có chức năng chuyển tiếp các khung Ethernet giữa các thiết bị trong mạng nội bộ. Thiết bị sử dụng bảng địa chỉ MAC để xác định cổng đích tương ứng, từ đó chỉ chuyển tiếp dữ liệu đến đúng thiết bị nhận thay vì phát đến toàn bộ mạng. Cơ chế này giúp giảm lưu lượng quảng bá và nâng cao hiệu quả sử dụng băng thông. Bên cạnh đó, các thông số về bộ đệm gói (Buffer memory) đóng vai trò quyết định đến hiệu suất hoạt động của thiết bị, giúp duy trì tính toàn vẹn của dữ liệu trong các kịch bản lưu lượng truy cập cao.

Trong quá trình triển khai thực nghiệm, đề tài sử dụng thiết bị Mercusys MW305R. Mặc dù đây là một thiết bị định tuyến (Router) dân dụng, nhưng hệ thống tận dụng tối ưu phân đoạn chuyển mạch tích hợp (Integrated Switch) bên trong thiết bị để kết nối các nút mạng. Việc sử dụng thiết bị này giúp giảm chi phí triển khai và thuận tiện cho quá trình kiểm thử, giám sát lưu lượng mạng cũng như cấu hình hệ thống trong giai đoạn phát triển.

Thiết bị định tuyến Mercusys MW305R được trang bị 01 cổng mạng diện rộng (WAN) và 03 cổng mạng nội bộ (LAN) với tốc độ truyền tải đạt 100 Mbps cho mỗi cổng. Trong phạm vi đề tài, 03 cổng LAN được sử dụng để kết nối vật lý giữa các Dịch vụ hệ thống; đồng thời, máy tính điều khiển thực hiện kết nối với thiết bị thông qua mạng không dây (Wi-Fi) nhằm phục vụ công tác kiểm thử và hiệu chỉnh các chức năng vận hành. Hình dưới đây minh họa hình ảnh thực tế của thiết bị được sử dụng.



Hình 2.5 Router Mercusys MW305R.

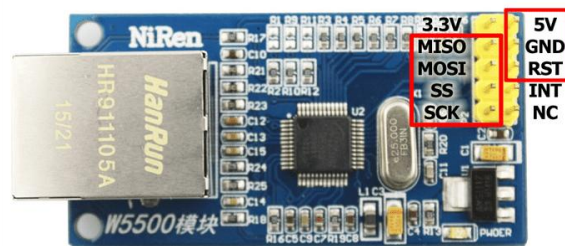
2.2.3 Dịch vụ giao tiếp Ethernet W5500

Bộ điều khiển Ethernet W5500 là giải pháp tối ưu cung cấp khả năng kết nối Internet cho các hệ thống nhúng thông qua giao diện ngoại vi nối tiếp (SPI). W5500 tích hợp nhân giao thức TCP/IP bằng phần cứng (Hardwired TCP/IP Core), hỗ trợ xử lý các giao thức TCP, UDP, ICMP, IPv4 và ARP mà không cần vi điều khiển thực hiện toàn bộ các tác vụ ở tầng mạng. Nhờ đó, vi điều khiển có thể tập trung xử lý các tác vụ ứng dụng thay vì phải xử lý trực tiếp các giao thức truyền thông mạng.

Bảng 2.3 Bảng thông số kỹ thuật W5500.

Thành phần	Đặc tính	Chi tiết (Thông số)
Ethernet	Tốc độ	10BaseT/100BaseTX (Hỗ trợ Tự động thương lượng)
Giao diện	SPI	Hỗ trợ tối đa 80 MHz (Chế độ SPI 0 và 3)
Bộ nhớ đệm	TX/RX	Tổng cộng 32 KB bộ nhớ đệm nội bộ cho các kênh truyền/nhận
Khả năng kết nối	Số lượng Socket	Hỗ trợ tối đa 08 socket độc lập hoạt động đồng thời
Giao thức hỗ trợ	Hardwired TCP/IP	TCP, UDP, ICMP, IPv4, ARP, IGMP, PPPoE
Điện áp	Hoạt động	3.3V (Các chân I/O có khả năng chịu được mức điện áp 5V)

Hình bên dưới mô tả các giao diện trên Dịch vụ W5500:



Hình 2.6 Sơ đồ giao diện của W5500.

Hệ thống thanh ghi của W5500 được tổ chức thành các khối để quản lý cấu hình chung và quản lý riêng cho từng kênh truyền thông. Các thanh ghi chính dùng để cấu hình thông tin mạng tổng thể cho thiết bị. Bảng bên dưới mô tả các thanh ghi chính (cách thanh ghi chung).

Bảng 2.4 Các thanh ghi chính trong Dịch vụ W5500.

Địa chỉ Offset	Tên thanh ghi	Chức năng chính
0x0000	MR (Mode Register)	Thiết lập chế độ hoạt động, Reset phần mềm, PPPoE

0x0001 - 0x0004	GAR	Địa chỉ Cổng mặc định (Gateway IP Address)
0x0005 - 0x0008	SUBR	Mặt nạ mạng con (Subnet Mask Address)
0x0009 - 0x000E	SHAR	Địa chỉ vật lý của thiết bị (Source MAC Address)
0x000F - 0x0012	SIPR	Địa chỉ IP của thiết bị (Source IP Address)
0x002E	PHYCFGR	Cấu hình và giám sát trạng thái tầng vật lý (PHY)

Các thanh ghi Socket dùng để điều khiển và giám sát trạng thái của từng socket cụ thể (từ 0 đến 7).

Bảng 2.5 Các thanh ghi Socket.

Địa chỉ Offset	Tên thanh ghi	Chức năng chính
0x0000	Sn_MR	Thiết lập giao thức (TCP, UDP, MACRAW) cho Socket _n
0x0001	Sn_CR	Gửi các lệnh điều khiển: OPEN, LISTEN, CONNECT, SEND, RECV
0x0002	Sn_IR	Chỉ thị các trạng thái ngắt (Kết nối thành công, nhận dữ liệu, Timeout)
0x0003	Sn_SR	Hiển thị trạng thái hiện tại của Socket (SOCK_ESTABLISHED, SOCK_CLOSE_WAIT,...)
0x001E - 0x001F	Sn_RXBUF/TXBUF	Cấu hình kích thước bộ nhớ đệm nhận và gửi cho từng Socket
0x0026 - 0x0027	Sn_RX_RSR	Kiểm tra dung lượng dữ liệu hiện có trong bộ đệm nhận

Do cấu trúc phần cứng của Dịch vụ Ethernet W5500 không tích hợp bộ nhớ lưu trữ không khả biến (Flash/ROM), các thông số cấu hình sẽ bị xóa khi mất

nguồn cung cấp hoặc khởi động lại hệ thống. Vì vậy, các tham số mạng như địa chỉ IP, Subnet Mask, Gateway và MAC Address cần được cấu hình lại sau mỗi lần khởi động hệ thống. Sơ đồ trình tự dưới đây mô tả chi tiết quá trình cấu hình các thành ghi hệ thống và quy trình kích hoạt Socket 0 vận hành ở chế độ MACRAW để thu thập các khung dữ liệu Ethernet.

2.2.4 Cảm biến khoảng cách siêu âm HC-SR04

Cảm biến khoảng cách siêu âm HC-SR04 là một loại cảm biến được sử dụng phổ biến trong các hệ thống nhúng, robot và tự động hóa để đo khoảng cách đến vật thể. Cảm biến hoạt động dựa trên nguyên lý Time of Flight (ToF), trong đó khoảng cách được xác định bằng cách đo thời gian truyền và phản xạ của sóng siêu âm từ cảm biến đến vật cản và ngược trở lại.

HC-SR04 được tích hợp một đầu phát (Transmitter) và một đầu thu (Receiver) sóng siêu âm có tần số hoạt động khoảng 40 kHz. Khi chân Trigger nhận xung kích hoạt từ vi điều khiển, cảm biến phát ra chuỗi sóng siêu âm có tần số khoảng 40 kHz. Sóng phản xạ từ vật cản sẽ được đầu thu tiếp nhận để xác định thời gian truyền và nhận tín hiệu. Dựa trên thời gian truyền và nhận tín hiệu cùng với vận tốc truyền âm trong không khí, khoảng cách đến vật cản được tính toán.



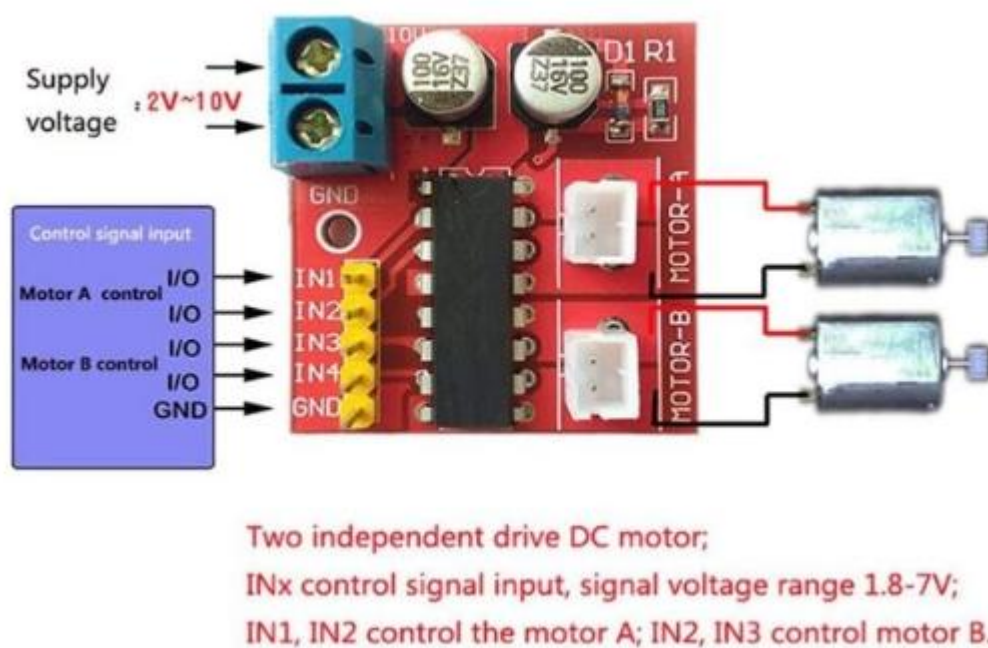
Hình 2.7 Cảm biến siêu âm HC-SR04.

2.2.5 Mạch cầu H

Mạch cầu H là một mạch điện tử công suất được sử dụng để điều khiển động cơ điện một chiều (DC Motor), cho phép động cơ quay theo cả hai chiều bằng cách thay đổi chiều dòng điện đi qua động cơ. Ngoài việc điều khiển chiều quay, mạch cầu H còn có thể kết hợp với tín hiệu điều chế độ rộng xung (PWM) để điều chỉnh tốc độ quay của động cơ.

Trong các hệ thống nhúng và robot, mô-đun điều khiển động cơ sử dụng IC L298N được sử dụng phổ biến nhờ khả năng điều khiển đồng thời hai động cơ DC hoặc một động cơ bước. IC L298N được chế tạo theo công nghệ cầu H kép (Dual Full-Bridge Driver), có khả năng điều khiển động cơ hoạt động ở điện áp từ 2V đến 10V, với dòng điện tối đa khoảng 1.5A đến 2A cho mỗi kênh.

Mô-đun L298N sử dụng các chân điều khiển logic (IN1, IN2, IN3 và IN4) để xác định chiều quay của động cơ, trong khi các chân ENA và ENB thường được sử dụng để nhận tín hiệu PWM nhằm điều chỉnh tốc độ quay. L298N hỗ trợ điều khiển đồng thời hai động cơ DC độc lập, tương thích với nhiều loại vi điều khiển và thường được sử dụng trong các hệ thống robot, xe mô hình và các ứng dụng điều khiển chuyển động.



Hình 2.8 Mô-đun mạch cầu H sử dụng IC L298N.

Bảng 2.6 Bảng thông số kỹ thuật mô-đun mạch cầu H.

Thông số	Giá trị
IC điều khiển	L298N
Điện áp cấp động cơ	2V – 10V

Điện áp logic	1.8 V – 7 V (Tương thích tốt với vi điều khiển 3.3V và 5V)
Dòng tối đa mỗi kênh	2 A
Số kênh điều khiển	2 động cơ DC độc lập
Điều khiển tốc độ	PWM
Điều khiển chiều quay	IN1–IN4

2.2.6 Mạch hạ áp DC-DC

Mạch hạ áp DC-DC (DC-DC Buck Converter) là bộ chuyển đổi nguồn có chức năng giảm điện áp một chiều từ mức điện áp đầu vào cao xuống mức điện áp đầu ra thấp hơn, đồng thời duy trì hiệu suất chuyển đổi cao. Khác với bộ ổn áp tuyến tính (Linear Regulator), mạch hạ áp DC-DC sử dụng kỹ thuật chuyển mạch kết hợp với cuộn cảm, diode và tụ điện để điều chỉnh điện áp đầu ra, nhờ đó giảm đáng kể công suất tiêu tán dưới dạng nhiệt và nâng cao hiệu suất hoạt động.

Trong các hệ thống nhúng, mạch hạ áp DC-DC được sử dụng để tạo ra các mức điện áp phù hợp cho từng thiết bị từ một nguồn điện chung. Các bộ chuyển đổi này có khả năng cung cấp dòng điện lớn, điện áp đầu ra ổn định và hiệu suất thường đạt trên 80%, phù hợp với các ứng dụng sử dụng pin hoặc nguồn điện một chiều.

Trong đề tài này, mô-đun hạ áp sử dụng IC LM2596S-ADJ. Đây là bộ chuyển đổi Buck có điện áp đầu vào rộng, điện áp đầu ra có thể điều chỉnh và dòng tải tối đa khoảng 3A. LM2596S-ADJ hỗ trợ dải điện áp đầu vào rộng, điện áp đầu ra có thể điều chỉnh và khả năng cung cấp dòng tải lớn, phù hợp với các hệ thống nhúng sử dụng nhiều mức điện áp khác nhau.



Hình 2.9 Mô-đun mạch hạ áp DC-DC sử dụng IC LM2596S-ADJ.

Bảng 2.7 Bảng thông số kỹ thuật của mô-đun hạ áp LM2596S-ADJ.

Thông số	Giá trị
IC điều khiển	LM2596S-ADJ
Điện áp đầu vào	4,5V – 40V
Điện áp đầu ra	1,25V – 37V (Có thể điều chỉnh mịn qua biến trở tinh chỉnh)
Dòng tải đầu ra tối đa	2A (Duy trì ổn định liên tục), lên đến 3A (Nếu có thêm tản nhiệt)
Hiệu suất chuyển đổi	Lên đến 92% (Phụ thuộc vào độ lệch áp giữa đầu vào và đầu ra)
Kiểu chuyển đổi	Buck Converter (Hạ áp)
Tần số chuyển mạch	150 kHz
Cơ chế bảo vệ tích hợp	Bảo vệ quá nhiệt, giới hạn dòng điện ngõ ra, bảo vệ ngắn mạch

2.3 CÁC CHUẨN GIAO TIẾP VÀ GIAO THỨC TRUYỀN DỮ LIỆU

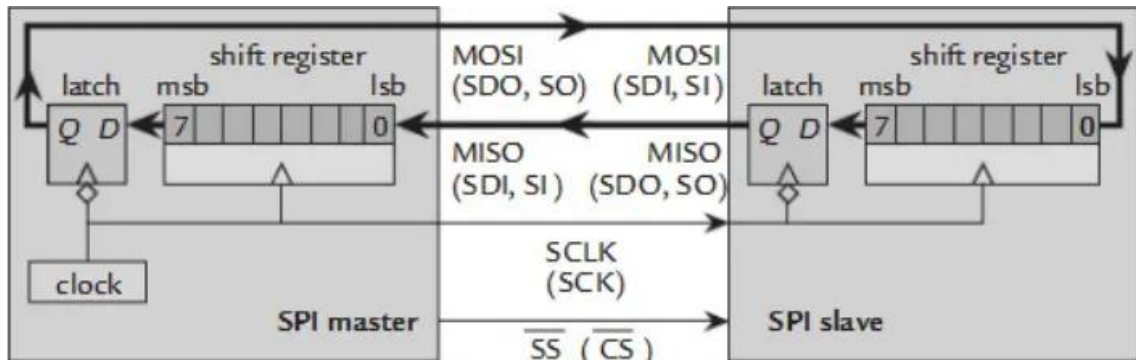
2.3.1 Chuẩn giao tiếp SPI

SPI là giao thức truyền thông nối tiếp đồng bộ, hỗ trợ chế độ song công (Full-duplex) cho phép trao đổi dữ liệu hai chiều đồng thời. Hệ thống điển hình gồm một Master điều khiển tín hiệu clock và nhiều Slave, tuy nhiên trong đề tài này, cấu hình được tối giản chỉ với 1 Master và 1 Slave. Nhờ cấu trúc phần cứng đơn giản, SPI đạt tốc độ cao hơn nhiều so với chuẩn bất đồng bộ, với dải tần phổ biến từ 1-70MHz.

Về mặt kết nối vật lý, một giao thức SPI tiêu chuẩn thường sử dụng một hệ thống gồm 4 đường tín hiệu chính:

- SCLK (Serial Clock): Đường tín hiệu cung cấp xung nhịp đồng bộ, được tạo ra và kiểm soát duy nhất bởi thiết bị Master.
- SS/CS (Slave Select / Chip Select): Đường tín hiệu do Master điều khiển để chọn và kích hoạt đúng thiết bị Slave mà nó muốn giao tiếp.

- MOSI (Master Output / Slave Input): Đường tín hiệu đơn hướng, mang đặc tính gửi luồng dữ liệu từ Master sang Slave.
- MISO (Master Input / Slave Output): Đường truyền tín hiệu mang đặc tính gửi luồng dữ liệu phản hồi từ Slave về lại Master.



Hình 2.10 Thiết kế khối SPI master và slave.

SPI không quy định cấu trúc khung dữ liệu cụ thể. Định dạng dữ liệu được quyết định bởi ứng dụng và giao thức được xây dựng trên SPI. Điều này xuất phát từ việc phần cứng giao thức không tích hợp sẵn cơ chế kiểm soát luồng (flow control) hay các bit xác nhận (acknowledgment). Về đặc tính truyền nhận trên hai đường dữ liệu chính, thiết bị Master sẽ chủ động khởi tạo việc gửi từng bit dữ liệu dọc theo chân MOSI, đồng thời Slave sẽ đọc các bit này ngay khi chúng đi tới. Song song đó, nếu Master có nhu cầu lấy dữ liệu phản hồi, Slave sẽ đẩy từng bit dữ liệu lên đường MISO để gửi về, và Master tiến hành đọc các bit nhận được tương ứng. Quá trình xoay vòng bit này thường được thực thi thông qua cấu trúc hai thanh ghi dịch (shift register) nằm bên trong cả Master và Slave.

Hoạt động của SPI được quyết định bởi hai thông số cấu hình quan trọng là CPOL (Clock Polarity) và CPHA (Clock Phase), giúp xác định thời điểm dữ liệu được truyền và lấy mẫu:

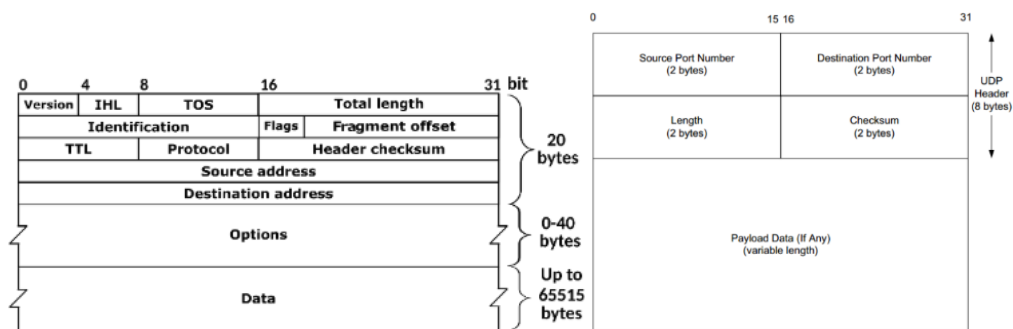
- SCLK (Serial Clock): Đường tín hiệu cung cấp xung nhịp đồng bộ, được tạo ra và kiểm soát duy nhất bởi thiết bị Master.
- CPOL (Clock Polarity): Quy định trạng thái nhàn rỗi (idle) của đường xung nhịp SCLK.
 - o CPOL = 0: Mức logic của SCLK ở trạng thái nhàn rỗi là thấp (Low).
 - o CPOL = 1: Mức logic của SCLK ở trạng thái nhàn rỗi là cao (High).

- CPHA (Clock Phase): Quy định thời điểm lấy mẫu dữ liệu dựa trên cạnh xung clock.
 - o CPHA = 0: Dữ liệu được lấy mẫu tại cạnh đầu tiên của mỗi chu kỳ xung nhịp.
 - o CPHA = 1: Dữ liệu được lấy mẫu tại cạnh thứ hai của mỗi chu kỳ xung nhịp.

Sự kết hợp của hai thông số này tạo ra 4 chế độ truyền SPI (Mode 0 đến Mode 3). Master và Slave phải được cấu hình cùng chế độ SPI để đảm bảo dữ liệu được lấy mẫu đúng thời điểm trong quá trình truyền nhận, đảm bảo dữ liệu không bị lệch pha trong quá trình truyền tải ở tốc độ cao (20MHz). Thông thường, Master và Slave phải được cấu hình chung một Mode trước khi bắt đầu phiên truyền.

2.3.2 Giao thức truyền thông UDP

Giao thức UDP là giao thức ở Tầng 4 trong mô hình OSI, sử dụng để gởi các gói dữ liệu độc lập (Datagram). Giao thức UDP kết hợp với IPv4, làm cho việc gởi dữ liệu trong mạng cục bộ trở nên dễ dàng. Hình bên dưới mô tả khung truyền của IPv4 và UDP.



a) Khung truyền IPv4.

b) Khung truyền UDP.

Hình 2.11 Khung truyền IPv4 và UDP.

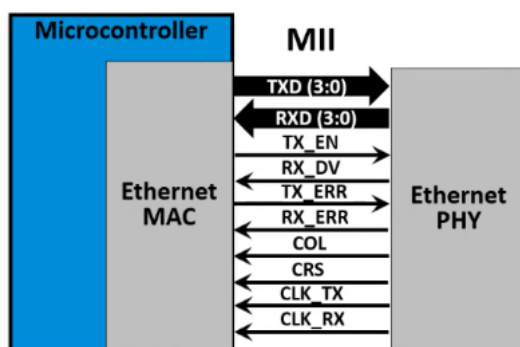
Đề tài sử dụng phần cứng có sẵn trên thị trường, nên đề tài chỉ tập trung vào các trường trong Bảng 2.8 gồm các trường được sử dụng trong đề tài:

Bảng 2.8 Danh sách các trường cần quan trọng của IPv4 và UDP.

Giao thức	Trường	Chú giải
IPv4	Protocol	Chỉ định gói tin đang được mang theo ở tầng trên là UDP. Giá trị cố định: 0x11.
	Source address (Địa chỉ nguồn)	Thông tin về địa chỉ IPv4 của thiết bị gửi/nhận.
	Destination address (Địa chỉ đích)	
UDP	Source port (Cổng nguồn)	Thông tin về cổng dịch vụ của thiết bị gửi/nhận.
	Destination Port (Cổng đích)	
	Length	Độ dài dữ liệu thực được vận chuyển với UDP.

2.3.3 Giao thức Ethernet trên ô tô

Ethernet là chuẩn truyền thông mạng cục bộ (LAN) có dây, được tổ chức IEEE chuẩn hóa (IEEE 802.3) nhằm định nghĩa Lớp 1 (Vật lý) và Lớp 2 (Liên kết dữ liệu) trong mô hình tham chiếu OSI. Một nút Ethernet (Ethernet Node) điển hình bao gồm các thành phần cơ bản là vi điều khiển, bộ điều khiển MAC (Media Access Control) và lớp vật lý PHY. Hình bên dưới mô tả cấu trúc một nút Ethernet:

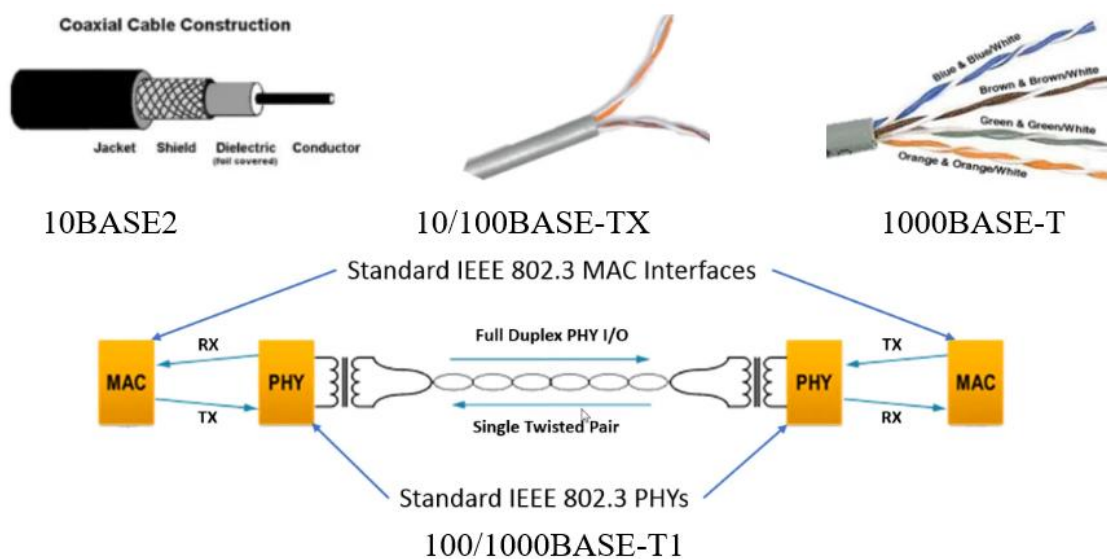


Hình 2.12 Cấu trúc nút Ethernet.

Sự khác biệt cốt lõi giữa Ethernet ô tô và Ethernet dân dụng nằm ở phương thức kết nối vật lý để đáp ứng các yêu cầu khắt khe trong môi trường phương tiện:

- Ethernet ô tô cho phép truyền thông hai chiều chỉ trên một cặp dây xoắn duy nhất (Single Twisted Pair), trong khi Ethernet dân dụng thường yêu cầu từ 2 đến 4 cặp dây.
- Việc giảm số lượng dây giúp tối ưu hóa khối lượng bằng cách cắt giảm đáng kể trọng lượng của bộ dây dẫn (wire harness), từ đó làm giảm tổng trọng lượng và tăng hiệu suất cho xe.
- Hệ thống trên xe hơi sử dụng các tiêu chuẩn chuyên dụng như IEEE 802.3bw cho tốc độ 100 Mbps và IEEE 802.3bp cho tốc độ 1 Gbps thay vì các chuẩn phổ biến trong gia đình hoặc văn phòng.

Hình 2.13 minh họa các loại môi trường truyền Ethernet từ truyền thống (10BASE2, 10/100BASE-TX, 1000BASE-T) đến Automotive Ethernet (100/1000BASE-T1), cùng với kiến trúc kết nối giữa MAC và PHY theo chuẩn IEEE 802.3.



Hình 2.13 Các loại cáp / môi trường truyền Ethernet.

Hiện nay, Ethernet ô tô đang trở thành "xương sống" kỹ thuật để hiện thực hóa các mục tiêu của ngành công nghiệp ô tô theo xu hướng CASE (Kết nối, Tự hành, Chia sẻ và Điện hóa). Công nghệ này cung cấp dải băng thông rộng hơn so với các môi trường truyền thông truyền thống, cho phép xử lý lượng thông tin khổng lồ từ các hệ thống lái xe tự hành và xe kết nối. Bên cạnh đó, Ethernet giúp hợp nhất các

giao thức truyền thông khác nhau về một chuẩn duy nhất, giúp đơn giản hóa cấu trúc mạng và nâng cao khả năng quản lý dữ liệu bên trong phương tiện.

2.3.4 Khung truyền Ethernet

Ethernet là giao thức truyền thông phổ biến tại Tầng Liên kết dữ liệu (Tầng 2) của mô hình OSI. Theo chuẩn IEEE 802.3, cấu trúc một khung truyền Ethernet (Ethernet Frame) bao gồm các thành phần chính sau:

- Preamble & SFD (8 byte): Phần mở đầu gồm 7 byte xen kẽ các bit 0/1 để đồng bộ hóa tốc độ, kết thúc bằng 1 byte phân cách mở đầu khung truyền (SFD - Start Frame Delimiter) để xác định điểm bắt đầu của khung dữ liệu.
- Header (14 byte): Bao gồm địa chỉ MAC đích (6 byte), địa chỉ MAC nguồn (6 byte) và trường độ dài hoặc loại giao thức (2 byte - EtherType/Length).
- Payload (46 - 1500 byte): Phần dữ liệu chính cần truyền tải.
- FCS (4 byte): Trường Chuỗi kiểm tra lỗi khung truyền (FCS-Frame Check Sequence) chứa mã kiểm tra lỗi (CRC 1) nhằm đảm bảo tính toàn vẹn của dữ liệu.

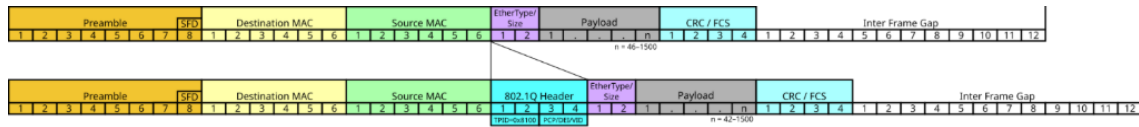
Layer	Preamble	Start frame delimiter (SFD)	MAC destination	MAC source	802.1Q tag (optional)	Ethertype (Ethernet II) or length (IEEE 802.3)	Payload	Frame check sequence (32-bit CRC)	Interpacket gap (IPG)
Length (octets)	7	1	6	6	(4)	2	42–1500 ^[c]	4	12
Layer 2 Ethernet frame	(not part of the frame)		← 64–1522 octets →						(not part of the frame)
Layer 1 Ethernet packet & IPG	← 72–1530 octets →								← 12 octets →

Hình 2.14 Khung truyền Ethernet theo chuẩn IEEE 802.3.

Ngoài khung truyền tiêu chuẩn với tối đa 1500 byte dữ liệu, còn khung truyền mở rộng (Ethernet II Frame) cho phép vùng dữ liệu lớn hơn 1500 byte. Đề tài chỉ sử dụng khung truyền tiêu chuẩn có tối đa 1500 byte dữ liệu.

Trong mạng cục bộ ảo (vLAN), trường kích thước dữ liệu (Length) sẽ được dời về sau 4 byte, vị trí được chèn thêm này là phần đánh thẻ mạng cục bộ ảo

(vLAN tag) theo chuẩn IEEE 802.1Q. Hình 2.6 mô tả sự khác nhau giữa hai khung truyền này.

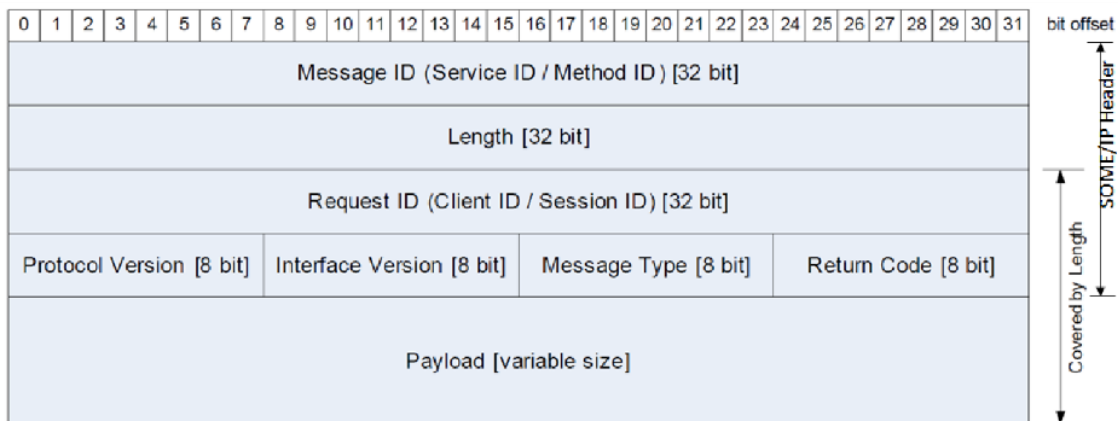


Hình 2.15 Khung truyền được gắn thẻ trong mạng cục bộ ảo.

2.3.5 Giao thức SOME/IP

SOME/IP (Scalable service-Oriented MiddlewarE over IP) là giao thức phần mềm trung gian hướng dịch vụ được thiết kế chuyên biệt cho ngành công nghiệp ô tô, vận hành trên nền tảng bộ giao thức mạng IP. Khác với cơ chế truyền thông hướng tín hiệu (Signal-based) truyền thông của mạng CAN, SOME/IP cho phép các nút mạng (ECU) tương tác thông qua các cơ chế: Yêu cầu/Phản hồi (Request/Response), Gọi hàm từ xa (Remote Procedure Calls - RPC), và Đăng ký/Phát sự kiện (Subscribe/Notify).

Để quản lý luồng dữ liệu hướng dịch vụ này, SOME/IP định nghĩa một cấu trúc Header ứng dụng cố định có kích thước 16 Bytes đặt ngay trước vùng dữ liệu Payload, bao gồm các trường thông tin nghiêm ngặt:



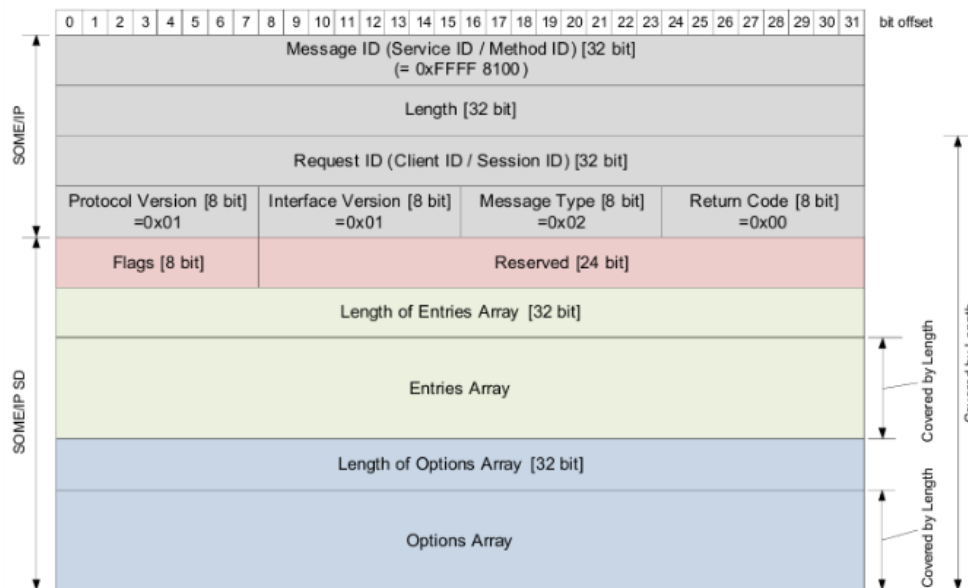
Hình 2.16 Khung truyền của SOME/IP.

- Message ID (32 bit): Trường định danh duy nhất bao gồm Service ID (16 bit) để xác định dịch vụ và Method/Event ID (16 bit) để xác định hành động. Bit cao nhất của trường này đóng vai trò phân loại: 0 cho Method (lời gọi thủ tục từ xa) và 1 cho Event (sự kiện thông báo).

- Length (32 bit): Xác định tổng kích thước của phần dữ liệu phía sau trường này (tính theo byte), giúp thiết bị thu xác định biên của gói tin.
- Request ID (32 bit): Cơ chế quản lý phiên truyền thông, bao gồm Client ID (16 bit) định danh thực thể gửi và Session ID (16 bit) dùng để theo dõi, kiểm soát thứ tự và phát hiện mất gói tin trong các tiến trình gọi-phản hồi.
- Protocol & Interface Version (16 bit): Hai trường 8 bit xác định phiên bản giao thức và giao diện dịch vụ, đảm bảo tính tương thích giữa các thành phần phần mềm.
- Message Type (8 bit): Phân loại loại hình thông điệp, bao gồm: REQUEST (0x00), REQUEST_NO_RETURN (0x01), NOTIFICATION (0x02) và RESPONSE (0x80).
- Return Code (8 bit): Chỉ báo trạng thái phản hồi, với giá trị 0x00 (E_OK) biểu thị thành công và các mã khác dùng để thông báo lỗi ứng dụng hoặc lỗi hệ thống.

2.3.6 Giao thức SOME/IP Discovery

Ngoài khung truyền thông tiêu chuẩn, SOME/IP còn định nghĩa về quá trình quảng bá dịch vụ, gọi là SOME/IP-SD. Hình bên dưới mô tả khung truyền của SOME/IP-SD.



Hình 2.17 Khung truyền quảng bá của SOME/IP.

- Message ID (32 bit): Định danh duy nhất cho toàn bộ thông điệp, bao gồm Service ID (16 bit) xác định loại dịch vụ và Method/Event ID (16 bit) xác định hành động cụ thể. Trong đó, bit cao nhất (MSB) của Method/Event ID đóng vai trò phân biệt: 0 biểu thị cho Method (lời gọi hàm từ xa), 1 biểu thị cho Event (sự kiện phát đi).
- Length (32 bit): Xác định tổng kích thước (tính bằng byte) của toàn bộ dữ liệu phía sau trường này, giúp bộ thu nhận biết giới hạn của gói tin để xử lý chính xác.
- Request ID (32 bit): Thành phần quản lý tiến trình nhằm tránh xung đột dữ liệu, bao gồm Client ID (16 bit) định danh thực thể gửi yêu cầu và Session ID (16 bit) dùng để theo dõi phiên truyền thông, giúp phát hiện các tình trạng mất gói, trùng lặp hoặc sai thứ tự.
- Protocol & Interface Version (16 bit): Các trường thông tin về phiên bản (mỗi trường 8 bit) giúp hệ thống nhận diện định dạng mã hóa (Protocol) và kiểm soát tính tương thích của giao diện dịch vụ (Interface), đảm bảo các bản cập nhật phần mềm không gây xung đột với cấu trúc dữ liệu hiện tại.
- Message Type (8 bit): Phân loại phương thức giao tiếp, bao gồm các loại chính như REQUEST (0x00), REQUEST_NO_RETURN (0x01), NOTIFICATION (0x02) cho dữ liệu sự kiện, và RESPONSE (0x80) cho gói phản hồi.
- Return Code (8 bit): Chỉ báo trạng thái thực thi của ứng dụng. Giá trị 0x00 (E_OK) biểu thị xử lý thành công; các mã khác được sử dụng để thông báo cụ thể về các lỗi hệ thống (như Timeout hoặc sai tham số).

CHƯƠNG 3

THIẾT KẾ VÀ XÂY DỰNG HỆ THỐNG

3.1 YÊU CẦU VÀ SƠ ĐỒ KHỐI HỆ THỐNG

Để hiện thực hóa mô hình điều khiển phân tán theo vùng sử dụng giao thức SOME/IP trên nền tảng Ethernet, hệ thống được xây dựng nhằm đáp ứng các yêu cầu chức năng và kỹ thuật liên quan đến thu thập dữ liệu, truyền thông mạng, điều khiển cơ cấu chấp hành và giám sát trạng thái hoạt động. Mô hình thực nghiệm được thiết kế theo định hướng kiến trúc vùng (Zonal Architecture), trong đó các bộ điều khiển vùng thực hiện xử lý dữ liệu cục bộ và trao đổi thông tin thông qua giao thức hướng dịch vụ SOME/IP.

Hệ thống phải có khả năng thu thập dữ liệu từ các cảm biến được bố trí tại các vùng khác nhau trên mô hình. Dữ liệu sau khi được thu thập sẽ được xử lý sơ bộ tại các bộ điều khiển vùng trước khi truyền qua mạng Ethernet. Việc xử lý tại vùng giúp giảm tải lưu lượng dữ liệu truyền trên mạng và nâng cao hiệu quả hoạt động của toàn hệ thống.

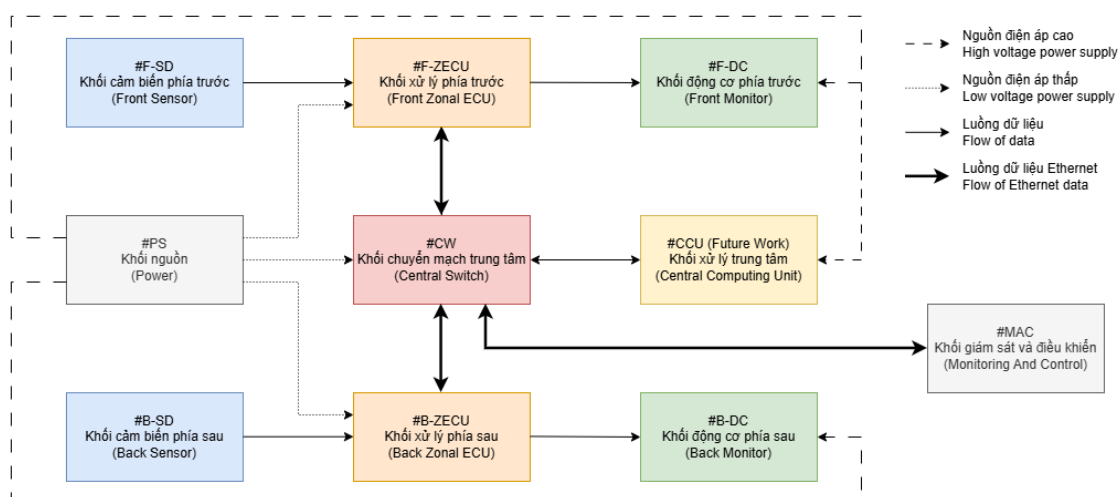
Bên cạnh chức năng thu thập dữ liệu, hệ thống cần hỗ trợ điều khiển các cơ cấu chấp hành như động cơ DC và các thiết bị đầu ra khác. Các lệnh điều khiển được truyền thông qua giao thức SOME/IP, cho phép kiểm chứng khả năng trao đổi dữ liệu và điều khiển từ xa trong kiến trúc hướng dịch vụ.

Toàn bộ các thành phần trong hệ thống được kết nối thông qua mạng Ethernet. Giao thức SOME/IP được triển khai nhằm thực hiện cơ chế cung cấp và sử dụng dịch vụ giữa các nút mạng, tạo tiền đề cho việc mở rộng hệ thống cũng như tích hợp thêm các thiết bị hoặc chức năng mới trong tương lai.

Ngoài ra, hệ thống cần cung cấp giao diện giám sát trực quan (Dashboard) cho phép theo dõi trạng thái kết nối, dữ liệu cảm biến và trạng thái hoạt động của các cơ cấu chấp hành theo thời gian thực. Giao diện này đồng thời hỗ trợ gửi các lệnh điều khiển phục vụ quá trình vận hành và thử nghiệm hệ thống.

3.1.1 Sơ đồ khối hệ thống và chức năng của từng khối

Hệ thống được xây dựng theo sơ đồ khối bên dưới:



Hình 3.1 Sơ đồ khối toàn hệ thống.

3.1.1.1 Khối điều khiển vùng F-ZECU và B-ZECU

Đây là các node mạng vùng (Zonal ECUs) sử dụng vi điều khiển ESP32. Khối này có nhiệm vụ tiếp nhận dữ liệu thô từ cảm biến (F-SD/B-SD), thực hiện xử lý sơ bộ và đóng gói dữ liệu theo giao thức SOME/IP để gửi về trung tâm. Đồng thời, đây cũng là nơi trực tiếp xuất tín hiệu điều khiển cho các động cơ tương ứng.

3.1.1.2 Khối cảm biến F-SD và B-SD

Khối này đóng vai trò là "giác quan" của hệ thống, bao gồm các cảm biến khoảng cách siêu âm (HC-SR04) được lắp đặt ở phía trước và phía sau xe. Ngoài ra, khối còn tích hợp các nút nhấn kiểm thử (Test Buttons) để mô phỏng các tình huống tác động hoặc kiểm tra phản hồi tức thời của hệ thống trong quá trình thử nghiệm.

3.1.1.3 Khối động cơ F-DC và B-DC

Bao gồm hệ thống 4 bánh xe độc lập được điều khiển bởi các động cơ DC và mạch cầu H. Khối này thực thi các lệnh vận hành (tiến, lùi, rẽ, dừng khẩn cấp) được truyền từ CCU thông qua các Zonal ECU, đảm bảo xe di chuyển đúng quỹ đạo và phản ứng kịp thời với vật cản.

3.1.1.4 Khối chuyển mạch trung tâm CW

Khối này đóng vai trò là "xương sống" (Backbone) của hạ tầng truyền thông trong toàn bộ hệ thống. CW là thiết bị chuyển mạch tập trung chịu trách nhiệm kết nối và điều phối dữ liệu giữa các Khối điều khiển vùng (F-ZECU, B-ZECU). Thông qua thiết bị này, các gói tin Ethernet được đóng gói theo giao thức SOME/IP có thể luân chuyển với băng thông rộng và độ trễ cực thấp, đảm bảo tính thời gian thực cho các lệnh điều khiển và dữ liệu cảm biến trong kiến trúc vùng.

3.1.1.5 Khối giám sát và điều khiển MAC

Bao gồm giao diện Dashboard chạy trên máy tính hoặc thiết bị ngoại vi. Khối này cho phép người dùng giám sát trạng thái thời gian thực của hệ thống (khoảng cách cảm biến, trạng thái kết nối các vùng) và thực hiện các lệnh điều khiển thủ công hoặc chuyển đổi chế độ vận hành (Manual/Auto).

3.1.1.6 Khối nguồn PS

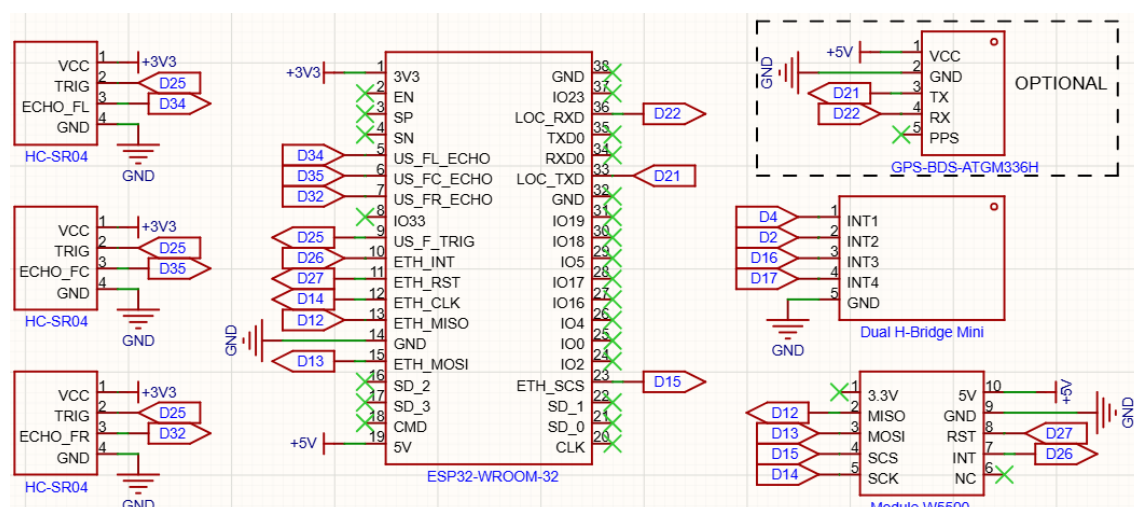
Cung cấp năng lượng cho toàn bộ hệ thống. Khối này bao gồm nguồn pin chính và các mạch hạ áp DC-DC để chuyển đổi mức điện áp phù hợp cho các Dịch vụ điện tử, đảm bảo hệ thống vận hành liên tục và ổn định.

3.2 THIẾT KẾ PHẦN CỨNG

3.2.1 Khối điều khiển vùng

Khối điều khiển vùng (Zone Electronic Control Unit - Zone ECU) là bộ điều khiển chính của mỗi vùng trong hệ thống kiến trúc Zonal Architecture. Khối này có nhiệm vụ thu thập dữ liệu từ các cảm biến, xử lý tín hiệu, điều khiển cơ cấu chấp hành và truyền thông với các Zone ECU khác thông qua mạng Ethernet. Việc

phân tách chức năng theo từng vùng giúp giảm chiều dài dây dẫn, tăng tính linh hoạt trong thiết kế và thuận tiện khi mở rộng hệ thống.



Hình 3.2 Sơ đồ nguyên lý khối điều khiển vùng Zone ECU.

Trung tâm của khối điều khiển là vi điều khiển ESP32-WROOM-32 hoạt động ở điện áp 3,3 V. ESP32 được lựa chọn nhờ sở hữu bộ xử lý lõi kép (kiến trúc Xtensa) xung nhịp lên đến 240MHz, tích hợp bộ nhớ Flash, giao tiếp SPI, UART, GPIO và khả năng xử lý thời gian thực, đáp ứng tốt yêu cầu điều khiển của hệ thống.

Ngoài các khối chức năng chính, hệ thống còn hỗ trợ kết nối mô-đun GPS ATGM336H thông qua giao tiếp UART. Mô-đun GPS được sử dụng để thu nhận thông tin vị trí và thời gian phục vụ các ứng dụng định vị. Trong giai đoạn hiện tại, Mô-đun này được thiết kế dưới dạng tùy chọn (Optional) và có thể được tích hợp trong các phiên bản mở rộng của hệ thống.

Bảng 3.1 Chức năng của các thành phần trong khối Zone ECU.

Thành phần	Chức năng
ESP32-WROOM-32	Điều khiển trung tâm, xử lý dữ liệu và giao tiếp với các ngoại vi
HC-SR04	Đo khoảng cách đến vật cản
Mô-đun Ethernet W5500	Truyền thông Ethernet giữa các Zone ECU

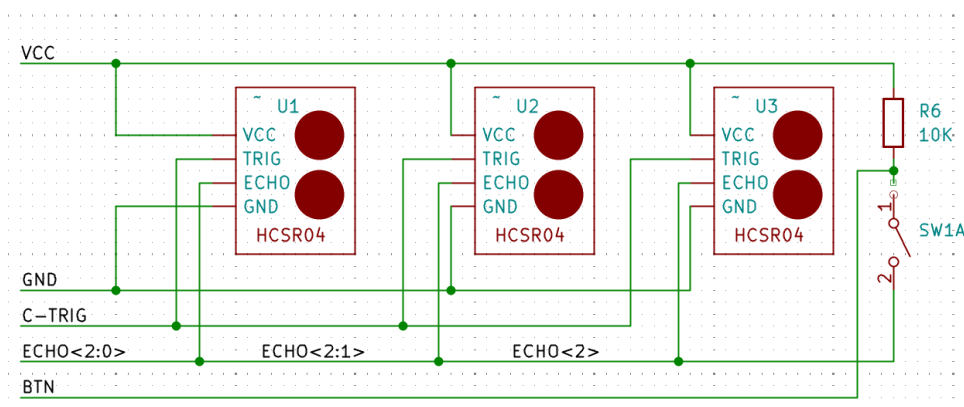
Dual H-Bridge Mini	Điều khiển động cơ DC
GPS ATGM336H (Optional)	Thu nhận thông tin vị trí và thời gian

3.2.2 Khối cảm biến

Khối cảm biến khoảng cách trên các phân vùng bao gồm hai khối chức năng độc lập: F-SD (Front Sensor Distance) bố trí ở vùng đầu xe và B-SD (Back Sensor Distance) bố trí ở vùng đuôi xe. Mỗi khối cảm biến được cấu hình bao gồm ba mô-đun siêu âm HC-SR04 định vị tại các góc: cạnh trái, chính diện, cạnh phải. Cấu trúc này thiết lập một hàng rào giám sát không gian phẳng, cho phép Zonal ECU thu thập liên tục dải khoảng cách tới vật cản xung quanh xe nhằm phục vụ các thuật toán an toàn chủ động (Phanh khẩn cấp, cảnh báo va chạm).

3.2.2.1 Thiết kế dùng chung ngõ vào TRIGGER

Khối cảm biến F-SD, B-SD có thiết kế giống nhau nhưng được đặt ở vị trí khác nhau và cung cấp dữ liệu đầu vào cho các khối xử lý vùng khác nhau. Khối bao gồm các cảm biến khoảng cách sử dụng cảm biến khoảng cách HCSR-04; Ngoài ra, khối cảm biến còn bao gồm nút nhấn để kích hoạt các tính năng nhất định cho việc kiểm thử. Hình bên dưới mô tả khối cảm biến, các chân kích hoạt được nối chung lại với nhau để tiết kiệm các GPIO và tăng tính đồng bộ.



Hình 3.3 Sơ đồ nguyên lý cảm biến F-SD, B-SD.

3.2.2.2 Nguyên lý đo đúng cách

Khối cảm biến F-SD và B-SD sử dụng cảm biến siêu âm HC-SR04 để xác định khoảng cách đến vật cản dựa trên nguyên lý đo thời gian bay (Time of Flight –

ToF) của sóng siêu âm. Cảm biến hoạt động bằng cách phát ra một xung siêu âm tần số khoảng 40 kHz và đo thời gian xung phản xạ. Khi nhận được xung kích hoạt từ chân TRIGGER, cảm biến sẽ phát một chuỗi sóng siêu âm truyền trong không khí theo hướng đo. Khi sóng siêu âm gặp vật cản, tín hiệu sẽ phản xạ ngược trở lại và được thu nhận bởi bộ thu. Bộ vi điều khiển bên trong Zonal ECU sẽ đo chính xác độ rộng xung của chân ECHO là khoảng thời gian từ lúc phát xung 40 kHz đến khi nhận được tín hiệu phản xạ. Độ rộng xung tỷ lệ thuận với quãng đường mà sóng siêu âm đã đi và quay về.

Khoảng cách đến vật cản được tính theo công thức:

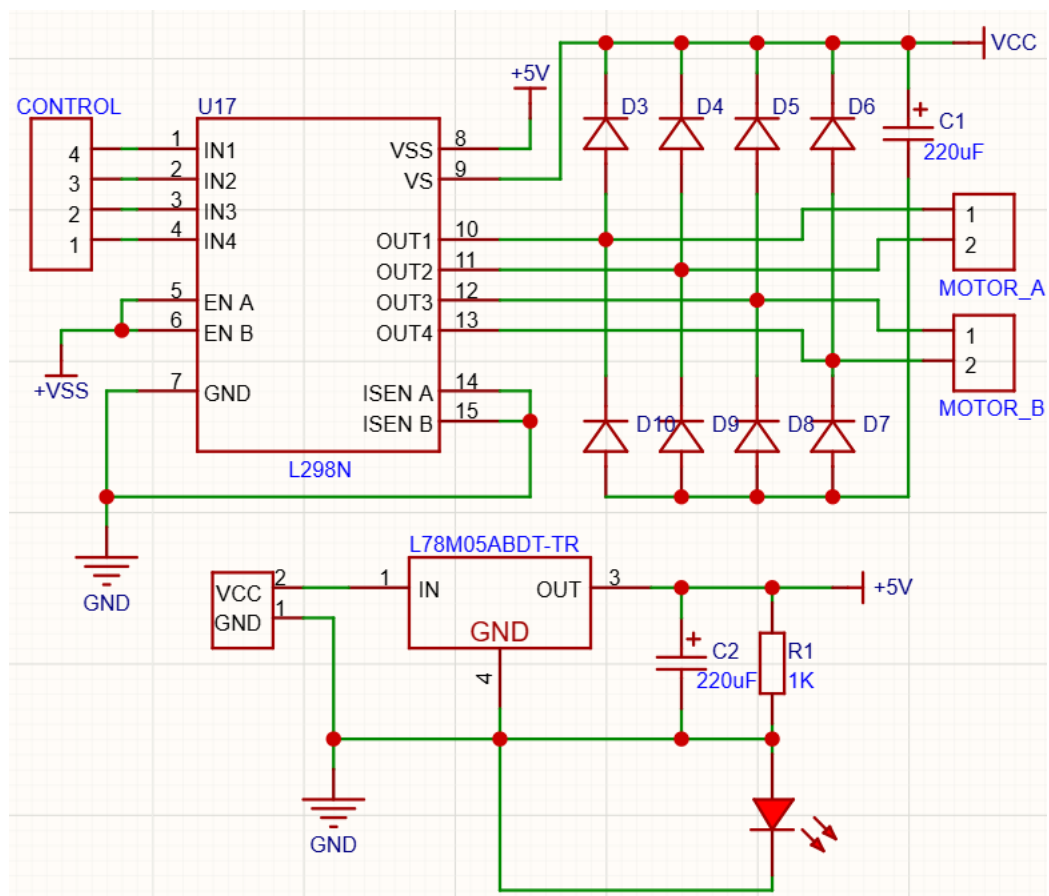
$$d = v_{\text{âm thanh}} \times \frac{t_{\text{độ rộng xung ECHO}}}{2} \quad (3.1)$$

Hệ số chia 2 được sử dụng do sóng siêu âm phải đi hai lần quãng đường (từ cảm biến đến vật cản và quay trở lại). Trong thực tế, hệ thống thường hiệu chỉnh thêm theo nhiệt độ môi trường nhằm tăng độ chính xác của phép đo, do vận tốc truyền âm có thể thay đổi theo điều kiện nhiệt độ và độ ẩm.

Dữ liệu khoảng cách sau khi được tính toán sẽ được Zonal ECU xử lý và lọc nhiễu bằng bộ lọc trung bình cộng trước khi đưa vào các thuật toán điều khiển như cảnh báo va chạm, hỗ trợ phanh khẩn cấp hoặc giám sát vùng an toàn xung quanh xe.

3.2.3 Khối động cơ

Khối động cơ F-DC và B-DC có nhiệm vụ nhận tín hiệu điều khiển từ vi điều khiển ESP32 và cung cấp công suất cần thiết để điều khiển hoạt động của các động cơ DC trong hệ thống. Khối mạch được thiết kế dựa trên IC điều khiển cầu H L298N, cho phép điều khiển đồng thời hai động cơ DC độc lập với khả năng thay đổi chiều quay.



Hình 3.4 Sơ đồ nguyên lý khởi động cơ.

Khởi điều khiển vùng ZECU gửi tín hiệu logic tới các chân điều khiển của L 298N để điều khiển trạng thái hoạt động của động cơ. Xem trạng thái hoạt động của động cơ ở Bảng 3.2.

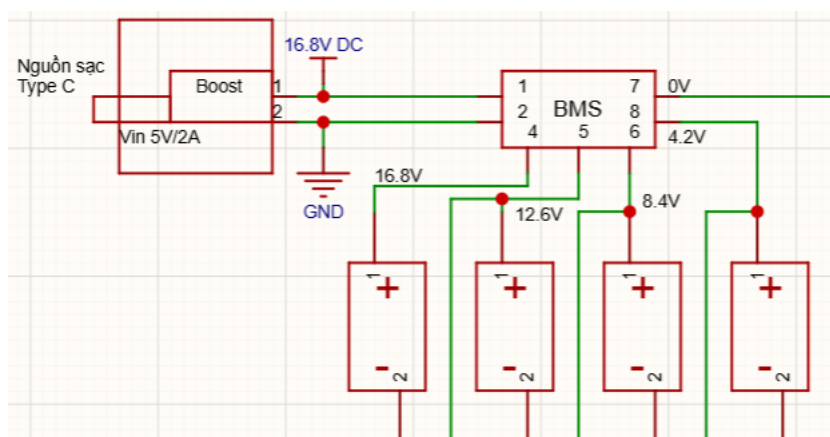
Bảng 3.2 Trạng thái điều khiển chiều quay động cơ.

IN0 (VDC)	IN1 (VDC)	IN2 (VDC)	IN3 (VDC)	Động cơ trái	Động cơ phải
0	0	0	0	Không truyền động.	Không truyền động.
$V_{DC}>0$	$V_{DC}>0$	$V_{DC}>0$	$V_{DC}>0$		
0	$V_{DC}>0$	0	$V_{DC}>0$	Truyền động thuận.	Truyền động thuận.
$V_{DC}>0$	0	$V_{DC}>0$	0	Truyền động nghịch.	Truyền động nghịch.

Trong đó: V_{DC} : là giá trị điện áp trung bình được tạo thông qua điều chế độ rộng xung (PWM) đặt vào các chân điều khiển của mạch cầu H.

3.2.4 Khối nguồn

Khối nguồn (Power Supply - PS) có nhiệm vụ cung cấp điện áp ổn định cho toàn bộ hệ thống, bao gồm các khối điều khiển vùng F-ZECU, B-ZECU, mô-đun giao tiếp Ethernet W5500, các cảm biến khoảng cách và mạch điều khiển động cơ. Do các thiết bị trong hệ thống sử dụng nhiều mức điện áp khác nhau nên khối nguồn được thiết kế theo mô hình chuyển đổi điện áp DC-DC kết hợp với hệ thống quản lý pin (Battery Management System - BMS), đảm bảo độ cân bằng của pin và tăng tính an toàn, an toàn và ổn định trong quá trình vận hành.

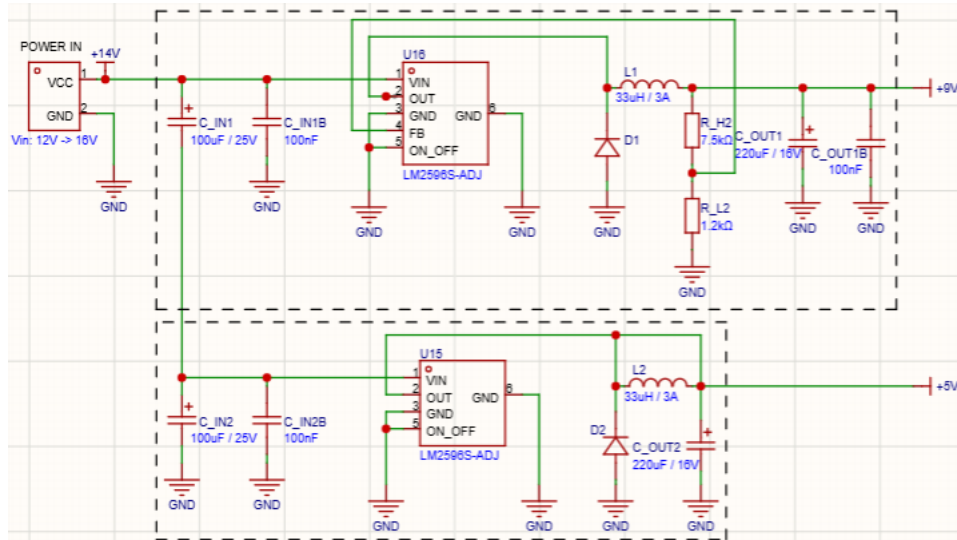


Hình 3.5 Mạch điện pin 4S - BMS.

Nguồn chính của hệ thống được cung cấp từ bộ pin Lithium-ion 4S, gồm bốn cell pin mắc nối tiếp. Mỗi cell có điện áp danh định khoảng 3,7 V và điện áp sạc đầy 4,2 V, do đó điện áp đầu ra của bộ pin dao động trong khoảng 12 V đến 16,8 V tùy theo mức dung lượng còn lại của pin. Để đảm bảo an toàn trong quá trình sử dụng, bộ pin được tích hợp mạch quản lý pin BMS (Battery Management System).

Mạch BMS có nhiệm vụ giám sát điện áp của từng cell, cân bằng điện áp giữa các cell trong quá trình sạc, đồng thời bảo vệ bộ pin trước các tình trạng quá sạc, quá xả, quá dòng và ngắn mạch. Nhờ đó, tuổi thọ của bộ pin được nâng cao, đồng thời đảm bảo nguồn cấp luôn ổn định và an toàn cho toàn bộ hệ thống. Ngoài ra, hệ thống sử dụng mạch Boost Converter kết hợp với nguồn sạc USB Type-C 5 V/2 A để nâng điện áp lên mức phù hợp nhằm sạc bộ pin 4S thông qua mạch BMS.

Sau khi qua mạch BMS, điện áp đầu ra của bộ pin được đưa đến hai bộ chuyển đổi hạ áp sử dụng IC LM2596S-ADJ nhằm tạo ra hai mức điện áp cung cấp cho các khối chức năng trong hệ thống.



Hình 3.6 Sơ đồ nguyên lý khối nguồn PS.

Nguồn đầu vào từ bộ pin có điện áp 12–16,8 V được lọc bởi các tụ điện đầu vào trước khi đưa vào hai bộ chuyển đổi Buck. Bộ chuyển đổi thứ nhất tạo điện áp 9 V để cấp nguồn cho mạch điều khiển động cơ. Bộ chuyển đổi thứ hai tạo điện áp 5 V để cấp nguồn cho mô-đun Ethernet W5500, cảm biến khoảng cách và các thiết bị ngoại vi. Riêng vi điều khiển ESP32 hoạt động ở mức điện áp 3,3 V được cấp bởi bộ ổn áp tích hợp trên bo mạch.

Bảng 3.3 Thông số nguồn cấp của các thiết bị trong hệ thống.

Khối	Nguồn áp (U)	Nguồn dòng (I)
Front ZONE ECU	5V / 3.3V	$\leq 1A$
Back ZONE ECU	5V / 3.3V	$\leq 1A$
Central Switch	5V	$\leq 0.5A$
Driver động cơ	9V	$\leq 2A$

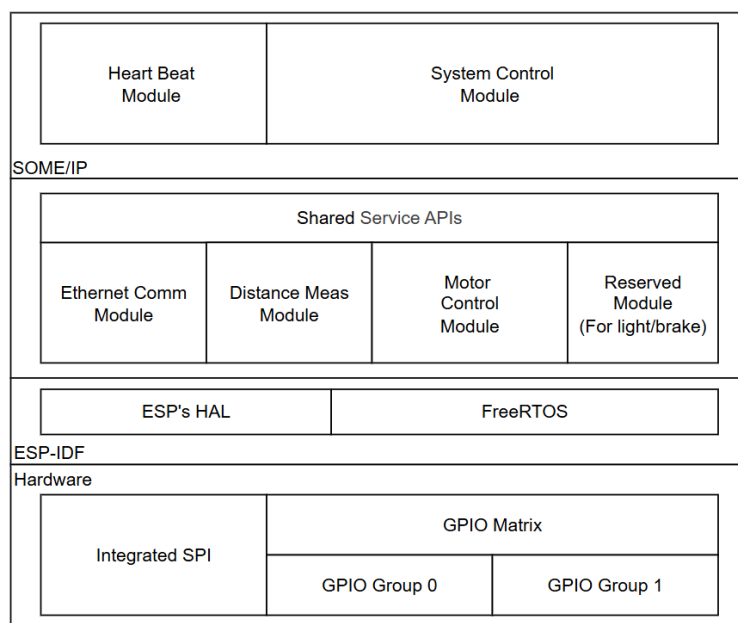
Khối nguồn sử dụng bộ pin Lithium-ion 4S kết hợp với mạch quản lý pin BMS và hai bộ chuyển đổi hạ áp LM2596S-ADJ để tạo các mức điện áp 9 V và 5 V từ nguồn đầu vào 12–16,8 V. Thiết kế này không chỉ đảm bảo cung cấp nguồn ổn

định cho toàn bộ hệ thống mà còn nâng cao độ an toàn, tăng tuổi thọ của bộ pin và tạo điều kiện thuận lợi cho việc mở rộng hệ thống trong tương lai.

3.3 THIẾT KẾ PHẦN MỀM

3.3.1 Sơ đồ phân tầng kiến trúc phần mềm

Kiến trúc phần mềm bên dưới mô tả một hệ thống nhúng phân tầng, dựa trên nền tảng vi điều khiển ESP (ESP-IDF). Cấu trúc này được thiết kế theo hướng Dịch vụ hóa với 4 lớp xếp chồng. Lớp thấp nhất trực tiếp quản lý tài nguyên phần cứng như khối GPIO và bộ điều khiển SPI. Kế đến là lớp nền tảng với hệ điều hành thời gian thực FreeRTOS và giao diện trừu tượng phần cứng (HAL). Lớp giữa đóng vai trò nòng cốt, cung cấp các API dịch vụ dùng chung cho các Dịch vụ chức năng cơ bản (giao tiếp Ethernet, đo khoảng cách, điều khiển động cơ). Tầng trên cùng là lớp ứng dụng chịu trách nhiệm quản lý hệ thống, giao tiếp nội bộ thông qua giao thức SOME/IP.



Hình 3.7 Cấu trúc phân tầng của firmware cho ZECU.

Mỗi dịch vụ đều phải tuân thủ theo một cấu trúc chung, bao gồm 03 thành phần chính:

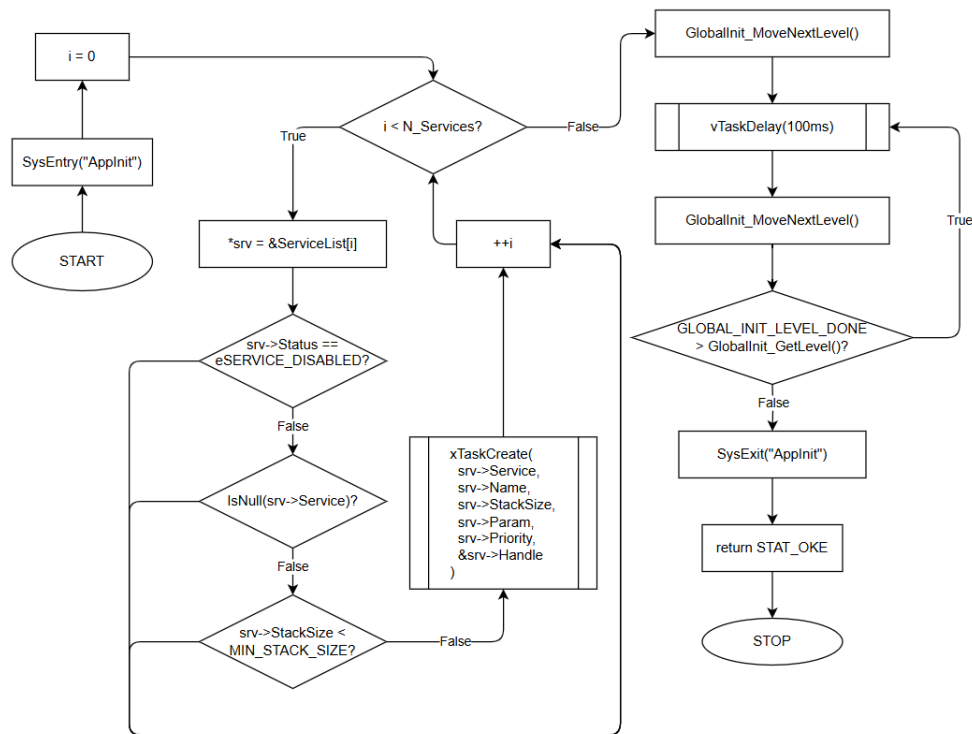
- Giao diện lập trình ứng dụng (API - Application Programming Interface): Đây là cổng giao tiếp công khai (public interface) của dịch vụ. Thành phần này bắt buộc phải cung cấp bộ API chuẩn (bao gồm các hàm cơ bản như khởi tạo, đọc,

ghi trạng thái) nhằm cho phép các dịch vụ hoặc tầng ứng dụng khác trong hệ thống có thể tương tác, điều khiển và trao đổi dữ liệu một cách an toàn mà không làm phá vỡ tính đóng gói của mã nguồn.

- Tác vụ thời gian thực (Runtime Task / Background Service): Đây là một tiến trình (task) hoạt động độc lập dưới sự điều phối của hệ điều hành thời gian thực (cụ thể trong đồ án là FreeRTOS). Tác vụ này đóng vai trò như "trái tim" của dịch vụ, chủ động gọi các API dùng chung và API nội bộ để giám sát, xử lý luồng dữ liệu và quản lý trạng thái. Bên cạnh đó, nó còn chịu trách nhiệm quản lý vòng đời hoạt động: định kỳ đánh thức hoặc kiểm tra trạng thái của các thành phần phụ trợ tuân theo chu kỳ đã được thiết lập.
- Lớp trừu tượng hóa phần cứng (HAL - Hardware Abstraction Layer): Tùy thuộc vào chức năng đặc thù của từng dịch vụ mà lớp HAL sẽ được thiết kế tương ứng. Thành phần này có nhiệm vụ cung cấp phương thức giao tiếp ở mức thấp nhằm tương tác trực tiếp với các bộ ngoại vi vật lý (như GPIO, SPI, PWM,...). Việc triển khai lớp HAL giúp che giấu đi sự phức tạp của kiến trúc thanh ghi, tạo ra một lớp đệm chuẩn hóa cho tầng điều khiển phía trên. Điều này không chỉ đảm bảo tính độc lập của logic chương trình mà còn tối ưu hóa khả năng tái sử dụng (portability) khi có sự thay đổi về nền tảng vi điều khiển.

3.3.2 Bộ khởi tạo dịch vụ

Bộ khởi tạo sẽ được gọi sau khi các phần cứng khởi tạo xong. Nhiệm vụ của bộ khởi tạo sẽ khởi tạo các dịch vụ được lên lịch trước. Các dịch vụ sẽ được khởi tạo theo thứ tự trong danh sách dịch vụ, tuy nhiên, các Task sau khi khởi chạy vẫn sẽ phải chờ cho đến lượt thông qua cơ chế khởi tạo theo cấp. Bộ khởi tạo sẽ được gán cấp 0 (cao nhất), các dịch vụ/Dịch vụ khác sẽ có các level khác nhau. Sau khi khởi tạo xong toàn bộ dịch vụ trong danh sách, bộ khởi tạo sẽ tăng level khởi tạo mỗi 100ms, cho đến 25.5s thì sẽ dừng. Điều này làm cho các dịch vụ có độ ưu tiên cao sẽ có thời gian khởi tạo.



Hình 3.8 Lưu đồ quá trình khởi tạo các mô-đun.

3.3.3 Dịch vụ giao tiếp mạng

3.3.3.1 Thiết kế tầng giao tiếp phần cứng

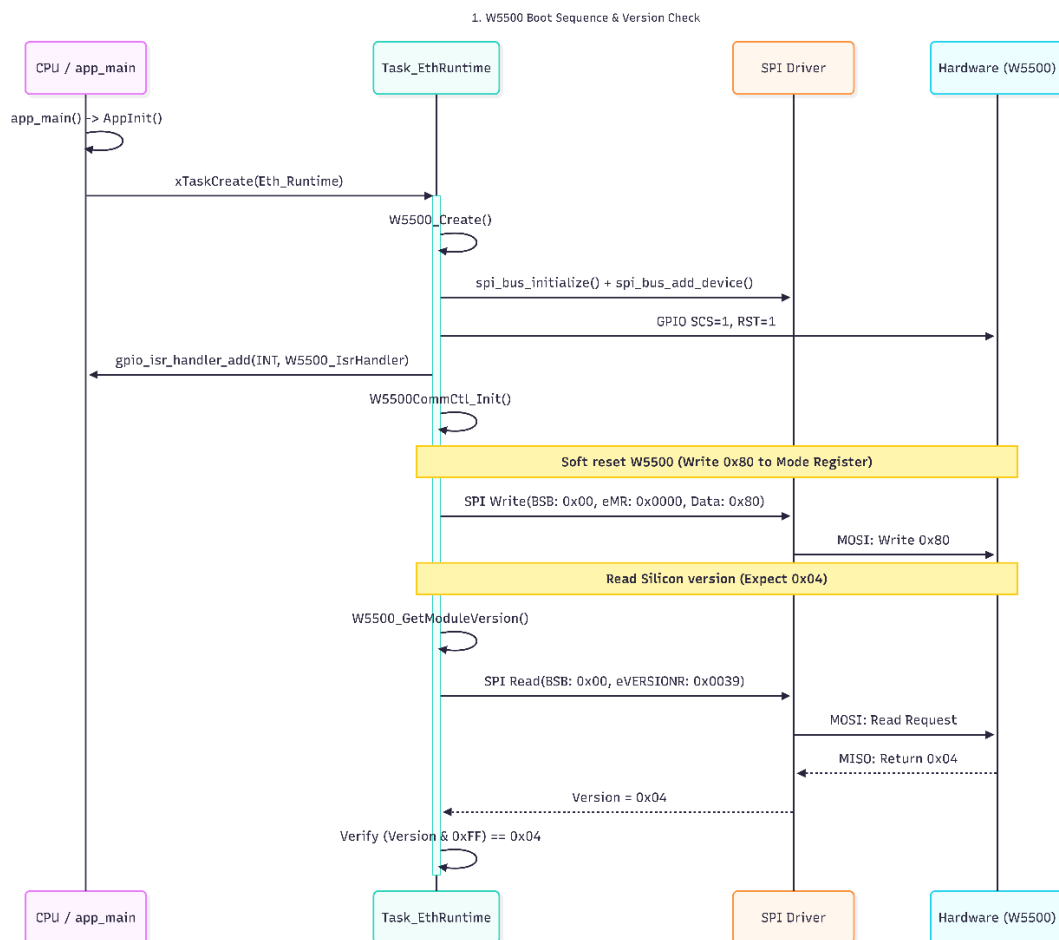
Để đảm bảo tính độc lập của mã nguồn phần mềm đối với phần cứng (Hardware Abstraction Layer - HAL), mô-đun giao tiếp Ethernet được thiết kế đóng gói hoàn toàn các tác vụ giao tiếp bus SPI cấp thấp. Quá trình khởi tạo được thực hiện thông qua hàm W5500CommCtl_Init với các bước được mô tả trong bảng bên dưới:

Bảng 3.4 Quá trình khởi tạo mô-đun giao tiếp Ethernet.

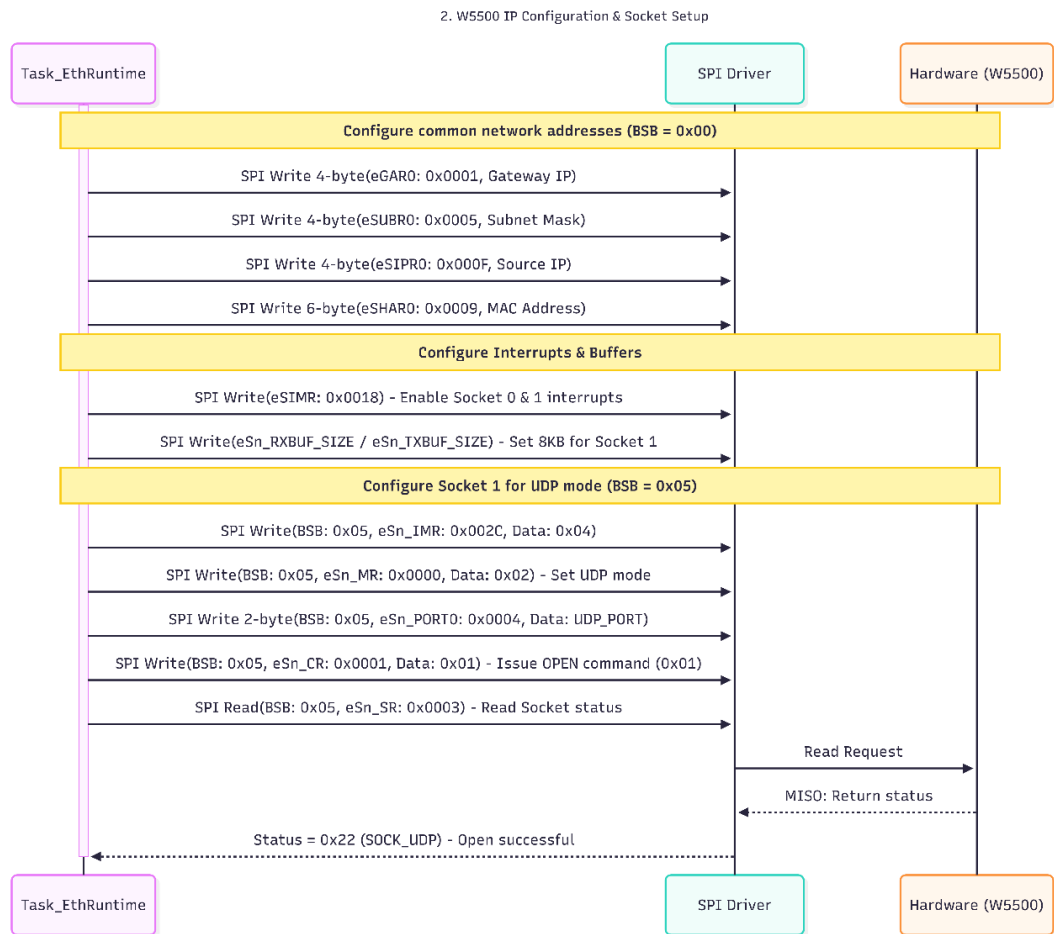
Bước	Nội dung	Mô tả
1	Kiểm tra phiên bản phần cứng	Đọc thanh ghi phiên bản (Version Register) của chip W5500 thông qua SPI để đảm bảo tính tương thích phần cứng trước khi cấu hình.

2	Cấu hình mạng tĩnh	Ghi các giá trị địa chỉ vật lý, địa chỉ mạng, mặt nạ mạng và Gateway vào các thanh ghi chung.
3	Phân bổ bộ đệm phần cứng	Thiết lập Socket 1 phục vụ giao tiếp UDP.

Sơ đồ trong Hình 3.9, Hình 3.10 bên dưới mô tả chi tiết quá trình khởi tạo mô-đun giao tiếp Ethernet:



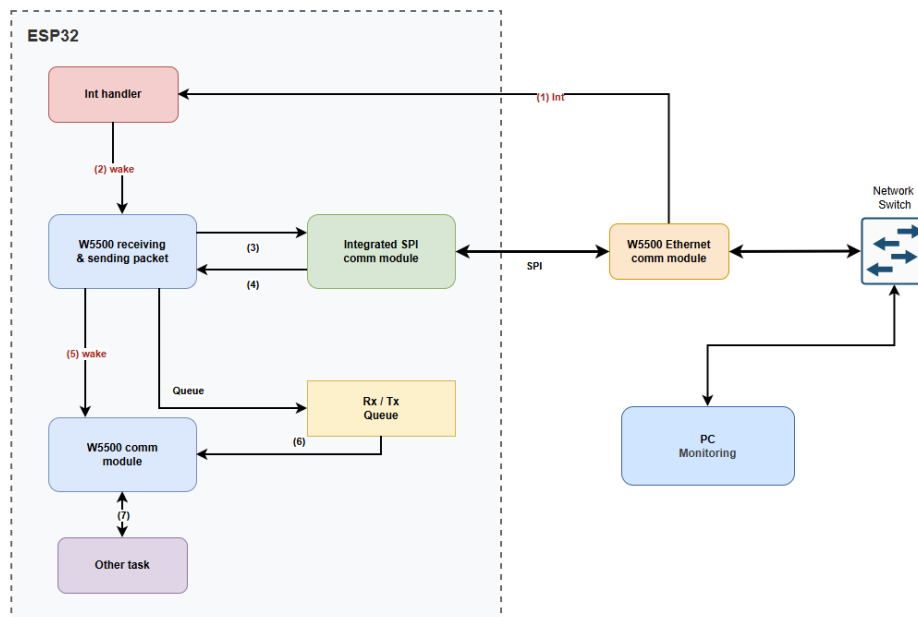
Hình 3.9 Quá trình định danh mô-đun giao tiếp Ethernet.



Hình 3.10 Quá trình định danh mô-đun giao tiếp Ethernet.

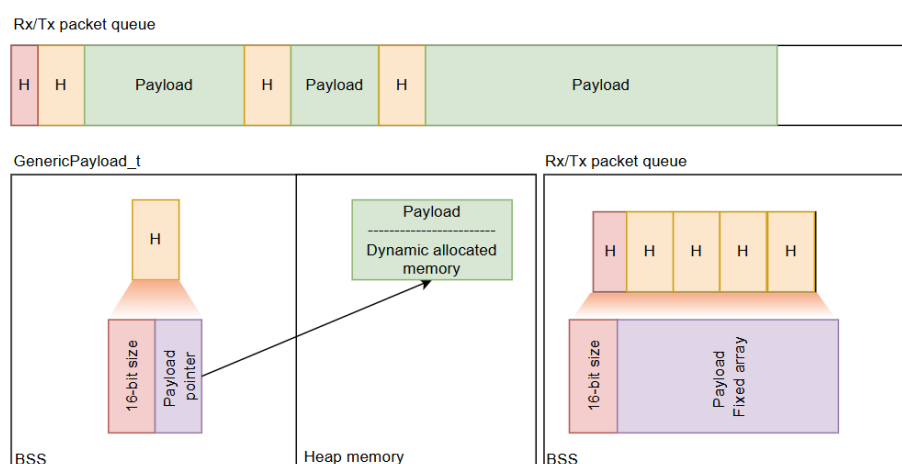
3.3.3.2 Cơ chế xử lý ngắt và quản lý hàng đợi gói tin

Tốc độ và số lượng các gói tin có thể trở nên lớn, nếu chỉ xử lý ngay lập tức cái gói tin ngay khi nhận có thể làm các gói tin đến sau bị bỏ lỡ hoặc bị bỏ vì không được nhận. Do đó một hàng đợi – bộ đệm được thêm vào cho quá trình gửi và nhận. Các gói ngay khi đến mô-đun giao tiếp Ethernet sẽ được di chuyển vào hàng đợi để xử lý. Hình 3.11 mô tả quá trình một gói tin được xử lý từ khi có tín hiệu ngắt (1) đến khi đến tần ứng dụng (7).



Hình 3.11 Quá trình xử lý gói tin.

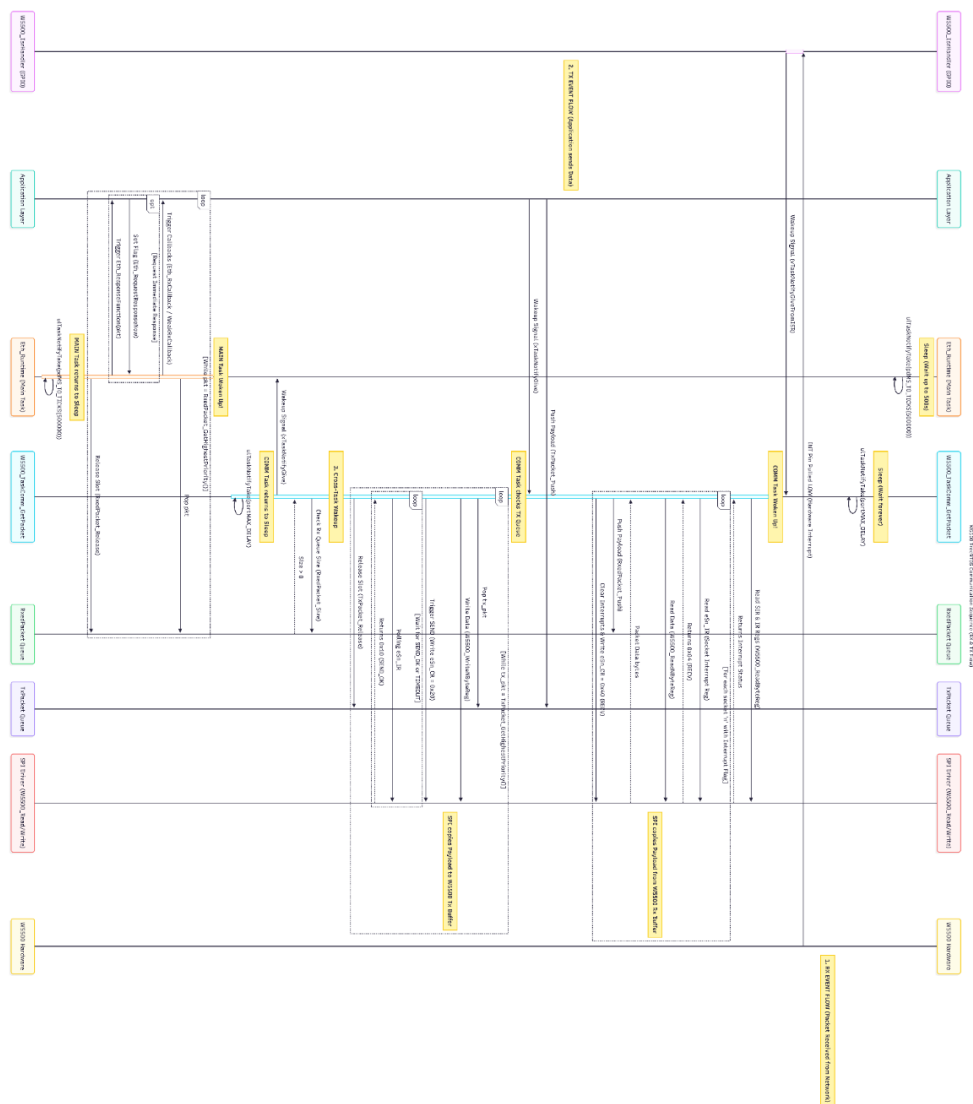
Cơ chế hàng đợi gói tin cho phép các phần tử trong hàng đợi có kích thước linh hoạt. Một mảng phần tử rỗng chứa các thông tin về phần dữ liệu bên trong (header) được khởi tạo trước và lưu vào trong vùng nhớ BSS. Trong quá trình hoạt động, khi nhận một gói tin từ mô-đun giao tiếp Ethernet, một phân vùng nhớ chứa gói tin sẽ được cấp phát động trong vùng nhớ heap để chứa gói tin và được giải phóng sau khi gói tin được xử lý. Hình bên dưới mô tả hàng đợi gửi/nhận và triển khai của hai hàng đợi này.



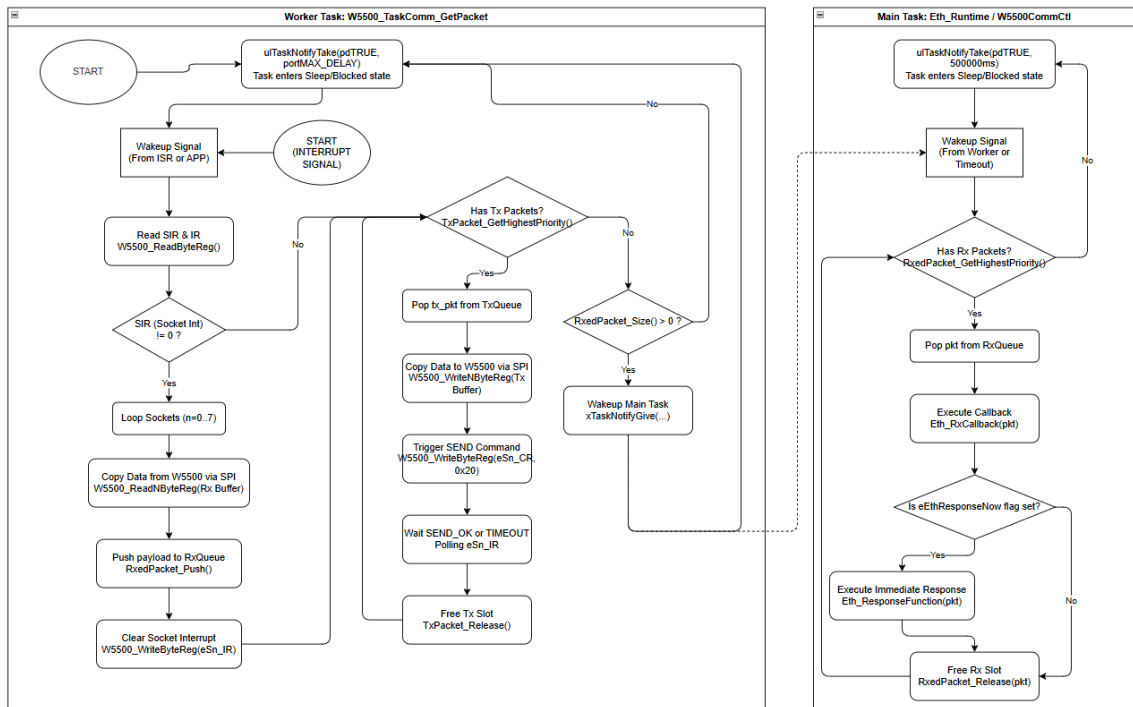
Hình 3.12 Hàng đợi gửi và nhận gói tin.

3.3.3.3 Cơ chế gửi và nhận gói tin Ethernet

Dịch vụ giao tiếp mạng là một trong các dịch vụ lõi của bộ xử lý vùng – ZECU. Bên ngoài, dịch vụ giao tiếp với các thiết bị khác thông qua chuẩn Ethernet, bên trong, dự vụ cung cấp các API giao tiếp giữa các dịch vụ khác. Một gói tin Ethernet có thể liên quan đến nhiều dịch vụ khác. Dịch vụ giao tiếp mạng sử dụng cơ chế ngủ/đánh thức (Sẽ được giới thiệu trong mục 3.3.7.3) để tăng tính ổn định của dịch vụ. Ngoài tác vụ chính để thực hiện các hành động gửi/nhận gói tin, bên trong dịch vụ còn có các tác vụ con thực hiện việc truyền tải dữ liệu đến mô-đun Ethernet W5500 trong khi tác vụ chính có thể xử lý các yêu cầu khác. Hình bên dưới mô tả quá trình gửi/nhận gói tin Ethernet.



Hình 3.13 Sơ đồ chuỗi gọi và nhận dữ liệu sử dụng cơ chế ngủ/đánh thức.



Hình 3.14 Lưu đồ gọi và nhận dữ liệu sử dụng cơ chế ngủ/đánh thức.

Đối với luồng nhận, khi có ngắt phần cứng xảy ra, Worker Task lập tức thức giấc, lấy gói tin từ W5500 qua giao thức SPI và nạp vào hàng đợi RxQueue. Kế tiếp, Worker Task phát tín hiệu đánh thức Main Task. Lúc này, Main Task sẽ rút gói tin từ hàng đợi, thực thi các hàm callback xử lý gói tin nhận được và cuối cùng là giải phóng bộ nhớ.

Đối với luồng truyền, Worker Task chủ động quét kiểm tra TxQueue. Nếu có gói tin chờ gửi, Worker Task sẽ ghi dữ liệu xuống W5500, kích hoạt lệnh truyền (SEND) và chờ trạng thái xác nhận (SEND_OK hoặc TIMEOUT) trước khi giải phóng khe cắm trong hàng đợi. Cách thiết kế này giúp tách biệt hoàn toàn tầng giao tiếp vật lý và logic ứng dụng, đảm bảo CPU không bị nghẽn (blocking).

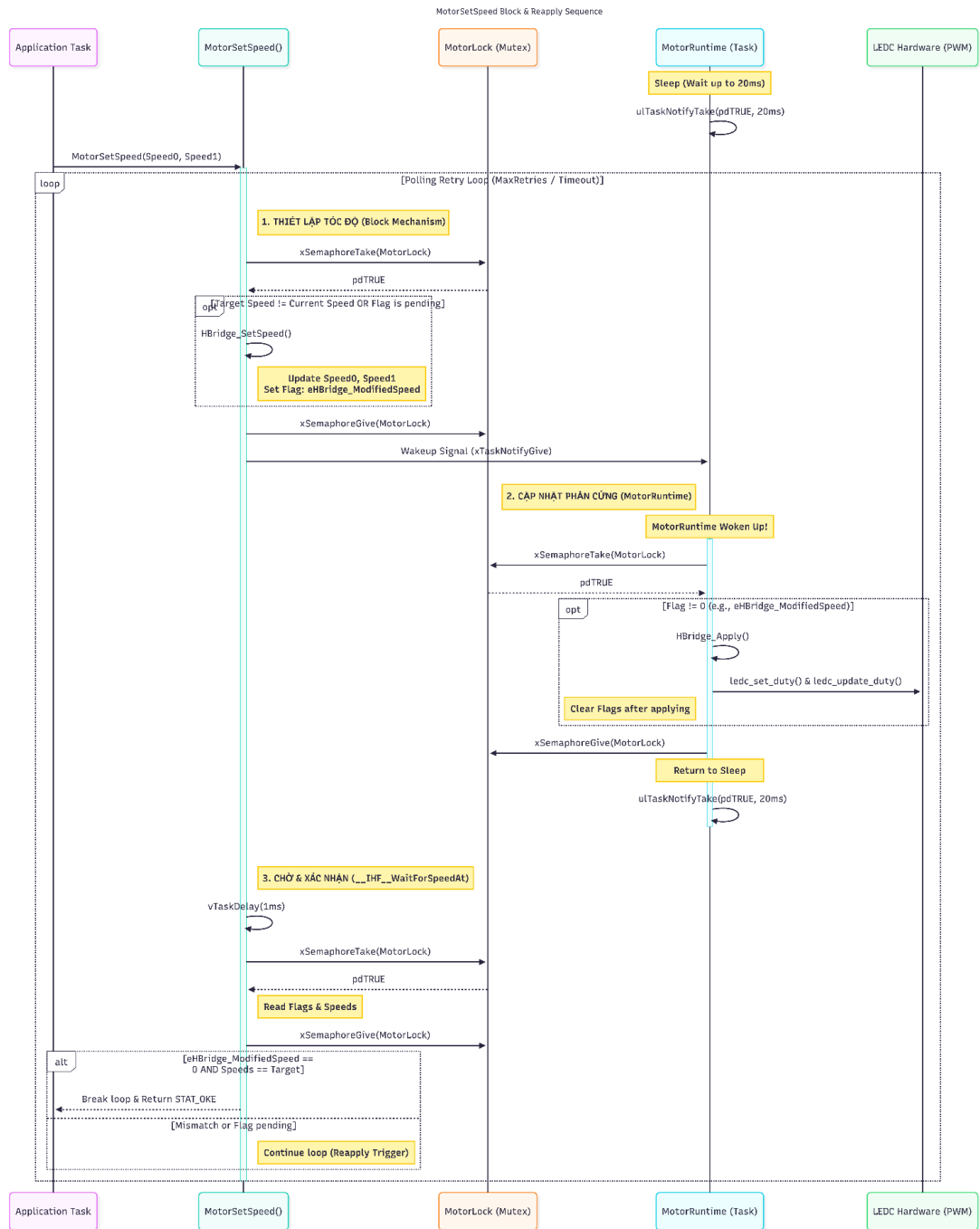
3.3.4 Dịch vụ điều khiển động cơ

Khối động cơ sử dụng Driver cầu H để điều khiển động cơ thông qua bộ ngoại vi LEDC (LED Control) của vi điều khiển ESP32, cấu hình hoạt động ở chế độ tần số thấp với tần số phát xung PWM cố định là 1000 Hz. Bộ điều khiển sử dụng độ phân giải băm xung 10-bit, tương ứng với dải giá trị chu kỳ nhiệm vụ (Duty Cycle) vật lý từ 0 đến 1023 đơn vị. Sơ đồ phần cứng phân bổ 4 kênh điều khiển độc lập

cho hai động cơ: Động cơ 0 điều phối qua chân ngõ ra IN0 (GPIO 4) và IN1 (GPIO 2), trong khi Động cơ 1 sử dụng chân IN2 (GPIO 16) và IN3 (GPIO 17). Giải thuật tính toán Duty Cycle thực hiện phân tích dấu của giá trị tốc độ có dấu 16-bit (dải đầu vào từ -1023 đến 1023) để kiểm soát chiều quay của động cơ; khi tốc độ nhận giá trị dương (quay thuận), xung PWM tương ứng sẽ được áp dụng trực tiếp lên chân điều khiển chiều thuận (IN0 hoặc IN2) và giữ chân ngược lại (IN1 hoặc IN3) ở mức logic thấp (0V) để tạo chênh lệch điện áp, và ngược lại khi giá trị tốc độ âm (quay nghịch) để đảo chiều dòng điện qua cầu H một cách an toàn và chính xác.

Nhằm đảm bảo tính toàn vẹn dữ liệu và an toàn luồng (Thread-safety) trong môi trường đa tác vụ FreeRTOS, quyền truy cập vào cấu trúc dữ liệu của động cơ được đồng bộ hóa nghiêm ngặt bằng cơ chế Mutex thông qua khóa MotorLock. Khi API MotorSetSpeed được gọi để cập nhật tốc độ, hệ thống sử dụng cơ chế chặn (blocking execution) kết hợp gửi thông báo trực tiếp qua xTaskNotifyGive nhằm đánh thức ngay lập tức tác vụ nền MotorRuntime để đẩy cấu hình Duty Cycle mới xuống thanh ghi phần cứng mà không phải chờ hết chu kỳ quét mặc định. Bên cạnh đó, hệ thống tích hợp giải thuật kiểm soát phản hồi và áp dụng lại (setspeed block/reapply) thông qua hàm hỗ trợ __IHF__WaitForSpeedAt; Hàm này duy trì vòng lặp kiểm tra trạng thái thực tế sau mỗi chu kỳ phản hồi ngắn (100ms hoặc cấu hình thời gian thực) và tự động ghi đè/áp dụng lại lệnh điều khiển nếu có sự sai lệch giữa tốc độ thực tế của động cơ và tốc độ đích, kéo dài cho đến khi ghi nhận trạng thái đồng bộ hoàn toàn hoặc chạm ngưỡng giới hạn thời gian chờ (timeout), giúp tối ưu hóa khả năng phản hồi và ngăn ngừa các lỗi mất mát khung truyền hoặc trôi tốc độ do nhiễu phần cứng.

Hình 3.15 mô tả quá trình thiết lập tốc độ cho động cơ.



Hình 3.15 Quá trình thiết lập tốc độ động cơ.

3.3.5 Dịch vụ giám sát hệ thống

3.3.5.1 Giám sát trạng thái cảm biến

Hệ thống thực hiện thu thập liên tục dữ liệu từ các cảm biến siêu âm thông qua các hàm đọc chuyên dụng, sau đó chuyển đổi đơn vị đo từ milimet sang centimet và giới hạn trong phạm vi 10-bit để tối ưu hóa không gian lưu trữ. Đồng thời, trạng

thái thực tế của hai động cơ được truy xuất qua các hàm (như MotorGetSpeed0) và quy đổi từ giá trị thô sang tỉ lệ phần trăm (từ -100% đến 100%) thông qua hàm ConvertRaw2Percent. Toàn bộ dữ liệu này được tổng hợp và sẵn sàng để đóng gói truyền tải.

3.3.5.2 Quản lý trạng thái cảm biến

Cơ chế đồng bộ được thực thi thông qua tác vụ chạy nền HeartBeatRuntime, hoạt động theo chu kỳ định thời (mặc định 300ms) hoặc khi có sự kiện đánh thức. Khung dữ liệu ZECUFrame_Generic_t đóng gói trạng thái cảm biến, động cơ, định vị GPS và kèm theo bộ đếm đồng bộ (sync_num) xoay vòng liên tục giúp CCU kiểm soát thứ tự gói tin. Ngoài ra, hệ thống hỗ trợ cơ chế phản hồi tức thời (App_SendImmediateHeartbeat), cho phép gửi gói tin Heartbeat ngay lập tức khi trạng thái thay đổi mà không cần đợi hết chu kỳ định thời.

3.3.5.3 Khẩn cấp

Cơ chế an toàn được tích hợp chặt chẽ trong vòng lặp giám sát để phát hiện và xử lý sự cố theo thời gian thực. Cụ thể, hệ thống liên tục so sánh khoảng cách đo được từ cảm biến siêu âm với tập ngưỡng an toàn (SF_ETT_Distance). Nếu phát hiện vật cản ở cự ly nguy hiểm, hoặc khi nhận được khung lệnh eFrameType_EmergencyStop từ mạng, hệ thống lập tức gọi hàm EmergencyStop() để khóa toàn bộ động cơ. Hơn nữa, phím cứng (BOOT pin) cũng được thiết lập để cho phép người dùng buộc ECU thoát khỏi trạng thái lỗi và khởi tạo lại kết nối mạng.

3.3.5.4 Điều khiển động cơ

Chức năng điều khiển tiếp nhận và phân tích các khung lệnh (EngineControl 1, 2, 3) từ CCU thông qua quá trình ParsePacket. Giá trị điều khiển nhận được dưới dạng phần trăm sẽ được hàm ConvertPercent2Raw dịch ngược thành giá trị thô và đẩy xuống phần cứng thông qua hàm MotorSetSpeed. Phương thức điều khiển này nổi bật với thiết kế phản hồi khép kín (sendback); ngay sau khi áp dụng lệnh mới, hệ thống tự động thiết lập cờ Eth_RequestResponseNow() nhằm ép ECU

gửi ngay một bản tin Heartbeat mang trạng thái cập nhật, giúp CCU xác nhận lệnh đã được thực thi chính xác.

3.3.5.5 Ghép nối CCU

Giai đoạn quảng bá: Quá trình ghép nối giữa thiết bị đầu cuối và CCU diễn ra thông qua một máy trạng thái tuần tự, bắt đầu từ việc phát sóng tìm kiếm. Khi chưa có thông tin đối tác, thiết bị liên tục phát đi các gói tin UDP vô hướng để quảng bá sự hiện diện của dịch vụ. Máy trạng thái sẽ tự động chuyển tiếp qua các bước xác thực khi có đối tác phản hồi. Đặc biệt, thông tin ghép nối thành công sẽ được lưu trữ phi tĩnh (NVS) nhằm tự động phục hồi kết nối sau mỗi lần khởi động lại.

Giai đoạn RequestPairing: Giai đoạn này được kích hoạt ngay khi ECU nhận được khung tin yêu cầu ghép nối hợp lệ từ CCU nhằm đáp lại gói quảng bá ban đầu. Hệ thống sẽ tiến hành phân tích khung tin, trích xuất địa chỉ IP mạng và cổng giao tiếp động của CCU để lưu trữ vào cấu trúc định tuyến nội bộ. Sự kiện này chính thức chấm dứt trạng thái phát sóng vô hướng, chuyển hệ thống sang giai đoạn xác thực bảo mật và thiết lập một kênh truyền thông trực tiếp, nhằm mục tiêu chuẩn xác.

Giai đoạn gửi thách thức (AuthTx): Sau khi nhận yêu cầu định tuyến, thiết bị chuyển sang trạng thái xác thực và đóng vai trò là bên khởi tạo thách thức. Thuật toán tiến hành sinh ra một giá trị ngẫu nhiên duy nhất (Challenge), đóng gói vào khung tin bảo mật AuthTX và đẩy trực tiếp đến địa chỉ mạng của CCU. Tiếp đó, hệ thống bước vào vòng lặp chờ với một giới hạn thời gian (timeout) nghiêm ngặt; nếu CCU không kịp phản hồi, toàn bộ quá trình ghép nối bị hủy bỏ nhằm ngăn chặn các kết nối treo hoặc không xác định.

Giai đoạn xác thực phản hồi (AuthRx): Giai đoạn cuối cùng của quá trình bắt tay mạng là tiếp nhận và xử lý gói tin phản hồi AuthRX từ đối tác. Khi nhận được khung tin, hệ thống sẽ trích xuất phần chữ ký số và sử dụng khóa gốc bảo mật cốt lõi để đối chiếu với giá trị thách thức đã gửi trước đó. Nếu chữ ký hoàn toàn khớp, quá trình xác thực thành công, thiết bị chuyển sang trạng thái kết nối hoàn toàn để trao đổi dữ liệu; ngược lại, hệ thống sẽ lập tức từ chối truy cập để đảm bảo tính an toàn nội mạng.

3.3.6 Cấu trúc phân bổ các dịch vụ

Các dịch vụ được định nghĩa trước bởi struct *AppService_t*, có thể bật hoặc tắt trong quá trình hoạt động. Bảng bên dưới mô tả các trường giá trị định nghĩa một dịch vụ thông qua *AppService_t*. Các dịch vụ có thể dễ dàng được thêm vào và khởi chạy thông qua danh sách các dịch vụ.

Bảng 3.5 Các trường giá trị của *AppService_t*.

Kiểu dữ liệu	Tên trường	Giá trị mặc định	Chức năng
char* [50 byte]	Name	“\0” (Chuỗi rỗng)	Tên dịch vụ.
void * (void *)	Service	NULL	Con trỏ đến dịch vụ.
int32_t	StackSize	2048	Kích thức ngăn xếp trong RAM.
void*	Param	NULL	Đối số cho dịch vụ.
int32_t	Priority *	eTask_Background	Mức độ ưu tiên của dịch vụ.
TaskHandle_t	Handle **	NULL	Biến quản lý tác vụ.

Chú thích:

- *: Mức độ ưu tiên được chia làm 06 cấp, bảng <> mô tả các mức độ ưu tiên của một dịch vụ.
- **: Biến quản lý tác vụ cho phép một dịch vụ có thể được đánh thức.
- ***: Biến trạng thái dịch vụ cho phép một dịch vụ có thể bị vô hiệu hoá hoặc tái kích hoạt trong lúc hoạt động mà không cần phải cập nhật lại firmware.

Bảng 3.6 Mức độ ưu tiên của dịch vụ.

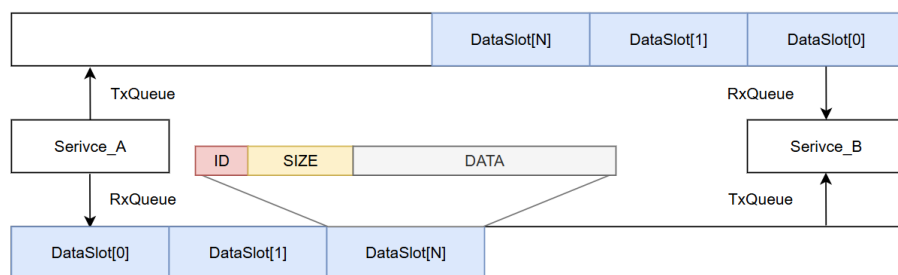
Độ ưu tiên	Chú thích
eTask_Background	Tác vụ nền: Dùng cho các công việc không đòi hỏi thời gian khắt khe như ghi nhật ký hệ thống, hoặc giám sát hệ thống.
eTask_Low	Tác vụ ít quan trọng: Xử lý các dịch vụ phụ trợ, không thiết yếu hoặc cập nhật trạng thái ngoại vi cơ bản.

eTask_Normal	Tác vụ thông thường: Dành cho các logic ứng dụng chung.
eTask_High	Tác vụ ưu tiên cao: Dành cho các tiến trình cần độ trễ thấp như đọc/lấy mẫu cảm biến liên tục hoặc điều khiển động cơ.
eTask_Urgency	Tác vụ cực kỳ quan trọng: Xử lý các chức năng mang tính sống còn của hệ thống như kiểm tra an toàn hoặc gửi/nhận tín hiệu duy trì kết nối mạng.
eTask_RealTime	Tác vụ thời gian thực: Yêu cầu CPU phải đáp ứng ngay lập tức. Dành cho các phản hồi phần cứng khẩn cấp ở cấp độ ngắt hoặc gần với ngắt nhất.

3.3.7 Cơ chế đồng bộ và giao tiếp giữa các dịch vụ

3.3.7.1 Cơ chế hàng đợi (Queue/Message)

Do tính chất chuyển đổi ngữ cảnh liên tục của FreeRTOS, các tác vụ có thể bị “sleep” đột ngột. Cơ chế hàng đợi sẽ giúp quá trình giao tiếp giữa các dịch vụ được toàn vẹn. Hàng đợi hoạt động theo nguyên lý “vào trước – ra trước” (FIFO). Hình bên dưới mô tả cơ chế hàng đợi giữa hai dịch vụ.



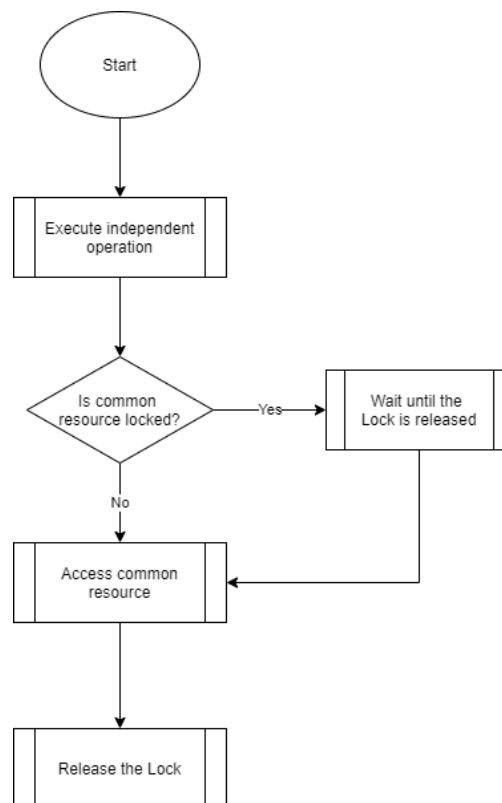
Hình 3.16 Sơ đồ cơ chế giao tiếp hàng đợi giữa hai dịch vụ.

Cấu trúc của một gói tin truyền tải trong hệ thống được thiết kế bao gồm ba thành phần chính là ID, SIZE và DATA, mang lại tính định tuyến và linh hoạt. Trong đó, trường ID đóng vai trò nhận diện loại thông điệp hoặc nguồn gốc của dịch vụ gửi, giúp dịch vụ đích biết chính xác cần áp dụng kịch bản nào để xử lý luồng dữ liệu này. Trường SIZE làm nhiệm vụ xác định kích thước thực tế của phần dữ liệu theo sau, cho phép hệ thống tối ưu hóa không gian bộ nhớ khi truyền

các thông điệp có độ dài ngắn khác nhau thay vì phải cấp phát một khung nhớ cố định gây lãng phí. Cuối cùng, phần DATA là nơi chứa nội dung cốt lõi cần truyền đạt. Việc đóng gói dữ liệu thành một bộ khung chuẩn mực này giúp quá trình phân tách (parsing) ở phía nhận diễn ra chính xác, đảm bảo tính toàn vẹn của luồng thông tin ngay cả khi hệ thống liên tục xảy ra các thao tác chuyển đổi ngữ cảnh phức tạp.

3.3.7.2 Cơ chế khóa (Mutex)

Trong quá trình giao tiếp, hoạt động, các dịch vụ có thể sẽ phải dùng chung một số tài nguyên. Hình 3.17 cho thấy hai dịch vụ A và B dùng chung hai hàng đợi để giao tiếp, tại một thời điểm bất kỳ, chỉ có một dịch vụ duy nhất được truy cập vào hàng đợi (bao gồm thao tác đọc/ghi). Do đó cần một cơ chế khoá để đảm bảo tính chất trên. Hình bên dưới mô tả quá trình truy cập tài nguyên chung có khoá.



Hình 3.17 Lưu đồ cơ chế truy cập tài nguyên chung có khóa.

Quá trình này diễn ra một cách tuần tự: ban đầu, mỗi tác vụ sẽ tự do thực thi các khối lệnh độc lập của riêng nó. Tuy nhiên, ngay trước thời điểm cần can thiệp vào tài nguyên dùng chung, hệ thống buộc tác vụ đó phải thực hiện một bước kiểm

tra trạng thái khoá. Nếu tài nguyên đang bị một tiến trình khác chiếm giữ, tác vụ hiện tại sẽ chủ động tạm dừng và rơi vào trạng thái “Chờ” cho đến khi “ổ khoá” này được mở. Ngược lại, nếu không có ai đang sử dụng, tác vụ sẽ lập tức giành quyền truy cập, hoàn tất các thao tác xử lý dữ liệu và cuối cùng là chủ động giải phóng khoá.

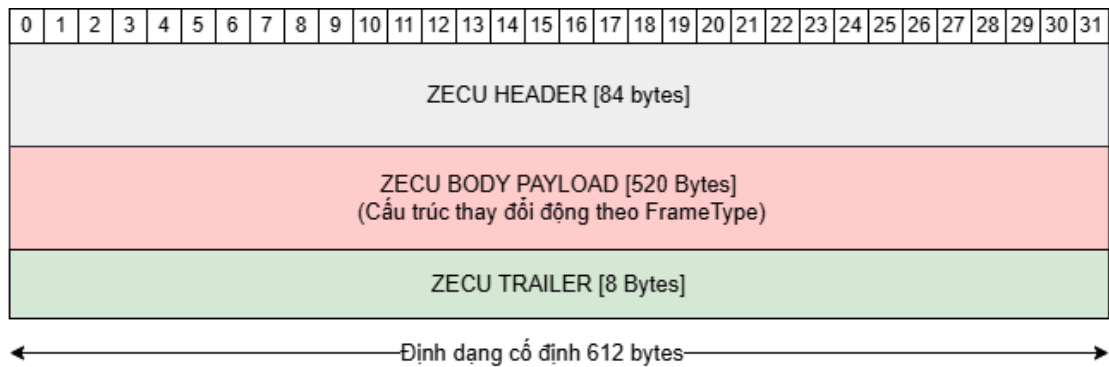
3.3.7.3 Cơ chế ngủ/đánh thức (Sleep/Wakeup)

Cơ chế ngủ - đánh thức (Sleep/Wakeup) được thiết kế đặc thù cho các tiến trình phụ trợ phân nhánh từ một tác vụ chính, hoặc các dịch vụ không yêu cầu thực thi theo chu kỳ cố định. Ngay sau khi khởi tạo hoặc khi đã hoàn tất quá trình xử lý, các dịch vụ này sẽ chủ động chuyển sang trạng thái ngủ (sleep) để nhường quyền điều khiển CPU cho các tiến trình khác. Chúng sẽ duy trì ở trạng thái chờ và chỉ “thức giấc” khi nhận được tín hiệu kích hoạt từ hệ thống. Cơ chế này giúp triệt tiêu các khoảng thời gian rỗi (idle) vô ích, từ đó tối ưu hóa hiệu suất tổng thể và tránh lãng phí chu kỳ CPU.

3.3.8 Thiết kế gói tin tầng ứng dụng

Đối với hệ thống Zonal ECU trong đề tài, gói tin truyền thông ZECU Frame được thiết kế chạy trên nền UDP. Cấu trúc gói tin kế thừa cơ chế hướng dịch vụ từ SOME/IP nhưng được tinh chỉnh chuyên biệt nhằm đáp ứng năng lực xử lý thực tế của ESP32 và Raspberry Pi 4.

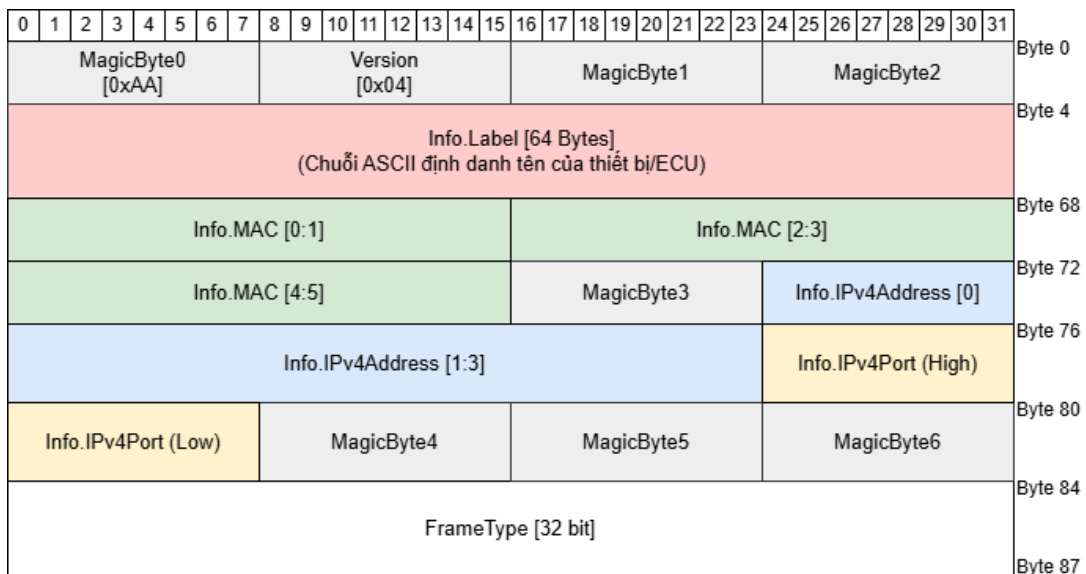
Tất cả các gói tin ZECU khi truyền tải đều có kích thước cố định là 612 byte. Phương pháp này giúp hệ thống quản lý cấp phát bộ đệm (Buffer Allocation) tại Socket tối ưu hơn, triệt tiêu hiện tượng phân mảnh (Fragmentation) ở tầng IP và đẩy nhanh tốc độ xử lý ngắt. Một ZECU Frame hoàn chỉnh bao gồm 3 phần tuần tự.



Hình 3.18 Sơ đồ cấu trúc phân tầng tổng quát của một Generic ZECU.

3.3.8.1 Thiết kế phần mở đầu ZECU Header

Phần mở đầu (Header) chiếm 84 bytes đầu tiên của các gói tin. Phần đầu chứa thông tin định danh của thiết bị gửi, giúp thiết bị nhận đối chiếu thông tin trong mỗi lần giao tiếp. Hình bên dưới mô tả phần mở đầu của gói tin Ethernet.



Hình 3.19 Sơ đồ cấu trúc phân bổ bit của ZECU Frame Header.

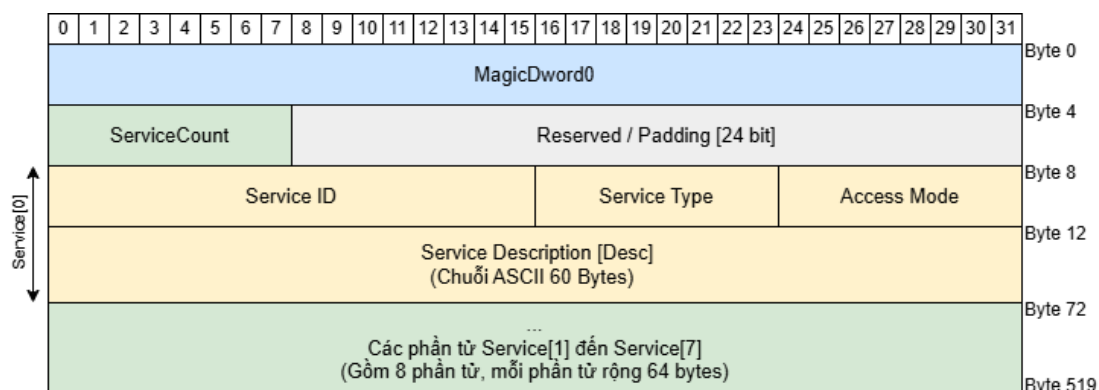
- MagicByte0, 1, 2 (8 bit mỗi trường / Byte 0, Byte 2, Byte 3): Chứa các byte magic định danh cố định (MagicByte0 = 0xAAU). Đây là bộ lọc phân cứng sơ cấp. Khi vi điều khiển ESP32 bóc tách gói tin từ Dịch vụ W5500, nếu 3 byte này không khớp giá trị mặc định, hệ thống sẽ lập tức từ chối xử lý, giảm tải cho CPU khỏi các gói tin rác trên mạng.

- Version (8 bit / Byte 1): Xác định phiên bản cấu trúc khung của đề tài, hiện tại cấu hình cố định ở phiên bản 0x04U.
- Info.Label (64 bytes / Byte 4 - 67): Chuỗi ký tự ASCII lưu trữ tên định danh vật lý của ECU gửi (ví dụ: "ZONAL_ECU_FRONT_LEFT"). Trường này giúp người lập trình dễ dàng giám sát, chẩn đoán lỗi (Diagnostic Logging) khi quét gói tin bằng phần mềm Wireshark mà không cần tra cứu danh mục mã số.
- Info.MAC (6 bytes / Byte 68 - 73): Lưu trữ địa chỉ MAC vật lý (48-bit) của nút mạng gốc. Nó hỗ trợ tăng ứng dụng kiểm tra chéo (Cross-verification) với địa chỉ MAC nhận được ở tầng liên kết dữ liệu nhằm phát hiện các cuộc tấn công giả mạo thiết bị (MAC Spoofing).
- Info.IPv4Address (4 bytes / Byte 75 - 78) & Info.IPv4Port (16 bit / Byte 79 - 80): Lưu trữ địa chỉ IP nguồn và cổng dịch vụ UDP của thiết bị gửi. Việc nhét các thông tin này vào Header ứng dụng giúp Node nhận có thể lập tức đóng gói khung phản hồi ngược lại (Response/Ack) một cách nhanh chóng mà không cần tốn tài nguyên bộ nhớ để duy trì một bảng trạng thái kết nối (Connection State Table).
- MagicByte3, 4, 5, 6 (8 bit mỗi trường): Các byte magic đan xen đóng vai trò làm các vạch phân định ranh giới bộ nhớ (Memory Alignment Markers), đảm bảo cấu trúc struct khi ép chặt (packed) không bị lệch ô nhớ trên các kiến trúc CPU khác nhau.
- FrameType (32 bit / Byte 84 - 87): Đóng vai trò quyết định, tương đương với sự hợp nhất giữa Message ID và Message Type của chuẩn SOME/IP. Nó chứa mã định danh loại khung tin (ví dụ: 0xFFFFFFFF cho Broadcast/Discovery, 0xFF00FF0A cho HeartBeat, 0xFF00CC0C cho EngineControl3). Dựa vào giá trị này, chương trình sẽ nhảy vào cấu trúc lệnh switch-case tương ứng để ép kiểu vùng Body phía sau sang đúng cấu trúc dữ liệu cần đọc.

3.3.8.2 Thiết kế phần thân Body Payload

Tùy thuộc vào giá trị của trường điều hướng FrameType được định nghĩa ở phân vùng Header, cấu trúc vùng đệm dữ liệu Body Payload (kích thước cố định

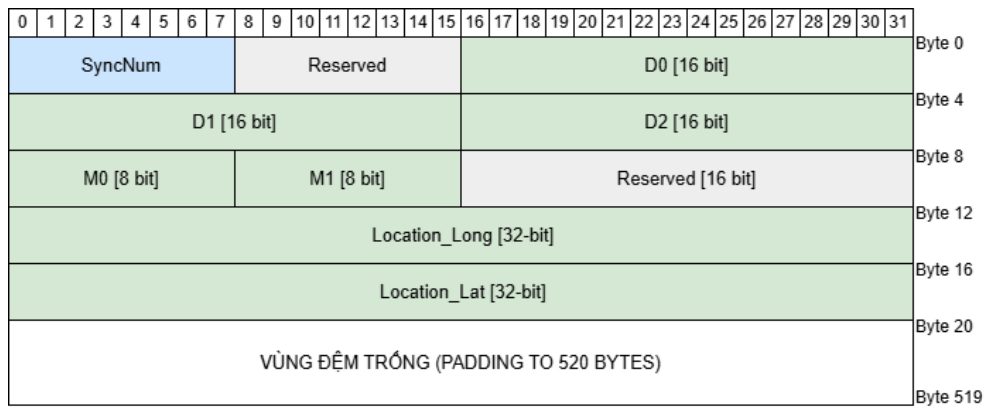
520 Bytes, kéo dài từ Byte thứ 88 đến Byte thứ 607 của gói tin tổng thể) sẽ được thay đổi.



Hình 3.20 Gói tin quảng bá dịch vụ.

Gói tin quảng bá được gửi định kỳ hoặc khi có sự kiện thay đổi cấu hình mạng, cần thiết lập lại kết nối với CCU. Thiết kế khung truyền này đảm nhận nhiệm vụ cốt lõi là quảng bá danh sách các dịch vụ sẵn có của một nút:

- MagicDWord0 (32 bit / Byte 88 - 91): Mã định danh từ kép ma thuật (Double Word) dùng để xác thực tầng ứng dụng sơ cấp, đảm bảo phân vùng Body này thuộc đúng lớp cấu trúc thông điệp khám phá dịch vụ.
- ServiceCount (8 bit / Byte 92): Biến đếm số lượng dịch vụ thực tế đang được kích hoạt và truyền tải trong gói tin (Tối đa là 8 dịch vụ).
- Reserved / Padding (24 bit / Byte 93 - 95): Vùng đệm trống được trình biên dịch tự động chèn vào nhằm thực hiện cơ chế căn lề bộ nhớ chẵn Word (32-bit Alignment), chuẩn bị không gian vùng nhớ phẳng cho mảng dữ liệu kế tiếp.
- Mảng Services (512 Bytes / Kéo dài từ Byte 96 đến Byte 607): Phân vùng nhớ phẳng lớn nhất, chứa danh sách tuyến tính gồm 8 phần tử dịch vụ cố định (mỗi phần tử rộng chính xác 64 Bytes). Nhằm tối ưu hóa việc bóc tách dữ liệu mẫu, thiết kế khung truyền chi tiết của phần tử đầu tiên Service[0].



Hình 3.21 Phần thân gói tin đồng bộ trạng thái.

Đây là khung truyền thông cấu hình phát liên tục (Cyclic Message) với chu kỳ ngắn cao (ví dụ: 10ms/20ms) để liên tục cập nhật trạng thái động học, tọa độ GPS và các tham số điều khiển cơ bản của xe về bộ xử lý trung tâm:

- SyncNum (8 bit / Byte 0): Số thứ tự đồng bộ hóa luồng dữ liệu, tự động tăng tiến liên tục sau mỗi chu kỳ truyền nhằm giúp hệ thống kiểm soát tính tuần tự thời gian thực, phát hiện sớm hiện tượng lặp gói hoặc trễ gói của các lệnh điều khiển cơ cấu chấp hành.
- Reserved (8 bit / Byte 1): Ô nhớ dự phòng rộng 1 byte dùng để căn lề bộ nhớ, đẩy trường dữ liệu cảm biến D0 về ranh giới 16-bit tiếp theo.
- D0 (16 bit / Byte 2 - 3): Lưu trữ giá trị số nguyên 16-bit thô nhận về từ cảm biến khoảng cách siêu âm (ví dụ: Cảm biến HC-SR04 vùng Front-Left).
- D1 & D2 (16 bit mỗi trường / Kéo dài từ Byte 4 đến Byte 7): Cặp trường dữ liệu cảm biến tiếp theo, được xếp song song trọn vẹn trong hàng 32-bit thứ hai của Body để tối ưu hóa tốc độ đọc ghi của CPU.
- M0 & M1 (8 bit có dấu - int8_t / Byte 8 và Byte 9): Chứa giá trị ra lệnh mô-men xoắn hoặc tốc độ vòng quay cho động cơ bên trái (M0) và động cơ bên phải (M1). Do sử dụng kiểu dữ liệu có dấu, dấu của giá trị quyết định chiều quay của bánh xe (giá trị dương ứng với chuyển động tiến, giá trị âm ứng với chuyển động lùi).
- Reserved (16 bit / Byte 10 - 11): Vùng bộ nhớ đệm dự phòng cuối hàng nhằm đảm bảo trường dữ liệu tọa độ 32-bit tiếp theo không bị bẻ đôi qua hai hàng ô nhớ.

- Location_Long (32 bit / Kéo dài từ Byte 12 đến Byte 15): Ô nhớ chiếm trọn một hàng ngang, lưu trữ dữ liệu Kinh độ thô nhận về từ Dịch vụ định vị toàn cầu GPS dưới dạng số nguyên đã qua mã hóa nhân chuỗi.
- Location_Lat (32 bit / Kéo dài từ Byte 16 đến Byte 19): Ô nhớ chiếm trọn một hàng ngang, lưu trữ dữ liệu Vĩ độ thô của thiết bị. Cặp dữ liệu Location_Long và Location_Lat sau khi gửi về máy tính trung tâm sẽ được giải mã để đẩy thẳng vào thuật toán định vị và dẫn đường trên nền tảng hệ điều hành Robot (ROS 2).
- VÙNG ĐỆM TRỐNG - PADDING (Kéo dài từ Byte 20 đến Byte 519): Toàn bộ các byte trống còn lại trong phân đoạn 520 bytes của Body sẽ tự động được trình biên dịch lấp đầy bằng các ký tự không mang dữ liệu (0x00U). Việc duy trì vùng đệm này đảm bảo gói tin luôn đạt kích thước cố định như thiết kế, giúp cấu trúc không bị thay đổi động, tăng tính định thời thời gian thực cho hệ thống mạng nhúng.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	
Magic0								SyncCtrlNum								Magic1								M0								Byte [3:0]
Magic2								Magic3								Checksum16																Byte [7:4]

a) Gói tin điều khiển động cơ trái.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	
Magic0								SyncCtrlNum								Magic1								M0								Byte [3:0]
Magic2								Magic3								Checksum16																Byte [7:4]

b) Gói tin điều khiển động cơ phải.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	
Magic0								SyncCtrlNum								Magic1								M0								Byte [3:0]
Magic2								M1								Checksum16																Byte [7:4]

c) Gói tin điều khiển động cơ trái/phải.

Hình 3.22 Gói tin điều khiển động cơ.

Gói tin điều khiển động cơ hoạt động theo cơ chế hướng sự kiện, chỉ được phát đi khi CCU cần thay đổi trạng thái vận hành của động cơ như tăng tốc, dừng truyền động hoặc đảo chiều. Cấu trúc lõi của gói tin bao gồm các trường dữ liệu quan trọng sau:

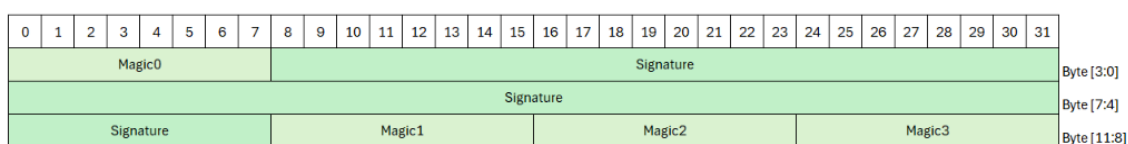
- CheckSum16: Mã tổng kiểm lỗi cục bộ (Local Checksum) được tính toán riêng cho lớp ứng dụng điều khiển. Vi điều khiển đích sẽ dùng mã này

để xác thực tính toàn vẹn của lệnh điều khiển động cơ trước khi cho phép dòng điện lớn đi vào driver phân cứng.

- M0 & M1: Lệnh điều khiển độ rộng xung PWM hoặc giá trị Moment xoắn mục tiêu truyền xuống động cơ Trái (Motor L) và động cơ Phải (Motor R).



a) Gói tin xác thực AuthTX.

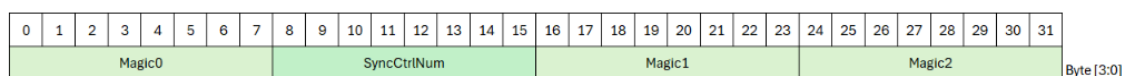


b) Gói tin xác thực AuthRX.

Hình 3.23 Phân thân gói tin xác thực bảo mật - AuthTX / AuthRX.

Gói tin xác thực bảo mật bao gồm hai loại theo cơ chế bắt tay thử thách – phản hồi (Challenge-Response Handshake) giữa Zonal ECU và CCU, ngăn chặn các cuộc tấn công giả mạo lệnh điều khiển trên mạng Ethernet ô tô:

- Challenge (8 Byte): Chuỗi số ngẫu nhiên (Cryptographic Nonce) được tạo ra dựa trên một khóa chung (Shared Key), nhằm mục đích thách thức và xác thực danh tính của thực thể đối tiếp trong phiên giao tiếp..
- Signature (8 Byte): Chữ ký số được tạo từ việc băm chuỗi Challenge cùng khóa bí mật (Secret Key). Các ECU chỉ chính thức thiết lập kết nối điều khiển khi chữ ký này được giải mã và xác thực trùng khớp.

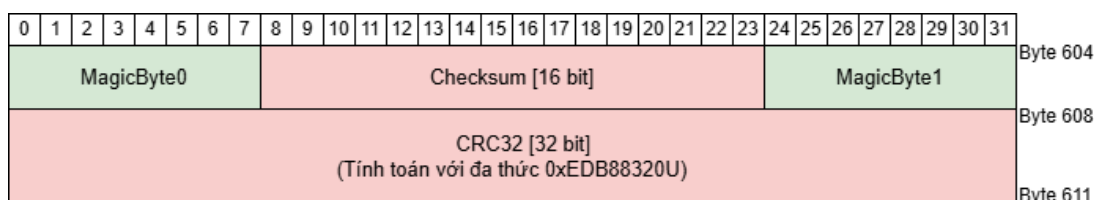


Hình 3.24 Gói tin giữ kết nối.

Khác với gói tin đồng bộ, ZECU gửi thông tin trạng thái và cảm biến đến CCU, gói tin giữ kết nối (keep alive) được CCU gửi định kỳ xuống ZECU để giữ kết nối, sau khoảng thời gian nhất định, ZECU không thấy gói tin, ZECU sẽ tiến hành dừng truyền động lực cho động cơ và chuyển trạng thái quảng bá để ghép nối lại với CCU. Hình bên dưới mô tả gói tin giữ kết nối.

3.3.8.3 Thiết kế phần cuối ZECU Trailer

Phần dữ liệu của gói tin được kết bởi ZECU Trailer. Đây là phân vùng đóng vai trò cốt lõi trong việc hiện thực hóa cơ chế bảo vệ an toàn dữ liệu từ đầu chí cuối (End-to-End - E2E), giúp hệ thống chủ động phát hiện và triệt tiêu các rủi ro sai lệch lệnh điều khiển do hiện tượng lật bit do nhiễu ngẫu nhiên hoặc nhiễu cụm trên đường truyền vật lý:



Hình 3.25 Phần kết thúc chung của gói tin - ZECU Trailer.

- MagicByte0 (8 bit / Byte 604): Byte ma thuật mở đầu phân đoạn Trailer (được cấu hình giá trị cố định). Trường này hoạt động như một vạch ranh giới bộ nhớ (Marker), báo hiệu cho bộ giải mã tầng ứng dụng biết phân vùng dữ liệu động Body Payload đã kết thúc để chuẩn bị chuyển sang các tầng lọc cấu trúc an toàn hệ thống.
- CheckSum (16 bit / Byte 605 - 606): Giá trị tổng kiểm bù 16-bit được tính toán nhanh trên toàn bộ cấu trúc dữ liệu trước đó. Trường này cung cấp một cơ chế xác thực sơ cấp tốc độ cao (Fast-path validation), cho phép CPU của vi điều khiển (ESP32) lập tức phát hiện và loại bỏ các gói tin bị lỗi cấu trúc nặng hoặc bị méo dạng khung truyền mà không cần tiêu tốn nhiều chu kỳ xung nhịp để chạy các thuật toán băm phức tạp hơn.
- MagicByte1 (8 bit / Byte 607): Byte ma thuật kết thúc hàng máy 32-bit đầu tiên của phân vùng Trailer, phối hợp với MagicByte0 để tạo thành một cặp dấu mốc đóng khung bảo vệ cố định cho trường CheckSum.
- CRC32 (32 bit / Kéo dài từ Byte 608 đến Byte 611): Chiếm trọn vẹn không gian của hàng máy 32-bit cuối cùng trong gói tin PDU. Đây là mã kiểm tra vòng bít chu kỳ ứng dụng thuật toán tiêu chuẩn mã hóa với đa thức cố định 0xEDB88320U. Trường này mang lại khả năng phát hiện sai lệch cực mạnh

(lên tới 99.99% lỗi bit vật lý), đảm bảo dữ liệu điều khiển phần cứng của xe đạt độ tin cậy tuyệt đối trước khi thực thi.

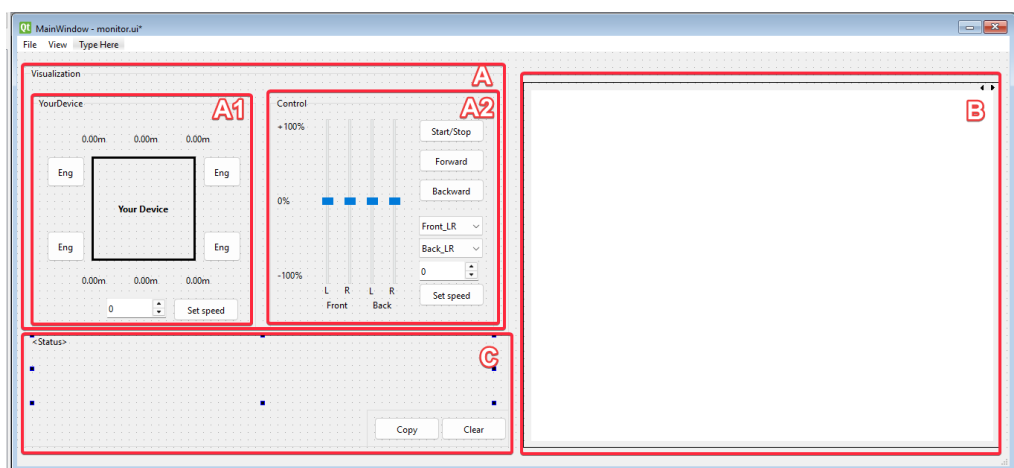
Cơ chế vận hành bảo vệ E2E thực tế: Khi hệ thống tiếp nhận thành công một gói tin dữ liệu đầy đủ qua Socket mạng UDP, hàm xử lý ngắt hệ thống `App_VerifyPacket()` sẽ được gọi ngay lập tức. Tiến trình sẽ thực hiện quét toàn bộ vùng nhớ phẳng của gói tin bắt đầu từ vị trí `MagicByte0` của Header cho tới hết byte cuối cùng của vùng Padding trong Body để tính toán lại giá trị CRC32 thực tế.

Nếu kết quả tính toán trùng khớp hoàn toàn với giá trị lưu trữ trong trường CRC32 ở Trailer, gói tin mới được xác nhận đạt độ tin cậy tuyệt đối và chuyển tiếp lên tầng logic ứng dụng để thực thi cơ cấu chấp hành phần cứng. Trong trường hợp xảy ra sai lệch dù chỉ 1 bit dữ liệu, gói tin lập tức bị hủy bỏ (Drop Packet) nhằm triệt tiêu hoàn toàn nguy cơ mất an toàn vận hành hệ thống xe.

3.3.9 Thiết kế ứng dụng điều khiển

3.3.9.1 Giao diện chính

Ứng dụng điều khiển có vai trò chuẩn đoán, điều khiển với các ZECU trong mạng thông qua việc kết nối với mạng cục bộ bên trong xe. Với các chức năng tương tự CCU nhưng có giao diện người dùng. Hình bên dưới minh họa giao diện tổng thể của ứng dụng điều khiển.

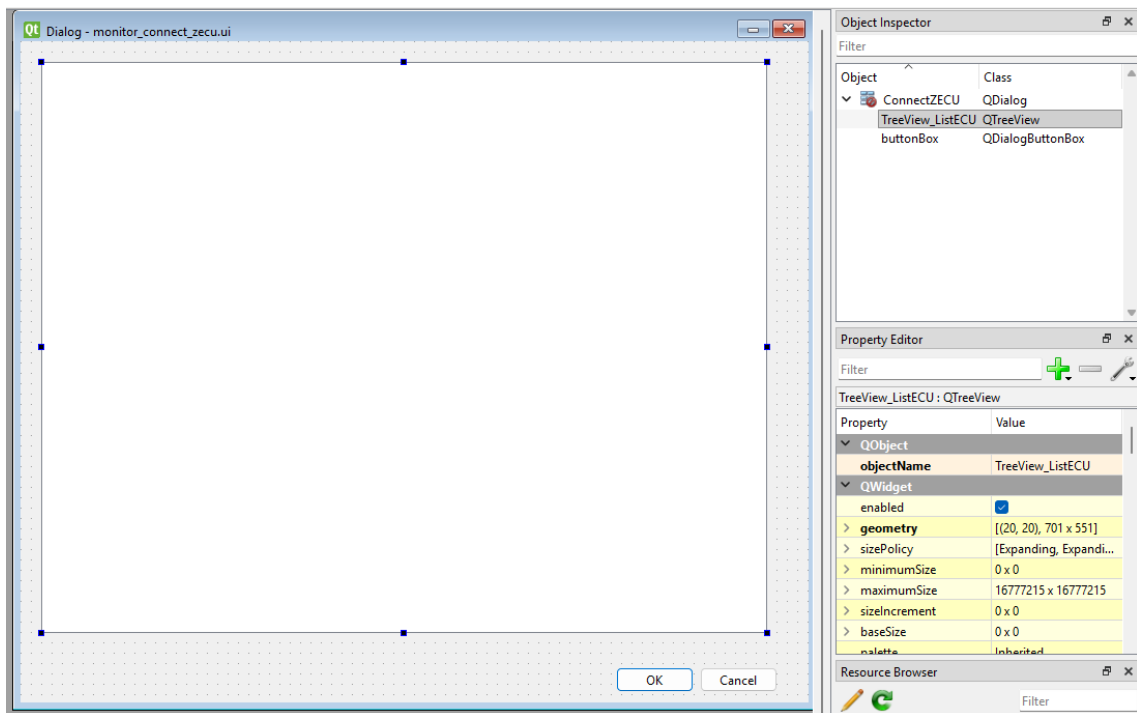


Hình 3.26 Giao diện tổng thể của ứng dụng điều khiển.

Ứng dụng điều khiển được chia làm 03 nhóm gồm nhóm minh hoạ - A, A1, A2, cung cấp thông tin tin về trạng thái của các ZECU, các nút điều khiển động cơ theo các cách khác nhau; Nhóm đồ thị - B bao gồm các đồ thị về khoảng cách, tốc độ cũng như thông tin về chu kỳ của các khung truyền đồng bộ (HeatBeat) được ZECU gửi về.

3.3.9.2 Giao diện ghép nối ZECU

Ứng dụng cũng hỗ trợ giao diện ghép nối ZECU. Các gói tin quản bá của ZECU trong chế độ ghép nối sẽ được phân giải và hiển thị danh dạng bảng các dịch vụ sẵn có. Người dùng có thể xem và tiến hành ghép nối với ZECU tương ứng. Hình bên dưới mô tả thiết kế giao diện ghép nối.



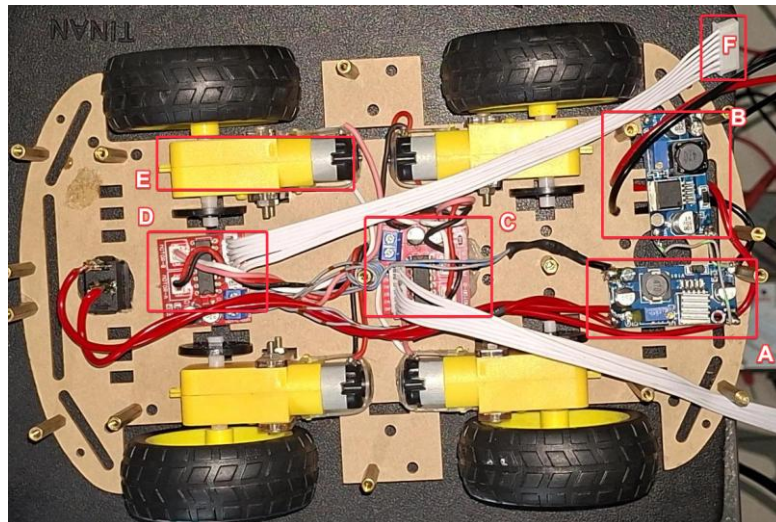
Hình 3.27 Giao diện ghép nối ZECU.

CHƯƠNG 4

THI CÔNG HỆ THỐNG

4.1 THI CÔNG KHỐI NGUỒN VÀ ĐỘNG CƠ

Mô hình xe được chia thành nhiều tầng, trong đó tầng dưới cùng bao gồm 04 động cơ, được điều khiển bởi 02 mạch cầu H. Khối nguồn (gồm 02 mạch hạ áp) cũng được đặt ở tầng này để đảm bảo không gian. Hình 4.1 minh họa tầng cuối cùng với chú thích trong Bảng 4.1.



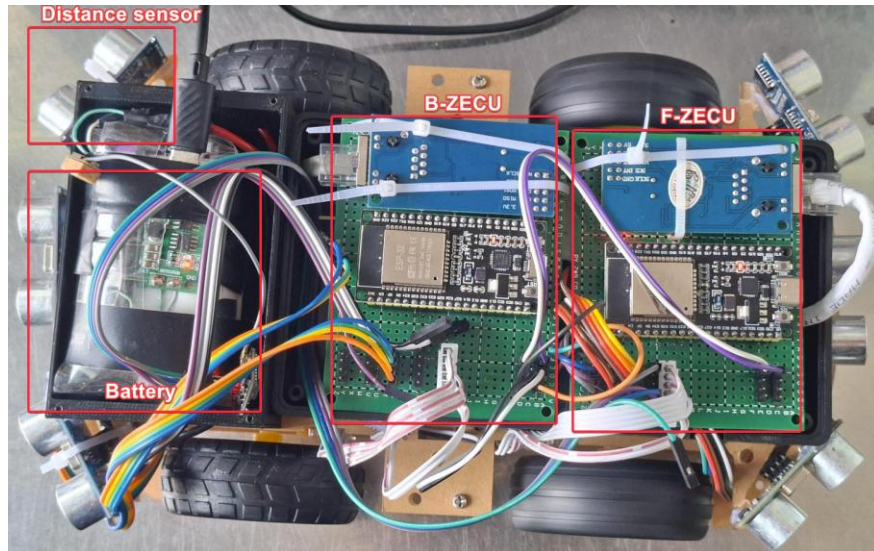
Hình 4.1 Tầng dưới cùng của mô hình.

Bảng 4.1 Chú thích các thành phần tầng dưới cùng của xe.

TT	Chú thích
A	Mạch hạ áp cho động cơ; Ghi chú: Điện áp đầu ra: 9V; Dòng tối đa 3A.
B	Mạch hạ áp cho các thiết bị điều khiển (F-ZECU, B-ZECU, CS). Ghi chú: Điện áp đầu ra: 9V; Dòng tối đa 3A
C	Mạch cầu H cho hai bánh trước
D	cầu H cho hai bánh sau

E	Động cơ sau phía bên phải
F	Đầu kết nối với khối điều khiển vùng

4.2 THI CÔNG MÔ HÌNH TỔNG THỂ



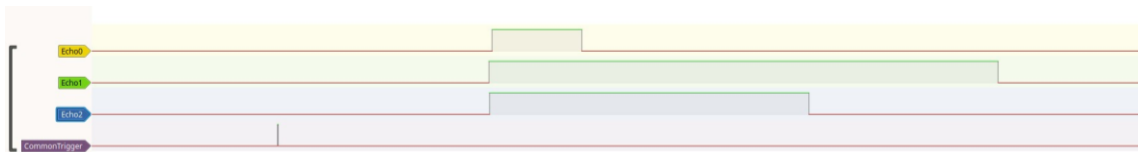
Hình 4.2 Mô hình xe hoàn chỉnh.

CHƯƠNG 5

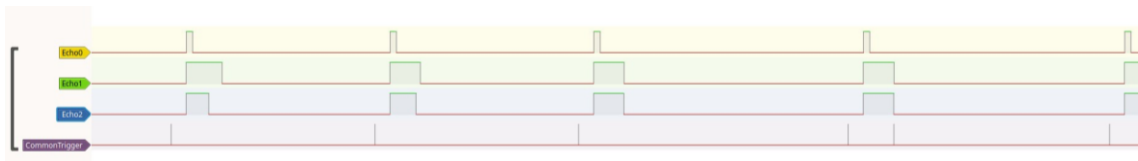
KẾT QUẢ VÀ ĐÁNH GIÁ HỆ THỐNG

5.1 KẾT QUẢ KHỞI CẢM BIẾN

Đánh giá độc lập hoạt động khối cảm biến như Hình 5.1 cho thấy quá trình hoạt động của khối cảm biến ổn định và chính xác như thiết kế. Không xuất hiện tình trạng nhiễu.



a) Dạng sóng một chu kỳ lấy mẫu.



b) Dạng sóng nhiều chu kỳ lấy mẫu.

Hình 5.1 Dạng sóng trên đường truyền giao tiếp với khối cảm biến.

5.2 KẾT QUẢ KHỞI ĐỘNG CƠ

Kiểm thử độc lập hoạt động của khối điều chế độ rộng xung (PWM) cho mạch cầu H trong khối động cơ, kết quả được thể hiện trong Hình 5.2 Độ rộng xung thay đổi đúng theo thiết kế, tuy nhiên do điều khiển tuần tự nên có độ lệch giữa các kênh PWM.



Hình 5.2 Dạng sóng ngõ ra bộ điều chế độ rộng xung điều khiển mạch cầu H.

5.3 KẾT QUẢ ỨNG DỤNG GIÁM SÁT

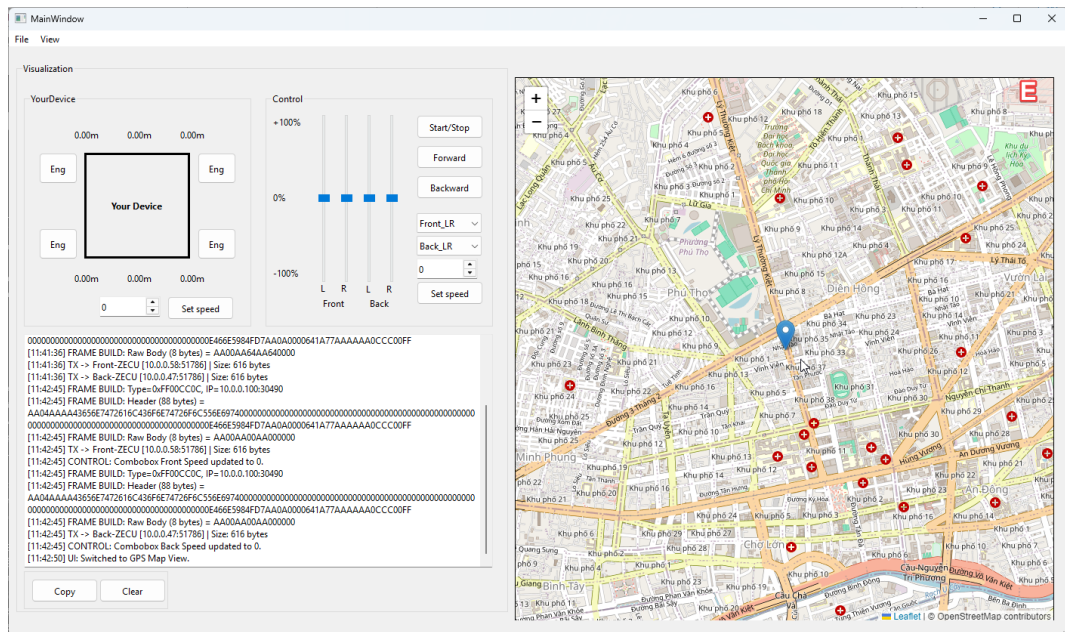
5.3.1 Tổng quan

Ứng dụng giám sát hoạt động như thiết kế có thể nhận, phân giải các gói tin đồng bộ (heartbeat) từ các ZECU, có khả năng điều khiển độc lập 04 động cơ. Hình 5.3, Hình 5.4 minh họa ứng dụng điều khiển/giám sát. Trong biểu đồ B, Các đường nét liền dạng răng cưa cho thấy các gói tin được gửi liên tục và tuần tự. Bên cạnh đó, khi có tín hiệu điều khiển, ZECU ngay lập tức gửi lại gói tin cập nhật, làm cho thứ tự gói tin trong bộ đếm xoay vòng bị thay đổi. Đường nét đứt ngang cho thấy chu kỳ đồng bộ của các gói tin này. Chu kỳ đồng bộ được thiết kế là 300ms.

Khung trạng thái A cho biết thông tin điều khiển của ứng dụng được gửi đến ZECU. Các biểu đồ C, D, E cho biết trạng thái hiện tại của ZECU.



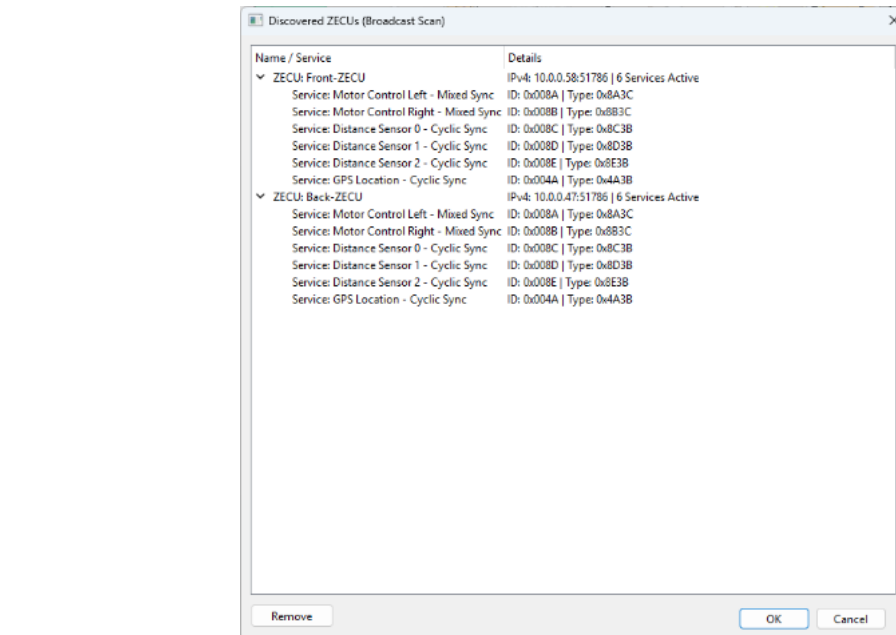
Hình 5.3 Giao diện chính với đồ thị.



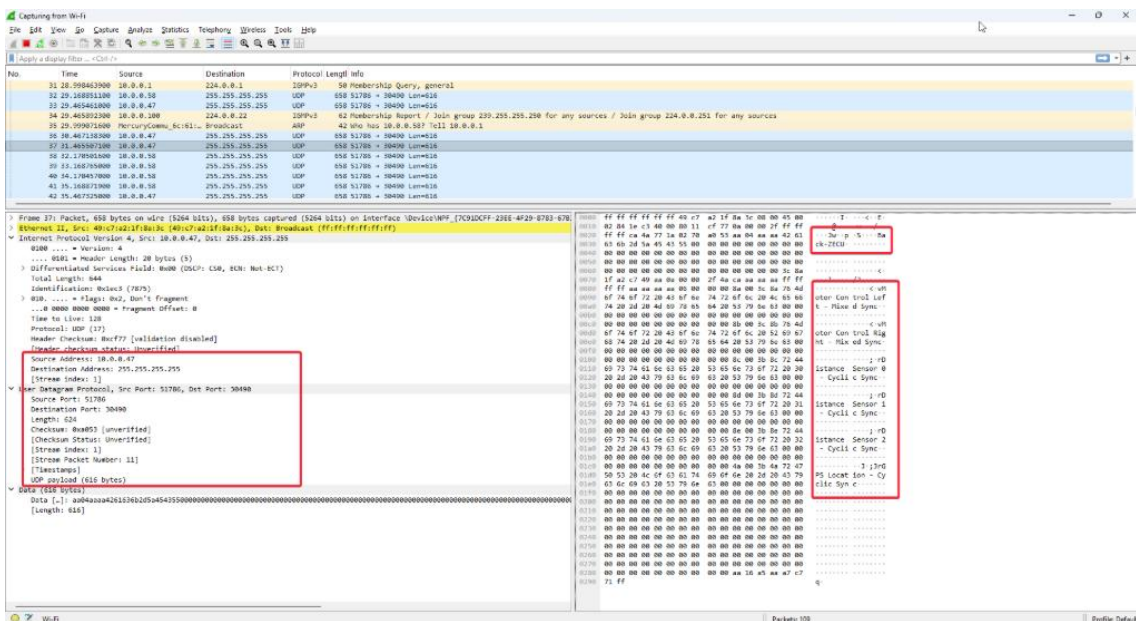
Hình 5.4 Giao diện chính với bản đồ.

5.3.2 Quá trình ghép nối

Quá trình ghép nối bắt đầu khi nhấn nút BOOT trên ZECU. Lúc này ZECU sẽ liên tục gửi các gói tin quảng bá dịch vụ của mình. Hình 5.5 minh họa kết quả sau khi phân giải các gói tin quảng bá của ZECU trước và sau. Hình 5.6 mô tả gói tin quảng bá được bắt bởi WireShark.



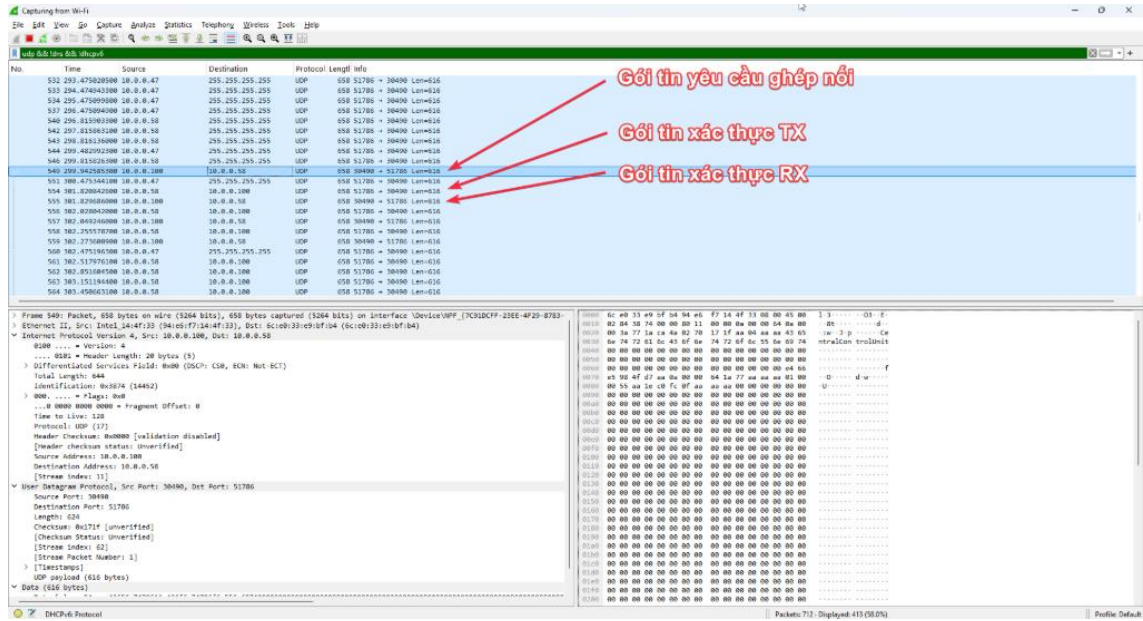
Hình 5.5 Giao diện kết nối ZECU của ứng dụng giám sát.



Hình 5.6 Gói tin quảng bá của Back-ZECU.

Hình 5.7 mô tả quá trình ghép nối F-ZECU. Quá trình bắt đầu khi ứng dụng giám sát (địa chỉ IP: 10.0.0.100) gửi gói tin yêu cầu ghép nối. Sau đó F-ZECU (địa chỉ IP: 10.0.0.58) sẽ gửi lại gói tin xác thực TX, F-ZECU chuyển từ gửi quảng bá ở địa chỉ 255.255.255.255 sang địa chỉ của ứng dụng giám sát. Sau đó ứng dụng giám sát sẽ tiến hành thêm “chữ ký” và gửi lại gói tin xác thực RX. F-ZECU xác

thực thành công tiến hành gửi các gói tin đồng bộ (HeartBeat) cho ứng dụng giám sát với chu kỳ 300ms.



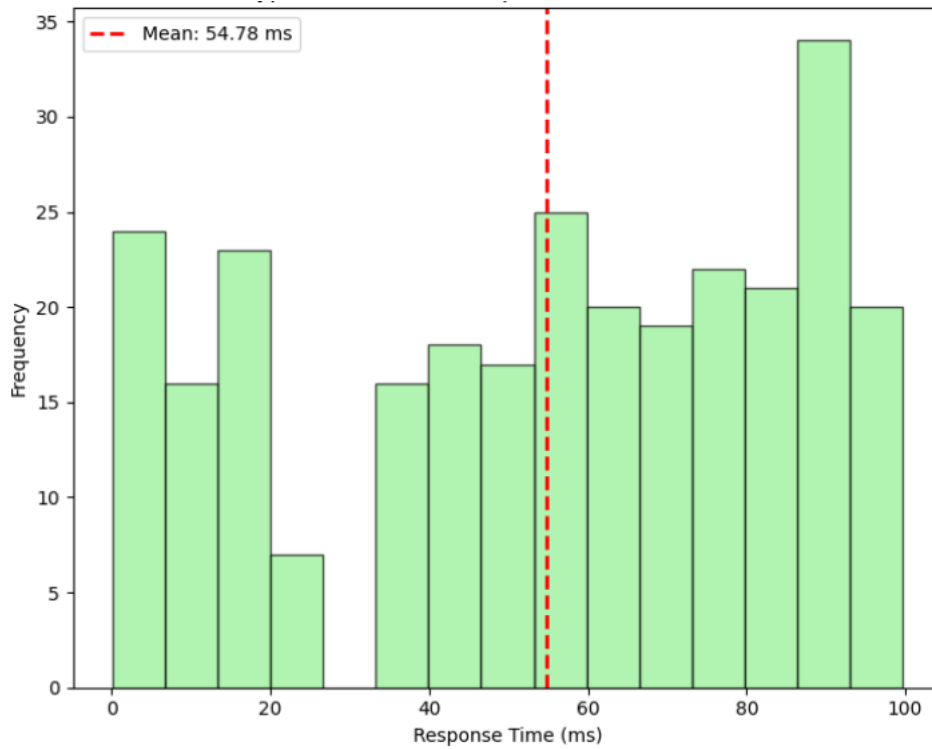
Hình 5.7 Quá trình ghép nối F-ZECU.

5.4 THỜI GIAN ĐÁP ỨNG CỦA HỆ THỐNG

Thời gian đáp ứng của hệ thống được đo từ lúc gửi gói tin điều khiển đến ZECU và gói tin phản hồi từ ZECU quay về, công thức bên dưới mô tả các thành phần của thời gian đáp ứng:

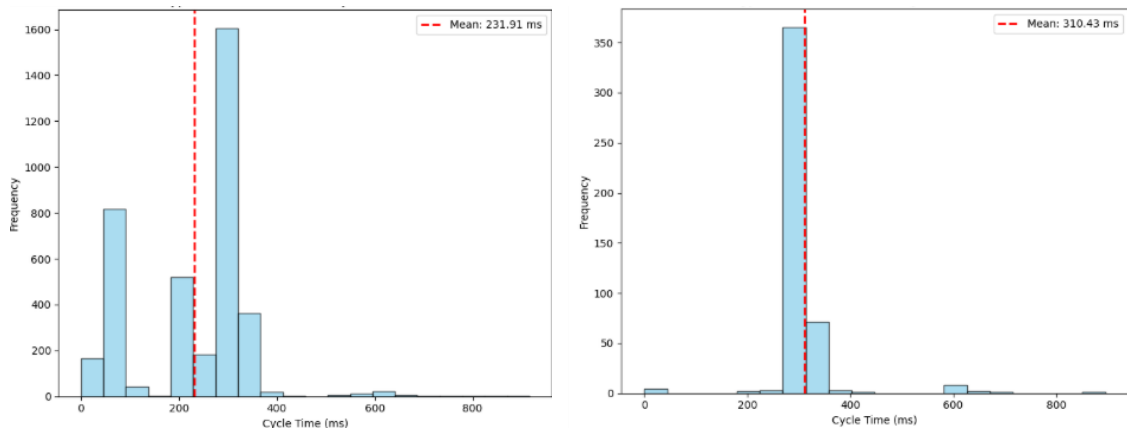
$$t_{\text{đáp ứng}} = t_{\text{gửi lệnh}} + t_{\text{xử lý}} + t_{\text{phản hồi}} \quad (5.1)$$

Sử dụng Wireshark để thu thập các gói tin và thông kê thu được biểu đồ Hình 5.8 Biểu đồ cho thấy thời gian đáp ứng trung bình là 55ms, và trễ nhất lên đến 100ms.



Hình 5.8 Tần suất thời gian phản hồi.

Ngoài ra, để đo lường tính ổn định của cái gói tin đồng bộ, một chương trình Python sẽ phân tích các gói tin tin mà WireSark thu thập để lập biểu đồ thống kê chu kỳ của các gói tin. Hình 5.9 mô tả tính ổn định của hệ thống.



a) Khi có các gói tin điều khiển (chu kỳ 1s) b) Khi không có gói tin điều khiển.

Hình 5.9 Tần suất các chu kỳ gói tin đồng bộ.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 KẾT LUẬN

Sau quá trình nghiên cứu, thiết kế và triển khai, đề tài đã xây dựng thành công mô hình hệ thống điều khiển phân tán theo vùng trên ô tô sử dụng giao thức hướng dịch vụ qua kết nối Ethernet. Hệ thống được phát triển dựa trên định hướng kiến trúc vùng (Zonal Architecture) và kiến trúc hướng dịch vụ (Service-Oriented Architecture – SOA), là những xu hướng đang được áp dụng rộng rãi trong quá trình chuyển đổi sang mô hình xe được định nghĩa bằng phần mềm (Software Defined Vehicle – SDV).

Về mặt phần cứng, hệ thống đã triển khai thành công hai bộ điều khiển vùng (ZECU) sử dụng vi điều khiển ESP32 kết hợp với bộ điều khiển Ethernet W5500. Các ZECU đảm nhiệm chức năng thu thập dữ liệu từ cảm biến khoảng cách siêu âm HC-SR04, xử lý dữ liệu tại biên và điều khiển cơ cấu chấp hành thông qua mô-đun cầu H. Bên cạnh đó, hệ thống nguồn sử dụng bộ chuyển đổi DC-DC LM2596S-ADJ để cung cấp các mức điện áp phù hợp cho từng thiết bị, đảm bảo quá trình vận hành ổn định của toàn bộ mô hình.

Về mặt phần mềm, đề tài đã xây dựng thành công kiến trúc phần mềm phân tầng trên nền tảng ESP-IDF và FreeRTOS. Các dịch vụ chức năng được tổ chức theo mô hình độc lập, bao gồm dịch vụ giao tiếp Ethernet, dịch vụ điều khiển động cơ, dịch vụ giám sát cảm biến và dịch vụ ghép nối với hệ thống điều khiển trung tâm. Việc sử dụng cơ chế hàng đợi, khóa đồng bộ và quản lý tác vụ của FreeRTOS giúp hệ thống có khả năng xử lý đồng thời nhiều tác vụ trong thời gian thực, đồng thời nâng cao tính ổn định khi vận hành.

Đề tài cũng đã xây dựng thành công giao thức truyền thông dựa trên Ethernet kết hợp với SOME/IP nhằm thực hiện trao đổi dữ liệu giữa các nút mạng trong hệ

thống. Kết quả thực nghiệm cho thấy các ZECU có khả năng truyền nhận dữ liệu cảm biến, nhận lệnh điều khiển và phản hồi trạng thái hoạt động thông qua hạ tầng mạng Ethernet với độ ổn định cao. Ứng dụng giám sát được phát triển bằng PySide6 cho phép theo dõi trạng thái hệ thống theo thời gian thực, thực hiện ghép nối các ZECU và hỗ trợ điều khiển các cơ cấu chấp hành từ giao diện người dùng.

Kết quả thử nghiệm cho thấy mô hình có khả năng trao đổi dữ liệu ổn định giữa các ZECU thông qua mạng Ethernet và thực hiện điều khiển cơ cấu chấp hành theo dữ liệu cảm biến nhận được. Điều này cho thấy kiến trúc vùng kết hợp với SOME/IP có thể được áp dụng trong các hệ thống điều khiển phân tán trên ô tô. Mô hình đáp ứng các chức năng đã đề ra trong phạm vi đồ án, đồng thời có thể được sử dụng làm nền tảng thử nghiệm cho các nghiên cứu tiếp theo liên quan đến Ethernet ô tô, SOME/IP và kiến trúc vùng. Kết quả thực hiện cho thấy việc phân tách chức năng giữa các bộ điều khiển vùng và bộ điều khiển trung tâm có thể góp phần giảm tải xử lý cục bộ và đơn giản hóa việc quản lý truyền thông trong hệ thống.

Mặc dù đã đạt được các mục tiêu đề ra, hệ thống vẫn còn một số hạn chế như quy mô thử nghiệm còn nhỏ, số lượng bộ điều khiển vùng còn hạn chế và chưa triển khai đầy đủ các cơ chế chuẩn của SOME/IP Service Discovery theo AUTOSAR. Ngoài ra, trong phạm vi đồ án, chức năng CCU được hiện thực dưới dạng ứng dụng giám sát chạy trên máy tính. Hệ thống chưa triển khai một bộ điều khiển trung tâm chuyên dụng có khả năng xử lý tập trung như các nền tảng SDV thực tế.

6.2 HƯỚNG PHÁT TRIỂN

Dựa trên kết quả đạt được, hệ thống vẫn còn nhiều nội dung có thể tiếp tục nghiên cứu và mở rộng trong thời gian tới. Trước hết, cần mở rộng quy mô hệ thống bằng cách tăng số lượng bộ điều khiển vùng và bổ sung thêm nhiều loại cảm biến cũng như cơ cấu chấp hành khác nhau. Việc mở rộng này sẽ giúp mô hình tiệm cận hơn với cấu trúc điện - điện tử của các phương tiện hiện đại, đồng thời cho phép đánh giá khả năng mở rộng của kiến trúc vùng trong các hệ thống có quy mô lớn.

Bên cạnh đó, cần xây dựng bộ điều khiển trung tâm (CCU) chuyên biệt dựa trên nền tảng Raspberry Pi hoặc máy tính nhúng hiệu năng cao. CCU sẽ đảm nhận các chức năng xử lý tập trung, quản lý dịch vụ, điều phối dữ liệu giữa các vùng và thực hiện các thuật toán điều khiển cấp cao. Đây là bước quan trọng để tiến tới mô hình kiến trúc vùng tập trung đang được áp dụng trên các phương tiện SDV hiện nay.

Một hướng phát triển khác là triển khai đầy đủ giao thức SOME/IP và SOME/IP Service Discovery theo tiêu chuẩn AUTOSAR. Việc chuẩn hóa cơ chế khám phá dịch vụ, đăng ký dịch vụ và quản lý phiên kết nối sẽ giúp hệ thống có khả năng tương thích tốt hơn với các nền tảng phần mềm ô tô hiện đại và tạo điều kiện thuận lợi cho việc tích hợp các mô-đun chức năng mới trong tương lai.

Ngoài ra, hệ thống có thể được nâng cấp bằng cách triển khai cơ chế cập nhật phần mềm từ xa (Software Over-The-Air – SOTA). Chức năng này cho phép cập nhật firmware hoặc các mô-đun phần mềm của ZECU thông qua mạng Ethernet mà không cần can thiệp trực tiếp vào phần cứng, phù hợp với định hướng phát triển của các phương tiện được định nghĩa bằng phần mềm.

Cuối cùng, mô hình có thể được tích hợp thêm các công nghệ truyền thông tiên tiến như Automotive Ethernet 100BASE-T1, Time-Sensitive Networking (TSN) hoặc các giao thức bảo mật dành cho hệ thống ô tô. Việc tích hợp các công nghệ này sẽ cho phép đánh giá khả năng đáp ứng thời gian thực, độ tin cậy của mạng truyền thông cũng như khả năng ứng dụng trong các hệ thống điều khiển ô tô có yêu cầu cao hơn.

TÀI LIỆU THAM KHẢO

- [1] P. McLellan, “What Is Zonal Architecture? And Why Is it Upending the Automotive Supply Chain?”, *Cadence Blog*, 2023. [online]. Available: https://community.cadence.com/cadence_blogs_8/b/breakfast-bytes/posts/zonal
- [2] AUTOSAR Foundation, “AUTOSAR SOME/IP Protocol Specification”, *AUTOSAR Standard Specification*, 2024. [Online]. Available: https://www.autosar.org/fileadmin/standards/R22-11/FO/AUTOSAR_PRS_SOMEIPProtocol.pdf
- [3] J. Harper, “Zonal Architecture 101: Reducing Vehicle System Development Complexity,” *onsemi Blog*, 2025. [Online]. Available: <https://www.onsemi.com/company/newsroom/blog/automotive/zonal-architecture-101-reducing-vehicle-system-development-complexity>
- [4] AUTOSAR Foundation, “AUTOSAR SOME/IP Service Discovery” Protocol Specification, *AUTOSAR Standard Specification*, 2024. [Online]. Available: https://www.autosar.org/fileadmin/standards/R22-11/FO/AUTOSAR_PRS_SOMEIPServiceDiscoveryProtocol.pdf
- [5] L. Mauser và cộng sự, “Towards Mixed-Criticality Software Architectures for Centralized HPC Platforms in Software-Defined Vehicles: A Systematic Literature Review,” *arXiv preprint arXiv:2506.05822*, 2025. [Online]. Available: <https://arxiv.org/pdf/2506.05822>
- [6] Reinhardt D. and Kucera M. (2013), “Domain Controlled Architecture- A New Approach for Large Scale Software Integrated Automotive Systems.” In *Proceedings of the 3rd International Conference on Pervasive Embedded Computing and Communication Systems*, pp. 221-226.

- [7] A. T. Vemuri, “Processing the Advantages of Zone Architecture in Automotive,” *Texas Instruments Technical Article*, 2023. [Online]. Available: <https://www.ti.com/document-viewer/lit/html/SSZT211>.
- [8] E. Liotou, D. Tzelalidou and G. Christodoulou, “The Rise of the Software-Defined Vehicle: Architectures, Enabling Technologies, and Future Opportunities,” *arXiv preprint arXiv:2605.30001*, 2026. [Online]. Available: <https://arxiv.org/pdf/2605.30001>
- [9] S. S. Shetiya, V. Vyas and S. Renukuntla, “Zonal Architecture Development with Evolution of Artificial Intelligence,” *arXiv preprint arXiv:2412.01840*, 2024. [Online]. Available: <https://arxiv.org/pdf/2412.01840>
- [10] AUTOSAR Foundation, “Specification on SOME/IP Transport Protocol”, 2024. AUTOSAR Standart Specification, 2021. [Online]. Available: https://www.autosar.org/fileadmin/standards/R21-11/CP/AUTOSAR_SWS_SOMEIPTransportProtocol.pdf